

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN**



BIG DATA ANALYSIS

**PHÂN TÍCH DỮ LIỆU BỆNH NHÂN COVID-19 ỨNG
DỤNG KIẾN TRÚC LAKEHOUSE**

GVHD: ThS. Lê Thị Minh Châu

Mã lớp học: BDAN333977_01

Nhóm sinh viên thực hiện: Nhóm 1

Nguyễn Văn Quang Duy 23110086

Đỗ Kiến Hưng 23133030

Phan Trọng Phú 23133056

Phan Trọng Quý 23133061

Tp. Hồ Chí Minh, tháng 4 năm 2026

LỜI CẢM ƠN

Trước tiên, nhóm báo cáo xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Lê Thị Minh Châu vì đã tận tình hướng dẫn và hỗ trợ nhóm trong suốt quá trình thực hiện đồ án. Nhờ vào sự chỉ dẫn chi tiết, những góp ý quý báu và nguồn cảm hứng mà cô mang lại, nhóm đã có cơ hội học hỏi, phát triển và áp dụng các kiến thức về Big Data, xử lý dữ liệu lớn.

Chúng em cũng xin cảm ơn đến Trường Đại học Công nghệ Kỹ thuật TP.HCM đã tạo điều kiện và môi trường học tập lý tưởng, giúp nhóm có thể hoàn thiện tốt bài đồ án này. Dù đã cố gắng hết sức để thực hiện đề tài, nhưng với hạn chế về thời gian và kinh nghiệm thực tế, nhóm khó tránh khỏi những thiếu sót trong báo cáo. Rất mong nhận được sự thông cảm và góp ý của cô để nhóm có thể hoàn thiện kiến thức và kỹ năng hơn nữa trong tương lai. Một lần nữa, nhóm xin chân thành cảm ơn cô Lê Thị Minh Châu và toàn bộ những người đã hỗ trợ nhóm hoàn thành đồ án này.

BẢNG PHÂN CÔNG

Mã nguồn: <https://github.com/darktheDE/healthcare-lakehouse-covid19>

Mình chứng triển khai công việc:

<https://sites.plane.so/issues/b18b3636a9c44636aea91add184a8ed9>

STT	MSSV	Họ Và Tên	Nhiệm Vụ
1	23110086	Nguyễn Văn Quang Duy	- 1.1.3. Iceberg – Định dạng bảng - 1.1.5 Trino – Công cụ truy vấn SQL phân tán - 1.1.6. Hive Metastore - 1.1.7. Apache Superset - 2.2.2: Làm sạch dữ liệu và chuẩn hóa kiểu dữ liệu - 2.3: Khai thác và trình diễn dữ liệu bằng Trino và Apache Superset
2	23133030	Đỗ Kiến Hưng	- 1.2.1 Tổng quan kiến trúc Medallion - 1.2.2 Thiết kế kiến trúc Decoupled Storage & Compute - 1.2.3 Quy trình dòng chảy dữ liệu - 1.2.4 Giới thiệu tập dữ liệu Synthea COVID-19 - 2.2.3 Gold Layer: Tổng hợp dữ liệu và phân tích các chỉ số COVID-19
3	23133056	Phan Trọng Phú	- 1.1.1 Apache Spark - Xử lý dữ liệu phân tán - 1.1.2 Apache Airflow - Điều phối luồng dữ liệu - 2.1.1 Xây dựng hạ tầng container hóa với Docker compose, - 2.1.5 Thiết lập môi trường lập lịch với Apache Airflow

			- 2.2.3 Gold Layer: Tổng hợp dữ liệu và phân tích các chỉ số COVID-19 - 2.2.4 Tự động hóa quy trình bằng Airflow DAGs
4	23133061	Phan Trọng Quý	- 1.1.4 MinIO - Lưu trữ đối tượng tương thích S3 - 1.1.9 PostgreSQL Hệ quản trị cơ sở dữ liệu nguồn - 2.1.2 Cài đặt và cấu hình PostgreSQL Source Database - 2.1.3 Cấu hình Datalake với MinIO và Hive Metastore - 2.2.1 Ingestion Trích xuất dữ liệu từ PostgreSQL sang lớp Bronze

Nhận xét của giảng viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tháng 04 năm 2026

Giảng viên chấm điểm

MỤC LỤC

CHƯƠNG 1: LÝ THUYẾT VỀ DATA LAKEHOUSE	1
1.1. Công nghệ và công cụ	1
1.1.1 Apache Spark – Xử lý dữ liệu phân tán	1
1.1.2 Apache Airflow – Điều phối luồng dữ liệu	3
1.1.3 Apache Iceberg – Định dạng bảng Lakehouse	5
1.1.4 MinIO – Lưu trữ đối tượng tương thích S3	11
1.1.5 Trino – Công cụ truy vấn SQL phân tán.....	13
1.1.6 Hive Metastore – Quản lý Catalog dữ liệu	19
1.1.7 Apache Superset – Trực quan hóa dữ liệu	21
1.1.8 PostgreSQL – Hệ quản trị cơ sở dữ liệu nguồn	21
1.2. Kiến trúc Medallion Lakehouse	24
1.2.1 Tổng quan kiến trúc Medallion.....	24
1.2.2 Thiết kế kiến trúc Decoupled Storage & Compute.....	25
1.2.3 Quy trình dòng chảy dữ liệu	26
1.2.4 Giới thiệu tập dữ liệu Synthea COVID-19	27
CHƯƠNG 2: TRIỂN KHAI VÀ XÂY DỰNG HỆ THỐNG	29
2.1. Thiết lập hạ tầng và môi trường	29
2.1.1 Xây dựng hạ tầng container hóa với Docker Compose	29
2.1.2 Cài đặt và cấu hình PostgreSQL Source Database	40
2.1.3 Cấu hình Datalake với MinIO và Hive Metastore	47
2.1.4 Cài đặt và tích hợp Trino Query Engine	50
2.1.5 Thiết lập môi trường lập lịch với Apache Airflow	53
2.1.6. Cài đặt Apache Superset để trực quan hóa dữ liệu	55
2.2. Xây dựng Pipeline xử lý dữ liệu.....	59
2.2.1 Ingestion: Trích xuất dữ liệu từ PostgreSQL sang lớp Bronze.....	59
2.2.2 Silver Layer: Làm sạch dữ liệu và chuẩn hóa kiểu dữ liệu.....	62
2.2.3 Gold Layer: Tổng hợp dữ liệu và phân tích các chỉ số COVID-19.....	69
2.2.4 Tự động hóa quy trình bằng Airflow DAGs	83
2.3. Khai thác và trình diễn dữ liệu bằng Trino và Apache Superset.....	89
2.3.1. Tạo datasets.....	91
2.3.2. Biểu đồ Cohort by Age Group	92

2.3.3. Biểu đồ Mortality by Age and Gender.....	94
2.3.4. Biểu đồ COVID Pathway Distribution	95
2.3.5. Dashboard hoàn chỉnh.....	99
CHƯƠNG 3: KẾT LUẬN	100
3.1. Kết quả đạt được.....	100
3.2. Hạn chế và thách thức	100
3.3. Hướng phát triển tương lai	101
TÀI LIỆU THAM KHẢO	102

MỞ ĐẦU

Trong kỷ nguyên số hóa y tế, dữ liệu lớn (Big Data) không chỉ là tài sản chiến lược mà còn là yếu tố sống còn giúp tối ưu hóa quy trình điều trị và dự báo dịch tễ. Tuy nhiên, các hệ thống truyền thống thường gặp khó khăn trong việc cân bằng giữa tính linh hoạt của Data Lake (lưu trữ dữ liệu thô đa dạng) và tính tin cậy của Data Warehouse (giao dịch ACID, quản lý dữ liệu có cấu trúc). Kiến trúc Lakehouse hiện đại ra đời như một sự hội tụ hoàn hảo, cho phép các tổ chức y tế vừa lưu trữ khối lượng lớn dữ liệu lâm sàng không cấu trúc với chi phí thấp, vừa đảm bảo hiệu suất truy vấn phân tích dữ liệu chuyên sâu.

Đề án này tập trung vào đề tài: “Xây dựng và triển khai Pipeline tự động hóa trên nền tảng Healthcare Lakehouse”, ứng dụng trực tiếp vào bộ dữ liệu dịch tễ học COVID-19. Mục tiêu cốt lõi của dự án là thiết lập một hệ thống xử lý dữ liệu đầu cuối (End-to-End) bền vững: từ việc thu thập dữ liệu thô (Batch/Streaming), thực hiện làm sạch và làm giàu dữ liệu qua mô hình Medallion Architecture (Bronze, Silver, Gold), đến việc phục vụ phân tích lâm sàng và trực quan hóa các chỉ số y khoa quan trọng.

Thông qua việc kết hợp các công nghệ hàng đầu như Apache Spark (xử lý song song), Apache Iceberg (định dạng bảng hiện đại), MinIO (lưu trữ đối tượng S3) và Airflow (điều phối tự động), dự án không chỉ dừng lại ở việc xử lý dữ liệu đơn thuần mà còn hướng tới các mục tiêu cao hơn:

Trình diễn dữ liệu: Trích xuất các KPIs y tế về tỷ lệ tử vong và lộ trình điều trị thông qua Apache Superset.

Phân tích dự báo: Ứng dụng mô hình Machine Learning (Logistic Regression) để phân loại và dự đoán nguy cơ dựa trên các dấu hiệu lâm sàng.

Báo cáo này sẽ trình bày chi tiết hành trình triển khai hệ thống, bao gồm: cấu hình hạ tầng Lakehouse bền vững, thiết lập các luồng công việc (Workflows) tự động với Airflow DAGs, và quy trình chuyển hóa dữ liệu qua các tầng Medallion để đạt được giá trị tri thức cuối cùng. Đây là minh chứng thực tiễn về khả năng vận hành một hệ thống Big Data hiện đại, đáp ứng các tiêu chuẩn khắt khe về hiệu năng và độ tin cậy trong ngành y tế.

CHƯƠNG 1: LÝ THUYẾT VỀ DATA LAKEHOUSE

1.1. Công nghệ và công cụ

1.1.1 Apache Spark – Xử lý dữ liệu phân tán

“Apache Spark là một công cụ phân tích thống nhất giúp xử lý dữ liệu quy mô lớn. Nó cung cấp các API cấp cao bằng Java, Scala, Python và R, cùng một công cụ được tối ưu hóa hỗ trợ đồ thị thực thi tổng quát. Nó cũng hỗ trợ một bộ công cụ cấp cao phong phú bao gồm Spark SQL để xử lý dữ liệu SQL và dữ liệu có cấu trúc.”.

Spark có thể chạy trên cả hệ điều hành Windows và các hệ thống giống Unix ví dụ như Linux hay MacOS, thậm chí là bất kỳ nền tảng nào chạy phiên bản Java được hỗ trợ. Việc chạy cục bộ trên một máy rất dễ dàng, tất cả những việc cần làm là cài đặt Java trên hệ thống của mình, dùng PATH và JAVA_HOME để thiết lập biến môi trường trỏ đến thư mục cài đặt Java.

Theo tài liệu chính thức của nhà phát hành Spark, nó đang chạy tốt nhất trên JAVA 17/21, Scala 2.13, Python 3.10+. Và có một lưu ý, khi API sử dụng Scala thì các ứng dụng cần phải sử dụng cùng phiên bản Scala và Spark được biên dịch, Từ spark 4.0.0 trở đi là Scala 2.13

Spark cũng đi kèm với một số chương trình mẫu, trong đó có nhóm dùng python để chạy, vì vậy trong phần này nhóm trình bày về các mẫu chạy bằng python. Cụ thể để chạy Spark tương tác trong trình thông dịch Python, hãy dùng bin/spark, lệnh thực hiện ./bin/pyspark --master "local[2]". Với lệnh trên sẽ chạy Spark cục bộ trên máy của bạn thân bằng 2 luồng. Nếu muốn dùng 1 luồng thì local hoặc dùng local[*] để dùng tất cả các lõi CPU hiện có.

Cách thực thi thứ hai là chạy ứng dụng đã viết sẵn. Giả sử ta đã viết code python vào một file, ta cần sử dụng công cụ spark-submit. Đây là công cụ tổng quát để gửi ứng dụng trên cụm Spark. lệnh ta có thể chạy với lệnh sau: ./bin/spark-submit examples/src/main/python/pi.py 10.

Từ phiên bản 3.4 trở đi, ta có thể kết nối từ xa Spark Connect. Đây là một kiến trúc hiện đại cho phép chạy code python từ bất cứ đâu kết nối một máy chủ Spark từ xa. ưu điểm là tách biệt môi trường của máy khách và máy chủ. Nó cũng hỗ trợ API DataFrame cho PySpark. Ta có thể nhúng Spark vào ứng dụng của mình một cách dễ dàng hơn

Khi dữ liệu quá lớn và cần chạy trên nhiều máy chủ, ta sẽ không thể dùng local nữa mà thay bằng các trình quản lý cụm với cơ chế Cluster Managers. Một vài chế độ của cơ chế này được giới thiệu như Standalone Mode (Chế độ đơn giản nhất, chạy trên các máy chủ riêng của Spark). Hadoop YARN giúp tận dụng của hệ sinh thái Hadoop, và Kubernetes, phù hợp khi dùng Docker hoặc các container Spark một cách linh hoạt. Các thông số quan trọng khi chạy gồm có `--master` : địa chỉ của cụm (local, yarn, k8s,...), `local[N]` giúp chạy cục bộ với N luồng xử lý. `--help` giúp hiển thị tất cả các tùy chọn cấu hình đi kèm.

Về mặt kiến trúc, Apache Spark vận hành theo mô hình Master-Slave. Thành phần trung tâm là Driver Program, chịu trách nhiệm điều hành toàn bộ ứng dụng, lập lịch các tác vụ và chuyển đổi các mã lệnh của người dùng thành một đồ thị có hướng không chu kỳ được gọi là Directed Acyclic Graph viết tắt là DAG. Các tác vụ này sau đó được phân phối đến các Executors chạy trên các Worker Nodes. Các Executor có nhiệm vụ trực tiếp tính toán và lưu trữ dữ liệu. Việc quản lý tài nguyên giữa Driver và các Executor được điều phối bởi *Cluster Manager* (như YARN, Mesos hoặc Kubernetes).

Điểm khác biệt lớn nhất của Spark so với các hệ thống trước đây như Hadoop MapReduce là khả năng xử lý dữ liệu trên bộ nhớ dựa trên khái niệm *RDD* tức Resilient Distributed Datasets. RDD là một tập hợp dữ liệu bất biến, phân tán, cho phép hệ thống tự phục hồi dữ liệu khi có lỗi xảy ra bằng cách truy xuất lại lịch sử các bước biến đổi dữ liệu. Hiện nay, người dùng chủ yếu tương tác qua các API cấp cao hơn là DataFrame và Dataset, giúp tối ưu hóa hiệu suất thông qua bộ công cụ Catalyst Optimizer và Tungsten.

Nhà phát triển của Apache Spark cũng nhấn mạnh rằng, nó không chỉ đơn thuần là một công cụ tính toán mà là một hệ sinh thái phân tích thống nhất bao gồm bốn module chính: đầu tiên là *Spark SQL* Hỗ trợ xử lý dữ liệu có cấu trúc và truy vấn SQL, cho phép kết nối linh hoạt với các kho dữ liệu Data Warehouse/Data Lake. Thứ hai là *Spark Streaming (Structured Streaming)* Cho phép xử lý các dòng dữ liệu thời gian thực với độ trễ thấp. Thứ ba là *MLlib* Thư viện cung cấp các thuật toán học máy phổ biến được thiết kế để chạy song song trên quy mô lớn. Cuối cùng là *GraphX* Công cụ xử lý và tính toán trên đồ thị.

Trong các kiến trúc dữ liệu hiện đại như *Medallion Architecture* với các lớp Bronze - Silver - Gold giống như nhóm đã thực hiện thì Spark đóng vai trò là "trái tim" của các đường ống ETL/ELT. Nhờ khả năng tích hợp mạnh mẽ với các định dạng lưu trữ như Delta Lake hay Apache Iceberg, Spark cho phép thực hiện các phép biến đổi dữ liệu phức tạp, làm sạch và chuẩn hóa dữ liệu từ các lớp thô (Bronze) sang các lớp chất lượng cao (Gold), đảm bảo tính toàn vẹn và hiệu suất truy vấn cho các hệ thống báo cáo phía sau.

1.1.2 Apache Airflow – Điều phối luồng dữ liệu

Apache AirFlow là một nền tảng mã nguồn mở được phát triển nhằm mục đích lập lịch, giám sát các quy trình công việc dưới dạng xử lý theo lô. Nó còn cho phép xây dựng các quy trình công việc kết nối với hầu hết mọi công nghệ. Giao diện được người dùng thiết kế trên web, giúp dễ dàng hình dung, quản lý và gỡ lỗi các quy trình công việc của bản thân. Airflow có thể chạy trong nhiều cấu hình khác nhau, từ một tiến trình duy nhất trên máy tính cá nhân đến một hệ thống phân tán có khả năng xử lý khối lượng công việc khổng lồ.

Quy trình làm việc của Airflow được định nghĩa hoàn toàn bằng python. điều này mang giúp các pipeline được định nghĩa trong các đoạn code được phép tạo và tham số hóa DAG một cách linh hoạt. Khung Airflow bao gồm nhiều toán tử tích hợp sẵn và có thể được mở rộng để phù hợp với nhu cầu của bản thân.

Phần này ta cũng sẽ nói rõ hơn về DAG đã nói ở mục 1.1.1, DAG là một mô hình bao hàm mọi thứ cần thiết để thực thi một quy trình công việc. Các thuộc tính của DAG bao gồm lịch trình (thời điểm quy trình công việc nên được thực thi), nhiệm vụ (các đơn vị công việc riêng biệt được thực hiện bởi), mối quan hệ giữa các tác vụ (thứ tự và điều kiện thực hiện các tác vụ), hàm gọi lại (các hành động cần thực hiện khi toàn bộ quy trình làm việc hoàn tất) và các thông số bổ sung khác. Một đoạn mã định nghĩa một DAG đơn giản sẽ có dạng như sau

```
from datetime import datetime

from airflow.sdk import DAG, task
from airflow.providers.standard.operators.bash import BashOperator
```

```

# A Dag represents a workflow, a collection of tasks
with DAG(dag_id="demo", start_date=datetime(2022, 1, 1), schedule="0
0 * * *") as dag:
    # Tasks are represented as operators
    hello = BashOperator(task_id="hello", bash_command="echo hello")

    @task()
    def airflow():
        print("airflow")

    # Set dependencies between tasks
    hello >> airflow()

```

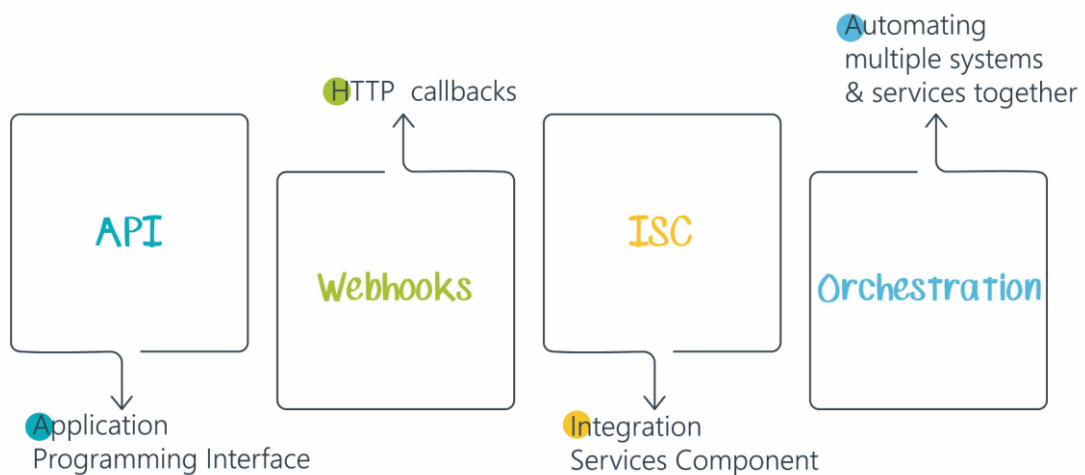
Đoạn mẫu trên đã định nghĩa một DAG có tên là “demo”, dự kiến sẽ chạy hàng ngày bắt đầu từ ngày 1 tháng 1 năm 2022, DAG là cách AirFlow biểu diễn một quy trình làm việc. Tiếp theo, code cũng có hai nhiệm vụ, một nhiệm vụ sử dụng BashOperator để chạy một tập lệnh shell, và một nhiệm vụ khác sử dụng task decorator để định nghĩa một hàm python. Trình >> điều khiển xác định mối quan hệ phụ thuộc giữa hai tác vụ và kiểm soát thứ tự thực thi.

Airflow là một nền tảng để điều phối các quy trình xử lý hàng loạt. Nó cung cấp một khung làm việc linh hoạt với nhiều toán tử tích hợp sẵn và giúp dễ dàng tích hợp các công nghệ mới. Airflow sẽ đặc biệt phù hợp với những yêu cầu có điểm bắt đầu và kết thúc rõ ràng và được thực hiện theo lịch trình. Một lý do quan trọng để AirFlow phổ biến vì nó là mã nguồn mở, đã được phát triển khá lâu vì vậy có vô số tài liệu học tập, bao gồm các bài đăng, sách báo,...

Mặc dù ta có thể kích hoạt DAG bằng CLI hoặc API REST, Airflow không dành cho các khối lượng công việc chạy liên tục, hướng sự kiện hoặc xử lý dữ liệu lỏng. Tuy vậy ta cần có thể dùng thêm Apache Kafka để giải quyết vấn đề thu thập dữ liệu theo thời gian thực.

Tóm lại để vận hành các quy trình phức tạp, kiến trúc của Airflow bao gồm các thành phần cốt lõi sau Scheduler, đây là "bộ não" của Airflow, chịu trách nhiệm theo

đổi các DAG, lập lịch các nhiệm vụ và gửi chúng đến Worker để thực thi khi các điều kiện phụ thuộc được đáp ứng. Thứ hai là Webserver, cung cấp giao diện người dùng để theo dõi, quản lý DAG và cấu hình hệ thống. Thứ ba là Metadata Database: Cơ sở dữ liệu (thường là PostgreSQL hoặc MySQL) lưu trữ mọi thông tin về cấu hình, lịch sử thực thi, người dùng và trạng thái của các tác vụ. Cuối cùng là Executor/Worker Thành phần trực tiếp thực hiện các tác vụ. Tùy vào cấu hình (Sequential, Local, Celery hoặc Kubernetes Executor) mà Airflow có thể chạy từ một máy đơn lẻ đến một cụm phân tán khổng lồ.



Hình 1.1.2.1 Kiến trúc Airflow

1.1.3 Apache Iceberg – Định dạng bảng Lakehouse

Định nghĩa

Apache Iceberg là định dạng bảng mã nguồn mở hiệu suất cao, được thiết kế chuyên biệt cho các khối lượng công việc phân tích dữ liệu khổng lồ trên hệ thống Data Lake như Amazon S3, HDFS hay Google Cloud Storage (GCS). Nó hoạt động như một lớp trừu tượng, tổ chức hàng ngàn hoặc hàng triệu tệp dữ liệu rời rạc thành các bảng có cấu trúc nhất quán, tương tự cơ sở dữ liệu quan hệ truyền thống, giúp quản lý dữ liệu lớn dễ dàng hơn.

Tại sao lại phải sử dụng Iceberg ?

Các Data Lake truyền thống trước đây chủ yếu sử dụng Hive Metastore làm catalog và Hive format table làm tiêu chuẩn để tổ chức dữ liệu. Nhưng qua thời gian, khi dữ liệu phình to ra, Hive bắt đầu bộc lộ những hạn chế. Đó là:

- Các hệ thống Data Lake không có cơ chế giao dịch ACID, khiến cho việc nhiều người dùng cùng đọc và ghi dữ liệu đồng thời rất dễ gây ra xung đột.
- Hiệu suất cực kỳ chậm do rào cản duyệt thư mục. Định dạng cũ quản lý bảng dựa trên danh sách các thư mục vật lý. Khi thực thi truy vấn, hệ thống phải duyệt qua toàn bộ các thư mục và tệp bên trong để lên kế hoạch, quá trình này cực kỳ chậm và dễ bị nghẽn mạng
- Khó khăn trong việc thay đổi cấu trúc khi bắt buộc phải ghi lại toàn bộ dữ liệu lịch sử của bảng, tốn thời gian và chi phí.
- Nhằm tránh các vấn đề trên và có được tốc độ truy vấn nhanh, các tổ chức buộc phải xây dựng các đường ống ETL phức tạp để sao chép dữ liệu từ Data Lake vào các hệ thống kho dữ liệu độc quyền và đắt đỏ. Việc này gây phân mảnh dữ liệu và tăng vọt chi phí.

Chính vì những lý do đó nên Apache Iceberg ra đời để biến Data Lake thành Data Lakehouse.

Đặc trưng cốt lõi

Iceberg nổi bật với các tính năng hỗ trợ quy mô lớn và độ tin cậy cao:

- Giao dịch ACID đầy đủ: Sử dụng cơ chế kiểm soát đồng thời, Iceberg đảm bảo tính nhất quán dữ liệu khi nhiều người dùng hoặc hệ thống đọc/ghi đồng thời. Mỗi giao dịch tạo snapshot riêng, giúp mọi người luôn đọc dữ liệu ở cùng một trạng thái nhất quán, tránh tình trạng đọc dữ liệu bản.
- Tiến hóa schema: Cho phép thay đổi cấu trúc bảng an toàn như thêm/xóa/đổi tên cột, hoặc nâng cấp kiểu dữ liệu (ví dụ: từ int lên long). Các thay đổi chỉ cập nhật metadata, không yêu cầu rewrite toàn bộ bảng hay làm gián đoạn truy vấn hiện tại – lý tưởng cho dữ liệu thay đổi liên tục.
- Tiến vùng phân vùng: Người dùng có thể cập nhật chiến lược phân vùng bất kỳ lúc nào mà không cần rewrite dữ liệu cũ tốn kém. Dữ liệu lịch sử giữ cấu trúc cũ, dữ liệu mới dùng cấu trúc mới; engine truy vấn tự động hợp nhất, đảm bảo hiệu suất quét tối ưu.

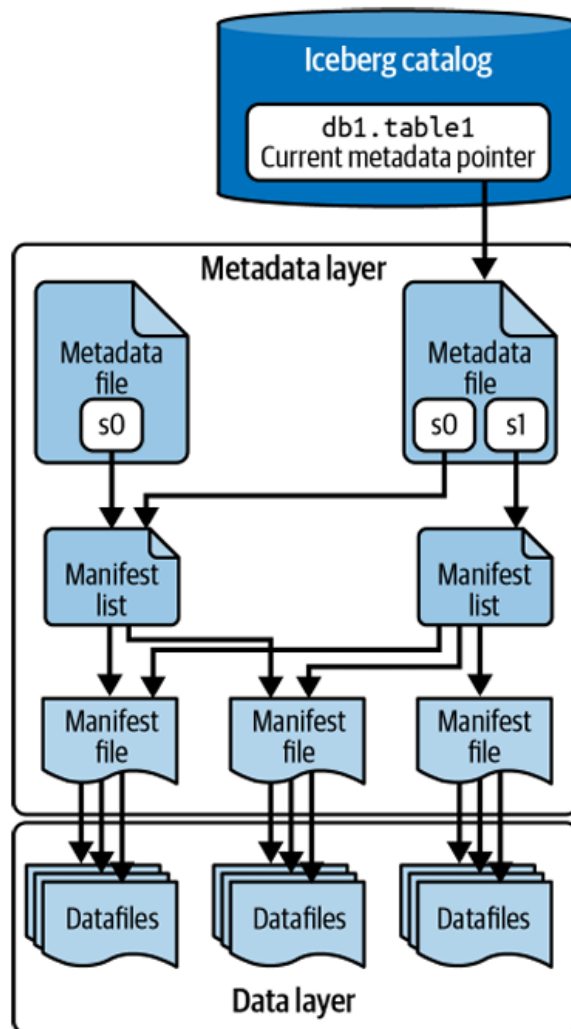
- Phân vùng ẩn: Tự động áp dụng biến đổi từ cột gốc để tạo giá trị phân vùng trong metadata. Điều này loại bỏ nhu cầu tạo cột dẫn xuất riêng, giúp truy vấn tự nhiên hơn mà tránh full table scan.
- Du hành thời gian và rollback: Duy trì danh sách snapshot bất biến theo thời gian, cho phép truy vấn trạng thái bảng tại bất kỳ điểm quá khứ nào (ví dụ: `SELECT * FROM table VERSION AS OF 123456789`) hoặc rollback dễ dàng nếu lỗi ghi dữ liệu xảy ra.

Kiến trúc

Iceberg sử dụng cấu trúc cây phân cấp với ba lớp chính:

- Lớp Catalog: Lưu trữ metadata về vị trí bảng (như Hive Metastore, AWS Glue).
- Lớp Metadata: Quản lý schema, phân vùng, snapshot dưới dạng file JSON/Avro bất biến.
- Lớp Data: Các file dữ liệu thực tế (Parquet, ORC, Avro), không thay đổi sau khi ghi.

Kiến trúc này giúp Iceberg mở rộng đến petabyte dữ liệu mà vẫn nhanh chóng (metadata chỉ vài KB/table), phù hợp cho phân tích lớn trên Data Lake.



Kiến trúc của Apache Iceberg (Nguồn: Figure 1-7 tài liệu [3])

Lớp Catalog: Đây là điểm truy cập đầu tiên đối với bất kỳ công cụ hoặc người dùng nào muốn tương tác với bảng. Danh mục có nhiệm vụ lưu trữ con trỏ tham chiếu đến tệp siêu dữ liệu hiện hành của bảng. Yêu cầu quan trọng nhất của lớp danh mục là phải hỗ trợ các thao tác cập nhật nguyên tử. Điều này giúp đảm bảo tính nhất quán của dữ liệu: khi có nhiều tiến trình đọc và ghi diễn ra đồng thời, tất cả đều sẽ thấy cùng một trạng thái của bảng.

Lớp Siêu dữ liệu: Lớp này là cốt lõi tạo nên khả năng quản lý mạnh mẽ của Iceberg, được cấu trúc dưới dạng một cây để theo dõi các tệp dữ liệu cũng như các thao tác tạo ra chúng. Lớp này bao gồm các thành phần:

- **Tệp siêu dữ liệu:** Lưu trữ thông tin tổng quan về trạng thái của bảng tại một thời điểm nhất định, bao gồm cấu trúc lược đồ, cấu trúc phân vùng, danh sách các phiên bản snapshots và xác định rõ snapshot nào đang là hiện hành.
- **Danh sách kê khai (Manifest list):** Đại diện cho một snapshot cụ thể của bảng tại một thời điểm. Tệp này chứa danh sách của tất cả các tệp kê khai thuộc về

snapshot đó, kèm theo thông tin về các phân vùng và thống kê (như giới hạn giá trị lớn nhất, nhỏ nhất của cột phân vùng) để tối ưu hóa việc lập kế hoạch truy vấn.

- **Tập kê khai (Manifest file):** Đóng vai trò là các node ở tầng cuối cùng của cây siêu dữ liệu. Nó theo dõi trực tiếp các tệp ở lớp dữ liệu (bao gồm tệp dữ liệu và tệp xóa), đồng thời lưu trữ các số liệu thống kê chi tiết cho từng tệp như: giới hạn dưới (lower bound), giới hạn trên (upper bound) và số lượng giá trị null của các cột. Nhờ các thống kê này, hệ thống có thể nhanh chóng loại bỏ các tệp không chứa dữ liệu cần thiết mà không phải mở tệp ra đọc

Lớp dữ liệu: Đây là lớp thấp nhất, nơi lưu trữ dữ liệu thực tế của bảng, thường nằm trên các hệ thống tệp phân tán hoặc lưu trữ đối tượng đám mây như Amazon S3, HDFS, GCS hoặc ADLS. Lớp này chứa:

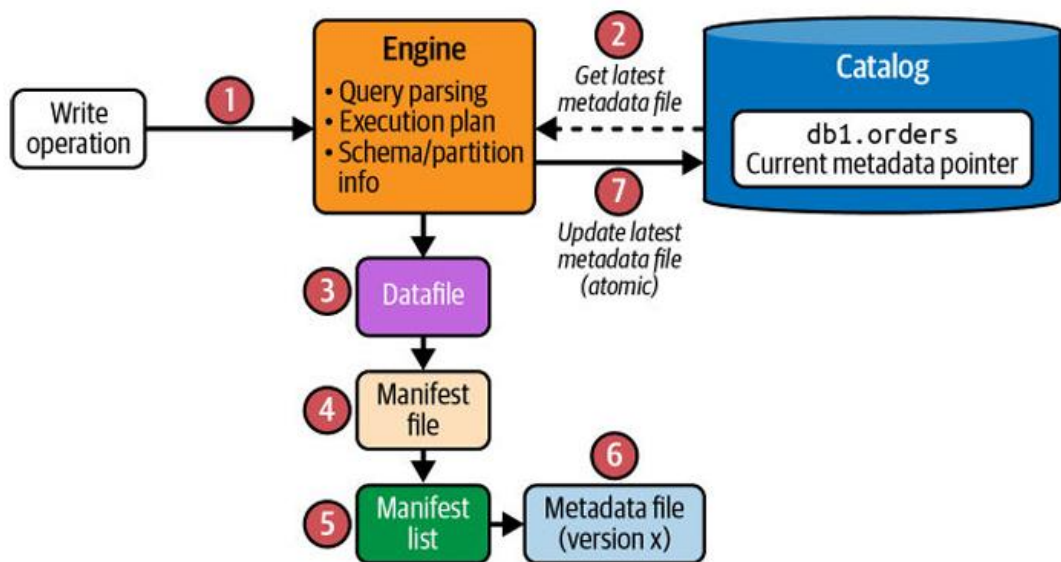
- **Tệp dữ liệu:** Nơi chứa các bản ghi dữ liệu thực. Iceberg không bị giới hạn ở một định dạng duy nhất mà hỗ trợ các chuẩn tệp mở như Apache Parquet, Apache ORC và Apache Avro. Trong đó, định dạng cột Parquet thường được sử dụng phổ biến nhất vì nó mang lại hiệu suất cao cho các khối lượng công việc phân tích quy mô lớn
- **Tệp xóa (Delete files):** Hỗ trợ xóa/cập nhật mà không rewrite dữ liệu. Sử dụng Merge-on-Read (MOR): engine truy vấn hợp nhất delete files với data files lúc đọc, giảm chi phí ghi (ví dụ: xóa 1 dòng chỉ cần 1 delete file nhỏ thay vì rewrite toàn bộ Parquet).

Cách thức hoạt động

Khi người dùng muốn **ghi dữ liệu**, quy trình sẽ được thực thi từ dưới lên để đảm bảo tính nguyên tử. Cụ thể qua các bước sau:

1. Kiểm tra Catalog: Lấy metadata hiện hành để biết schema/phân vùng.
2. Ghi Data Files: Viết Parquet mới (COW/MOR).
3. Tạo Manifest File: Liệt kê data files + stats (min/max/nulls).
4. Tạo Manifest List: Gộp manifests cho snapshot mới.
5. Tạo Metadata File: Ghi snapshot mới làm current.
6. Commit Catalog: Atomic update con trỏ; retry nếu conflict.

Ví dụ: `MERGE INTO table USING updates ON ...` sẽ tạo snapshot mới ngay lập tức.

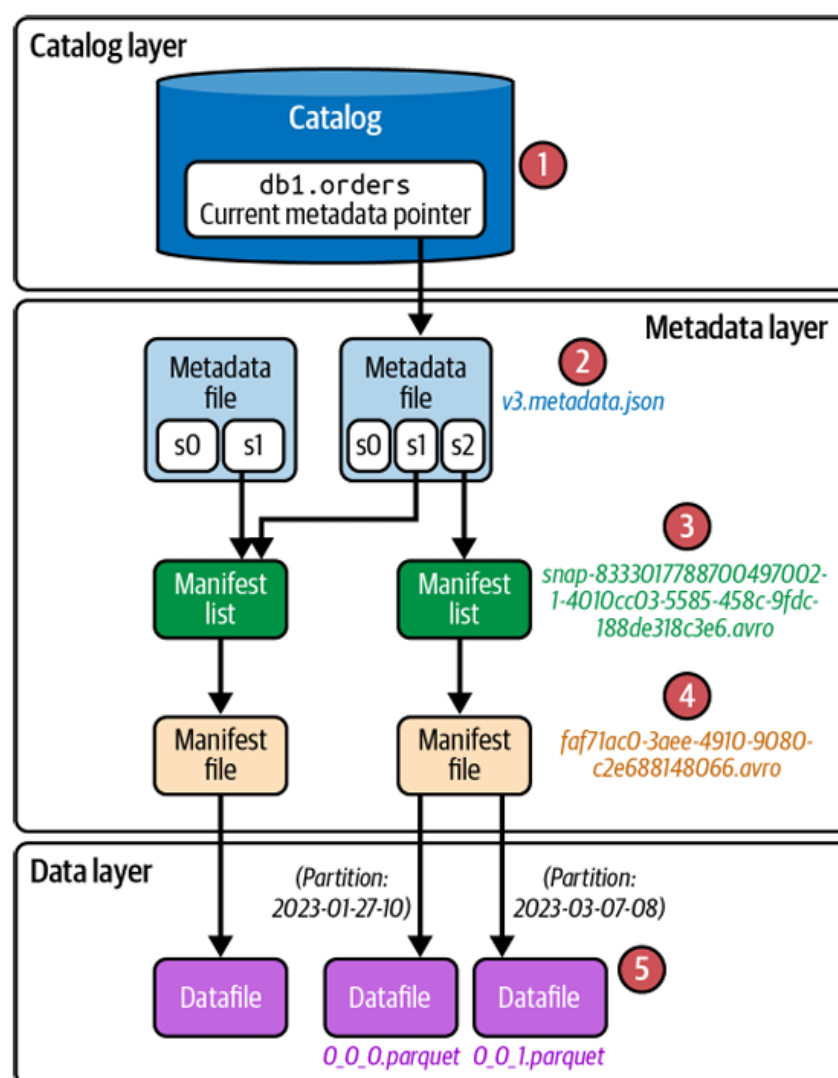


Quy trình vận hành khi ghi file trong Iceberg (Nguồn: Figure 3-2 tài liệu [3])

Khi sử dụng các công cụ thực thi truy vấn để **đọc dữ liệu** như Trino, quy trình sẽ được thực hiện từ trên xuống nhằm tận dụng các metadata để lọc bớt dữ liệu, tránh quét toàn bộ bảng:

1. Catalog: Lấy metadata hiện hành.
2. Metadata File: Xác định snapshot + manifest list.
3. Manifest List: Lấy manifest files.
4. Pruning: So WHERE clause với stats để skip file..
5. Đọc Data Files: Chỉ scan files cần thiết + apply deletes (MOR).

Ví dụ: Đối với câu truy vấn `SELECT * FROM table WHERE col > 100`, engine chỉ đọc 10% files nếu 90% dữ liệu của col bé hơn bằng 100.



Quy trình vận hành khi đọc file trong Iceberg (Nguồn: Figure 3-6 tài liệu [3])

Du hành thời gian hoạt động tương tự read, nhưng chọn snapshot cũ qua AS OF timestamp/version-id ở bước 2. Truy vấn lịch sử không ảnh hưởng current data.

1.1.4 MinIO – Lưu trữ đối tượng tương thích S3

Tổng quan

MinIO là một hệ thống lưu trữ đối tượng (Object Storage) mã nguồn mở, hiệu năng cao, được thiết kế để tương thích hoàn toàn với API của Amazon S3. Khác với các hệ thống tệp truyền thống, MinIO quản lý dữ liệu dưới dạng các đối tượng (Objects) trong các thùng chứa (Buckets), tối ưu hóa cho các khối lượng công việc hiện đại như AI, Machine Learning và đặc biệt là kiến trúc Data Lakehouse.

Các đặc tính kỹ thuật quan trọng

Tương thích hoàn toàn với S3 API: MinIO cung cấp khả năng tương thích cao nhất với Amazon S3 API, cho phép các ứng dụng và công cụ hiện đại kết nối dễ dàng mà không cần thay đổi mã nguồn.

Cơ chế Erasure Coding: Thay vì sử dụng RAID truyền thống, MinIO sử dụng mã hóa xóa để bảo vệ dữ liệu. Dữ liệu được chia nhỏ và phân tán giúp hệ thống vẫn có thể phục hồi ngay cả khi nhiều ổ cứng hoặc node bị lỗi đồng thời.

Chống suy hao dữ liệu (Bitrot Protection): MinIO tự động tính toán mã băm cho từng đối tượng để phát hiện và tự động sửa các lỗi dữ liệu phát sinh âm thầm do vấn đề vật lý từ phần cứng.

Hiệu năng vượt trội: Được viết bằng ngôn ngữ Go và tối ưu hóa bằng tập lệnh SIMD trên CPU, MinIO đạt tốc độ đọc/ghi cực nhanh, đáp ứng nhu cầu truy xuất dữ liệu lớn của các hệ thống phân tán.

Lý do lựa chọn sử dụng Trong phạm vi dự án

MinIO được lựa chọn nhờ các ưu điểm vượt trội:

- Khả năng mở rộng linh hoạt: Dễ dàng triển khai dưới dạng Container và mở rộng dung lượng theo chiều ngang mà không làm gián đoạn hệ thống.
- Tiêu chuẩn cho Data Lake: Là lớp lưu trữ phổ biến nhất để xây dựng Data Lake ngoài đám mây công cộng, giúp doanh nghiệp tự chủ về hạ tầng (On-premise).
- Tiết kiệm tài nguyên: Nhẹ hơn và dễ quản trị hơn rất nhiều so với hệ thống Hadoop HDFS truyền thống nhưng vẫn đảm bảo độ tin cậy tương đương.

Khả năng tích hợp mạnh mẽ với hệ sinh thái

MinIO đóng vai trò là "linh hồn" của lớp lưu trữ (Storage Layer) trong dự án:

- Tích hợp với Apache Spark: Thông qua giao thức s3a, Spark có thể đọc và ghi các tệp dữ liệu trực tiếp trên MinIO. Trong dự án, Spark sử dụng MinIO để lưu trữ các bảng định dạng Parquet theo tiêu chuẩn của Apache Iceberg.
- Hỗ trợ Hive Metastore & Trino: MinIO cung cấp nơi lưu trữ vật lý cho các file dữ liệu, trong khi Hive Metastore quản lý siêu dữ liệu (Metadata). Công cụ truy vấn Trino kết nối đến MinIO để thực hiện các câu lệnh SQL phân tích với hiệu suất cao.
- Tương tác qua MinIO Client (mc): Cung cấp bộ công cụ dòng lệnh mạnh mẽ để tự động hóa các tác vụ quản lý dữ liệu, phân quyền và kiểm tra trạng thái bucket thông qua các script hoặc DAG của Airflow.

Vai trò cốt lõi trong dự án hiện tại

Trong kiến trúc Medallion mà dự án đang triển khai, MinIO đóng vai trò là kho lưu trữ tập trung cho mọi giai đoạn của dữ liệu:

- Vùng dữ liệu thô (Raw/Bronze): Lưu trữ các file CSV gốc được đẩy từ PostgreSQL qua và các bảng Iceberg sơ khai chứa toàn bộ lịch sử dữ liệu y tế.
- Lớp xử lý tinh lọc (Silver/Gold): Là nơi lưu trữ các kết quả trung gian và lớp dữ liệu tổng hợp đã được xử lý sạch, sẵn sàng cho việc phân tích và hiển thị lên Superset.
- Nền tảng cho Apache Iceberg: Cung cấp hạ tầng vật lý ổn định để duy trì các tính năng nâng cao của Iceberg như Time Travel (truy vấn dữ liệu trong quá khứ) và Upsert (cập nhật/chèn dữ liệu) một cách hiệu quả.

1.1.5 Trino – Công cụ truy vấn SQL phân tán

Định nghĩa

Trino là một công cụ truy vấn SQL phân tán, mã nguồn mở được thiết kế để truy vấn hiệu quả các tập dữ liệu khổng lồ với quy mô từ gigabyte đến petabyte trên nhiều nguồn dữ liệu khác nhau.

Tại sao lại sử dụng Trino ?

Nếu thiếu đi một engine truy vấn phân tán như Trino, hệ thống dữ liệu của tổ chức sẽ nhanh chóng gặp phải các giới hạn về chi phí, hiệu suất và sự phức tạp trong vận hành khi dữ liệu ngày càng phình lớn. Cụ thể:

- Để có thể phân tích chung các dữ liệu nằm rải rác, ta bắt buộc phải xây dựng các đường ống ETL cực kỳ phức tạp để liên tục sao chép, di chuyển dữ liệu từ các hệ thống nguồn về một kho dữ liệu trung tâm. Quá trình này tốn nhiều thời gian, rủi ro lỗi cao, dữ liệu bị trùng lặp và thiếu tính cập nhật.
- Khi không có Trino làm cầu nối, dữ liệu của tổ chức sẽ bị cô lập trong các hệ thống khác nhau (RDBMS, NoSQL, Data Lake). Người dùng, các nhà phân tích kinh doanh sẽ không biết dữ liệu nằm ở đâu, và nếu biết, họ cũng không có cách nào dễ dàng kết hợp chúng lại với nhau. Việc không tận dụng được dữ liệu toàn cục làm giảm sút chất lượng ra quyết định.
- Nếu đưa tất cả dữ liệu vào một Data Warehouse truyền thống độc quyền, ta sẽ bị khóa chặt vào định dạng lưu trữ và hệ sinh thái của nhà cung cấp đó. Chi phí sẽ

tăng vọt khi khối lượng dữ liệu phình to do đó ta không thể mở rộng riêng biệt phần lưu trữ và tính toán.

- Nếu lưu dữ liệu trên Data Lake và dùng các engine đời cũ để truy vấn, người dùng sẽ phải chịu đựng thời gian chờ đợi phản hồi rất lâu cho mỗi truy vấn. Điều này triệt tiêu khả năng phân tích ad-hoc và làm chậm đi tốc độ tìm ra insight quan trọng cho doanh nghiệp

Trong quá trình tìm hiểu các công cụ làm query engine thì nhóm cũng phát hiện ra được có một công cụ khác cũng làm được điều tương tự như Trino là Spark Thrift Server (STS). Cả 2 công cụ này đều có thể đọc vào Hive Metastore và siêu dữ liệu của Iceberg. Nhưng có một số điểm khác biệt như sau:

- Về bản chất thì STS vẫn mang dáng dấp của Spark vốn dùng để chạy Batch nên khi có nhiều người cùng truy vấn thì STS xử lý chậm hơn Trino. Do Trino là công cụ được xây dựng riêng và chuyên dụng cho việc truy vấn phân tích.
- STS phù hợp khi muốn sử dụng chung hệ sinh thái Spark, đồng nhất code ETL, tiết kiệm RAM khi không phải chạy thêm Trino làm query engine, và khả năng chịu lỗi bằng cách mượn đĩa cứng nếu thiếu RAM, Trino mà thiếu RAM thì sập luôn.

Nhóm quyết định sử dụng Trino bởi vì tốc độ của nó và khả năng truy vấn linh hoạt từ nhiều nguồn. Hơn thế nữa, khi tải các dashboard các công cụ BI sẽ nhanh chóng hơn và nhiều người có thể truy vấn liên tục mà không bị gián đoạn.

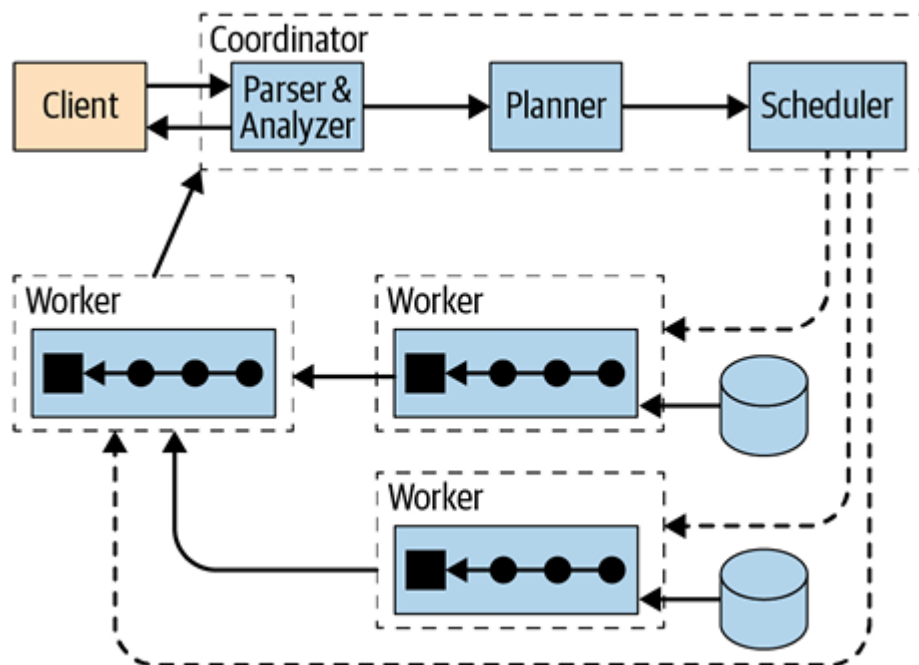
Đặc trưng cốt lõi

- Mặc dù Trino hiểu và thực thi các câu lệnh SQL cực kỳ tốt, nó hoàn toàn không có hệ thống lưu trữ dữ liệu riêng biệt. Nó không được thiết kế để trở thành cơ sở dữ liệu quan hệ đa dụng nhằm thay thế MySQL hay PostgreSQL, và đặc biệt không dùng để xử lý giao dịch trực tuyến (OLTP). Trino phục vụ chủ yếu cho các khối lượng công việc phân tích (OLAP).
- Trino có thể truy vấn dữ liệu trực tiếp ngay tại nơi nó đang được lưu trữ, bao gồm hệ thống lưu trữ đối tượng, cơ sở dữ liệu quan hệ, cơ sở dữ liệu NoSQL, v.v. Điểm mạnh của Trino là nó không yêu cầu ta phải di chuyển hay sao chép dữ liệu về một kho chung tập trung mới có thể phân tích được. Đây là một trong những điểm mạnh của Trino khi có khả năng truy vấn liên kết dữ liệu từ nhiều nguồn khác nhau.

- Trong cấu trúc hoạt động, Trino đóng vai trò thuần túy là lớp tính toán, còn các hệ thống dữ liệu bên dưới là lớp lưu trữ. Thiết kế này cho phép tài nguyên tính toán và lưu trữ có thể được mở rộng một cách độc lập với nhau nhằm tối ưu chi phí và hiệu suất. Ngoài ra, mọi thao tác Trino đều thực hiện trên bộ nhớ nên tốc độ truy vấn mang lại sẽ vượt trội hơn so với các công nghệ cũ như Hive vốn dựa vào MapReduce có độ trễ cao khi khởi động.
- Kiến trúc xử lý song song quy mô lớn có thể mở rộng theo chiều ngang thay vì dựa vào việc nâng cấp một máy chủ duy nhất, Trino phân tán việc xử lý truy vấn trên một cụm máy chủ. Cụm này được điều hành bởi một máy chủ điều phối (Coordinator) chuyên nhận và lên kế hoạch truy vấn, cùng nhiều máy chủ xử lý (Worker nodes) chuyên thực thi tác vụ và quét dữ liệu. Ta có thể nâng cấp bằng cách thêm vào các máy Worker giúp mở rộng Cluster khi các tác vụ truy vấn trở nên nặng nề hơn.

Kiến trúc

Trino hoạt động trong mô hình cluster nên sẽ gồm 2 thành phần thực thi chính là Coordinator và Worker.



Kiến trúc tổng quát của Trino gồm coordinator và workers (Nguồn: Figure 4-1 tài liệu [2])

Coordinator là Trino server chuyên tiếp nhận và xử lý các câu truy vấn SQL từ người dùng, lên kế hoạch thực thi truy vấn và quản lý các worker node.

Worker là các máy đảm nhận việc thực thi các tác vụ được giao bởi coordinator, bao gồm truy xuất dữ liệu từ các nguồn, sau đó xử lý dữ liệu. Trong một cluster thì các

máy worker có thể giao tiếp với nhau để trao đổi dữ liệu và coordinator có thể giao tiếp với mọi máy Worker trong một cluster.

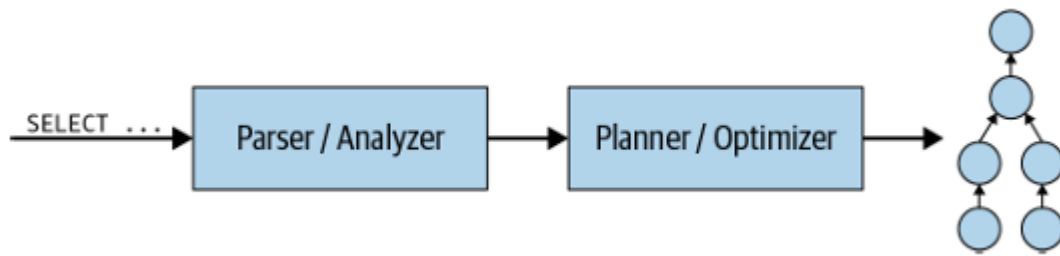
Dịch vụ khám phá (discovery service) được chạy trên máy coordinator dùng để tìm các node trong một cluster. Định kỳ giám sát tín hiệu từ các máy Worker, coordinator sẽ thu được danh sách các Worker hiện đang khả dụng để thực hiện việc lập kế hoạch phân công task.

Kết nối dựa trên kiến trúc (Connector-Based Architecture) là một mối nối cung cấp cho Trino một giao diện để có thể truy cập vào các nguồn dữ liệu độc lập. Đây là tính chất của Trino khi công cụ này được tách bạch giữa lớp tính toán và lớp lưu trữ. Như ta đã biết thì Trino có thể truy vấn từ nhiều nguồn khác nhau, có thể là từ Data Lake, cơ sở dữ liệu quan hệ hay phi quan hệ, mỗi mối nối cung cấp một bảng trừu tượng dựa trên những nguồn dữ liệu khác nhau. Qua đó mà Trino có thể thực hiện các câu lệnh truy vấn thông thường, giúp cho người sử dụng cảm thấy quen thuộc hơn đối với các dạng bảng.

Cách thức hoạt động

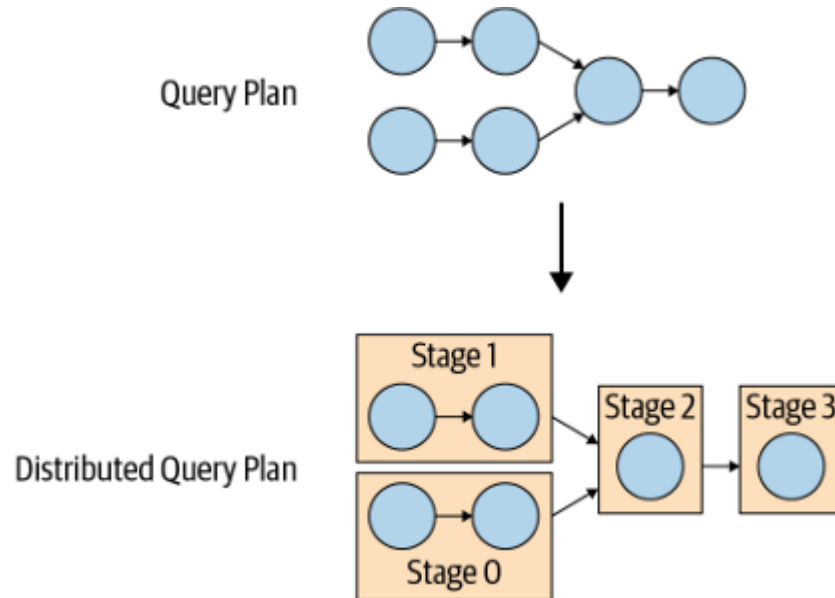
Trino hoạt động dựa trên mô hình thực thi truy vấn. Cụ thể là khi người dùng gửi một câu truy vấn, quá trình vận hành sẽ diễn ra như sau:

1. Parsing & Analysis: Coordinator nhận câu lệnh văn bản SQL, kiểm tra cú pháp và dùng Metadata SPI để đối chiếu xem các bảng và cột có tồn tại và hợp lệ hay không.
2. Query Optimization:
 - a. Trino áp dụng các quy tắc tối ưu hóa như đẩy điều kiện lọc xuống để loại bỏ dữ liệu rác ngay từ nguồn. Loại bỏ Join chéo và gộp thao tác ORDER BY kết hợp LIMIT thành toán tử TopN để tiết kiệm bộ nhớ.
 - b. Đặc biệt, Trino sử dụng trình tối ưu hóa dựa trên chi phí (Cost-Based Optimizer - CBO). Bằng cách sử dụng các số liệu thống kê từ Data Statistics SPI, CBO sẽ ước tính kích thước bảng và chi phí (CPU, bộ nhớ, mạng) để sắp xếp lại thứ tự JOIN sao cho tối ưu nhất.



Xử lý câu truy vấn để tạo query plan (Nguồn: Figure 4-6 tài liệu [2])

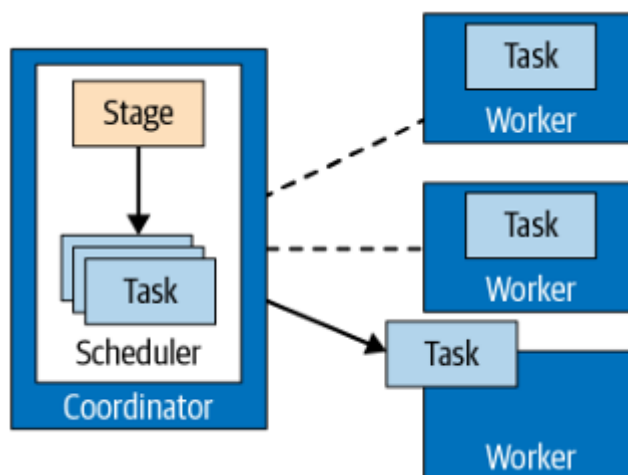
3. Distributed Query Plan: Kế hoạch logic sau khi tối ưu được cắt nhỏ thành nhiều giai đoạn (Stages) tạo thành một kế hoạch truy vấn phân tán.



Chuyển đổi từ kế hoạch truy vấn thành kế hoạch truy vấn phân tán (Nguồn: Figure 4-8 tài liệu [2])

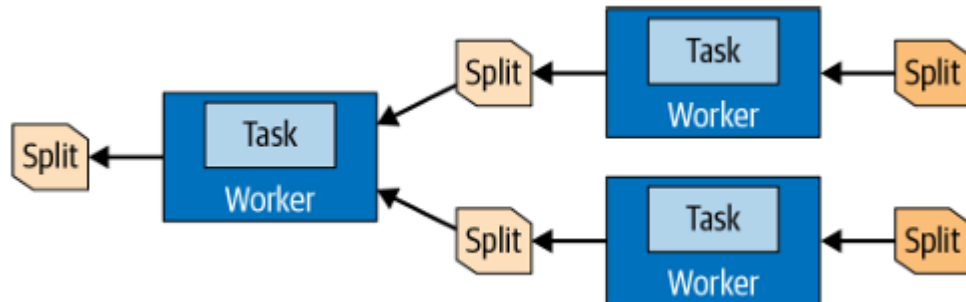
4. Tasks & Splits:

- a. Mỗi Stage được Coordinator giao cho các Worker dưới dạng các Tasks.



Coordinator quản lý và chia task cho các Worker (Nguồn: Figure 4-9 tài liệu [2])

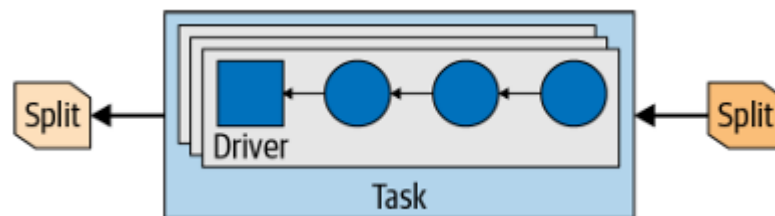
- b. Dữ liệu cần quét được chia thành các phần nhỏ gọi là Splits (Phân mảnh). Split là đơn vị song song nhỏ nhất mà một worker sẽ xử lý. Worker có thể nhận split được thực thi từ Worker khác làm đầu vào và đầu ra cũng có thể được đưa cho một Worker khác làm đầu vào.



Các phân mảnh được chuyển giao qua từng task với Workers khác nhau (Nguồn: Figure 4-10 tài liệu [2])

5. Operators & Pages:

- a. Bên trong mỗi Task trên Worker, Trino sẽ khởi tạo các Drivers (luồng chạy). Mỗi Driver là một đường ống chứa chuỗi các toán tử (như quét bảng, lọc, nhóm, join)



Các luồng chạy song song trong một task với đầu vào và đầu ra các split (Nguồn: Figure 4-11 tài liệu [2])

- b. Thay vì xử lý từng dòng một, các toán tử của Trino xử lý và luân chuyển dữ liệu dưới dạng các Trang – là các khối chứa nhiều dòng dữ liệu được tổ chức theo định dạng cột trong bộ nhớ để tăng tốc độ xử lý
- c. Các trang dữ liệu này được trao đổi giữa các worker thông qua các Exchange operators trên mạng
6. Return value: Khi dữ liệu đi qua toàn bộ các giai đoạn và dồn về Stage cuối cùng (thường là Stage 0 chạy trên Coordinator), Coordinator sẽ thu thập và trả luồng kết quả này về cho người dùng một cách liên tục ngay cả khi các worker vẫn đang xử lý các tập dữ liệu còn lại,

1.1.6 Hive Metastore – Quản lý Catalog dữ liệu

Định nghĩa

Hive Metastore (HMS) là một dịch vụ trung tâm làm nhiệm vụ lưu trữ siêu dữ liệu, mô tả cách thức dữ liệu vật lý nằm trên các hệ thống lưu trữ phân tán được ánh xạ thành các cấu trúc logic như cơ sở dữ liệu, bảng và cột. Để lưu trữ bền vững các siêu dữ liệu này, HMS thường sử dụng một cơ sở dữ liệu quan hệ đứng sau như MySQL, PostgreSQL hoặc Derby. Trong kiến trúc định dạng bảng Hive truyền thống, HMS theo dõi các đường dẫn thư mục vật lý định nghĩa nên một bảng, từ đó cung cấp cho các động cơ truy vấn (như Hive, Spark, Trino) địa chỉ chính xác để tìm kiếm và quét dữ liệu.

Tại sao lại phải dùng HMS ?

Mặc dù Apache Iceberg quản lý cấu trúc dữ liệu của mình bằng một cây siêu dữ liệu nằm ngay trên lớp lưu trữ vật lý, nó vẫn bắt buộc phải có một lớp Danh mục. Trino cần kết hợp với HMS (hoặc một dịch vụ Catalog tương đương như AWS Glue, Nessie) khi truy vấn bảng Iceberg vì những lý do cốt lõi sau:

- Xác định con trỏ siêu dữ liệu hiện hành: Điểm truy cập đầu tiên đối với bất kỳ công cụ nào khi muốn đọc hay ghi một bảng Iceberg là tìm vị trí của tệp siêu dữ liệu hiện tại. Bản thân kiến trúc Trino không chứa sẵn hệ thống lưu trữ siêu dữ liệu bên trong nó. Khi dùng HMS làm Iceberg Catalog, thuộc tính của bảng trong HMS (như location) sẽ đóng vai trò lưu trữ đường dẫn tuyệt đối trỏ tới tệp metadata.json mới nhất của bảng Iceberg đó.
- Đảm bảo tính nguyên tử trong giao dịch: Một yêu cầu bắt buộc để vận hành Iceberg an toàn trên môi trường thực tế là Catalog phải cung cấp khả năng cập nhật nguyên tử. Khi Trino thực hiện các tác vụ ghi và tạo ra một tệp metadata mới cho bảng Iceberg, HMS đóng vai trò làm chốt chặn an toàn để hoán đổi con trỏ siêu dữ liệu. Điều này ngăn ngừa tình trạng hỏng hóc hoặc mất mát dữ liệu khi có nhiều tiến trình đọc, ghi diễn ra đồng thời (concurrent writes) trên cùng một bảng.
- Để Trino có thể truy vấn dữ liệu nằm trong Iceberg thì HMS đóng vai trò như là một bản đồ để biết phải nhìn vào Catalog nào. Thế nên Trino phải kết nối được với HMS trước rồi sau đó mới có thể thực hiện các thao tác đọc, ghi trong Iceberg.

Kiến trúc

Mô hình triển khai và giao thức giao tiếp: Về mặt tương tác, HMS cung cấp hai mô hình kết nối chính giữa client (như Hive, Trino, Spark) và kho siêu dữ liệu:

- Kết nối trực tiếp qua JDBC: Client có thể mở kết nối trực tiếp đến cơ sở dữ liệu JDBC backend để thao tác và lấy siêu dữ liệu
- Máy chủ Thrift Metastore: Đây là kiến trúc phổ biến và bảo mật hơn dành cho kiến trúc phân tán đa người dùng. HMS chạy như một dịch vụ máy chủ độc lập (gọi là ThriftMetastore). Client sẽ kết nối từ xa đến máy chủ này thông qua giao thức Thrift (mặc định trên cổng 9083), và chính máy chủ Thrift này mới là nơi thực hiện các lệnh gọi qua JDBC xuống cơ sở dữ liệu backend. Mô hình này cũng cho phép các client không sử dụng Java có thể lấy được siêu dữ liệu thông qua Thrift

Trong hai mô hình kể trên thì mô hình Thrift Metastore được sử dụng trong phạm vi dự án này bởi vì Thrift chính là giao thức gốc mà bản thân hệ sinh thái Hive sử dụng để cung cấp API ra bên ngoài. Trong đó, Trino đóng vai trò là một máy khách cần kết nối để lấy thông tin siêu dữ liệu, nó bắt buộc dùng chung giao thức mà hệ thống HMS hỗ trợ.

Khái niệm về Thrift: Thrift là một framework được thiết kế để phát triển các dịch vụ có khả năng mở rộng và hỗ trợ giao tiếp xuyên ngôn ngữ. Nó là một thủ tục gọi từ xa (RPC - Remote Control Procedure). Thrift cung cấp một ngôn ngữ định nghĩa giao diện. Dựa vào bản định nghĩa giao diện này, trình biên dịch của Thrift sẽ tự động sinh ra mã nguồn để tạo các network RPC client cho rất nhiều ngôn ngữ lập trình khác nhau. Hơn thế nữa, thay vì truyền dữ liệu kiểu chữ tồn diện tích, Thrift nén dữ liệu thành dạng nhị phân khiến cho hệ thống trở nên cực nhẹ và nhanh.

1.1.7 Apache Superset – Trực quan hóa dữ liệu

Định nghĩa

Apache Superset là một nền tảng BI mã nguồn mở, hoạt động theo mô hình web application, cho phép kết nối nhiều nguồn dữ liệu thông qua lớp SQLAlchemy và cung cấp hai năng lực cốt lõi: khám phá dữ liệu ad-hoc trên SQL Lab, và xây dựng dashboard vận hành trên các chart đã chuẩn hóa. Mục đích sử dụng Superset là để cho phép nhóm chuyển từ dữ liệu tổng hợp ở Iceberg Gold sang các dashboard diễn giải câu hỏi nghiệp vụ như phân bố cohort theo độ tuổi, khác biệt tỷ lệ tử vong theo giới tính, hoặc phân bố pathway điều trị COVID. Đây là điểm quan trọng vì báo cáo dữ liệu y tế cần đồng thời

bảo đảm tính đúng về số liệu và tính dễ hiểu khi truyền đạt cho người không chuyên kỹ thuật.

Đặc trưng cốt lõi

Các đặc trưng nổi bật của Superset trong dự án này gồm:

- Khả năng kết nối engine truy vấn Trino qua URI SQLAlchemy
- Khả năng tạo dataset trực tiếp từ bảng hoặc từ truy vấn SQL.
- Khả năng thiết kế dashboard kéo thả và tái sử dụng chart giữa nhiều báo cáo.
- Việc tích hợp driver Trino được xử lý trong image tùy biến ở Dockerfile, bảo đảm Superset nhận diện đúng Trino dialect khi kết nối.

Cách thức hoạt động

Superset thực thi theo chu trình: admin cấu hình kết nối dữ liệu → user tạo dataset → hệ thống sinh câu lệnh SQL tương ứng với chart → SQL được gửi đến Trino → Trino đọc metadata từ Hive Metastore và dữ liệu vật lý từ MinIO/Iceberg → kết quả truy vấn trả về Superset để render biểu đồ.

Khi người dùng thao tác filter hoặc đổi metric trên dashboard, Superset phát sinh truy vấn mới theo thời gian thực. Cơ chế này cho phép dashboard luôn phản ánh dữ liệu hiện có, đồng thời giảm tải cho frontend vì toàn bộ xử lý truy vấn nặng nằm ở query engine.

1.1.8 PostgreSQL – Hệ quản trị cơ sở dữ liệu nguồn

Tổng quan

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ đối tượng (Object-Relational Database Management System - ORDBMS) mã nguồn mở, được phát triển dựa trên dự án POSTGRES tại Đại học California, Berkeley¹. Với lịch sử phát triển hơn 30 năm, PostgreSQL đã khẳng định được vị thế là một trong những hệ quản trị cơ sở dữ liệu ổn định và mạnh mẽ nhất thế giới, tuân thủ chặt chẽ các tiêu chuẩn SQL.

Các đặc tính kỹ thuật quan trọng

Tuân thủ chuẩn ACID: PostgreSQL đảm bảo tuyệt đối các tính chất Atomicity (Nguyên tử), Consistency (Nhất quán), Isolation (Cô lập), và Durability (Bền vững).

¹ PostgreSQL Global Development Group, “What Is PostgreSQL?,” *PostgreSQL Documentation*, 2026. [Online]. Available: <https://www.postgresql.org/docs/current/intro-what-is.html>. [Ngày truy cập: 10/4/2026].

Điều này đặc biệt quan trọng đối với các hệ thống yêu cầu sự chính xác cao về dữ liệu như tài chính và giao dịch thương mại.

Cơ chế MVCC (Multi-Version Concurrency Control): Giúp quản lý việc truy cập đồng thời hiệu quả, cho phép nhiều người dùng cùng đọc và ghi dữ liệu mà không gây ra tình trạng khóa bảng (table locking), từ đó tối ưu hóa hiệu suất hệ thống.

Hỗ trợ dữ liệu đa dạng: Ngoài các kiểu dữ liệu quan hệ truyền thống, PostgreSQL hỗ trợ mạnh mẽ dữ liệu bán cấu trúc (JSONB, XML) với tốc độ truy vấn nhanh tương đương các cơ sở dữ liệu NoSQL, cùng khả năng xử lý dữ liệu không gian (GIS) qua tiện ích mở rộng PostGIS.

Tính mở rộng (Extensibility): Cho phép người dùng tự định nghĩa các kiểu dữ liệu, hàm (functions), và chỉ mục (indexes) riêng. Hệ thống cũng hỗ trợ thực thi mã nguồn bằng nhiều ngôn ngữ như Python, Java, và C/C++.

Lý do lựa chọn sử dụng

Trong phạm vi dự án, PostgreSQL được lựa chọn nhờ vào các ưu điểm sau:

- **Hiệu suất truy vấn phức tạp:** Khả năng tối ưu hóa các câu lệnh SQL lồng nhau và các phép Join trên tập dữ liệu lớn rất tốt.
- **Tương thích mã nguồn mở:** Giúp tiết kiệm chi phí vận hành nhưng vẫn đảm bảo nhận được sự hỗ trợ mạnh mẽ từ cộng đồng nhà phát triển quốc tế.

Khả năng tích hợp mạnh mẽ với hệ sinh thái

PostgreSQL không chỉ là một kho lưu trữ dữ liệu tĩnh mà còn đóng vai trò là "điểm khởi đầu" (Source) linh hoạt trong các đường ống dữ liệu hiện đại:

- **Tích hợp với Apache Airflow:** Thông qua PostgresOperator và PostgresHook, PostgreSQL cho phép Airflow thực thi các truy vấn SQL điều khiển, kiểm tra trạng thái dữ liệu (Data Quality Checks) và phối hợp nhịp nhàng các tác vụ trích xuất dữ liệu một cách tự động.
- **Kết nối thông qua chuẩn JDBC/ODBC:** Đây là "cầu nối" quan trọng giúp các công cụ xử lý dữ liệu phân tán như Apache Spark có thể đọc dữ liệu từ Postgres theo kiểu song song. Trong dự án, Spark sử dụng JDBC để kết nối trực tiếp đến PostgreSQL, trích xuất hàng triệu bản ghi và chuyển đổi chúng thành Spark DataFrame để xử lý nhanh chóng trên bộ nhớ (In-memory).
- **Hỗ trợ CDC (Change Data Capture):** Với Logical Replication, PostgreSQL dễ dàng tích hợp với các công cụ như Debezium để bắt các thay đổi dữ liệu theo thời

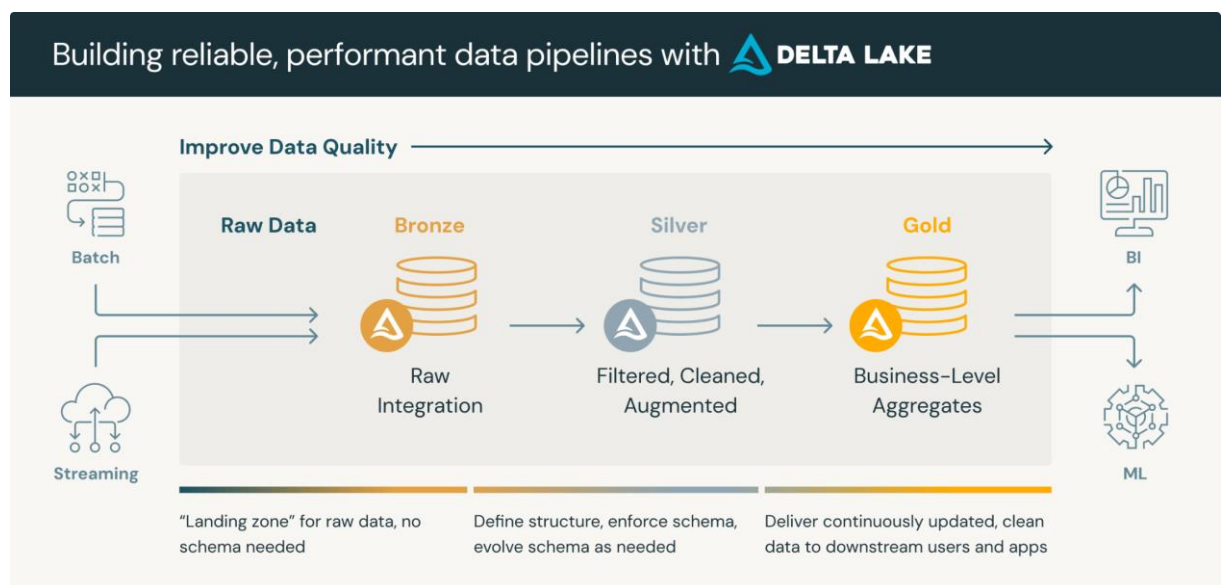
gian thực, giúp hệ thống Data Lakehouse luôn được cập nhật liên tục mà không gây áp lực lớn lên cơ sở dữ liệu nguồn.

Vai trò cốt lõi trong dự án hiện tại

Trong kiến trúc Medallion (Bronze - Silver - Gold) mà dự án đang triển khai, PostgreSQL đóng vai trò là lớp Source of Truth (Nguồn dữ liệu tin cậy):

- Lưu trữ dữ liệu Healthcare gốc: Đóng vai trò là hệ thống Operational Database lưu trữ các bảng dữ liệu cốt lõi như Patients, Encounters, và Conditions. Mọi quy trình phân tích đều bắt nguồn từ dữ liệu chuẩn xác tại bước này.
- Nguồn cấp cho quy trình Ingestion: PostgreSQL cung cấp giao diện truy xuất dữ liệu ổn định cho các script Python (sử dụng thư viện psycopg2) để đẩy dữ liệu thô vào vùng Raw Zone trên MinIO.
- Đảm bảo tính vẹn toàn tại điểm đầu: Các ràng buộc khóa ngoại (Foreign Keys) và kiểm tra miền trị (Check Constraints) của Postgres giữ vai trò như một bộ lọc chất lượng dữ liệu đầu tiên, giúp loại bỏ các sai sót logic trước khi dữ liệu được nạp vào Big Data pipeline, từ đó giảm thiểu đáng kể chi phí sửa lỗi ở các giai đoạn sau của Lakehouse.

1.2. Kiến trúc Medallion Lakehouse



Hình: Kiến trúc Medallion (nguồn [What is Medallion Architecture? | Databricks](#))

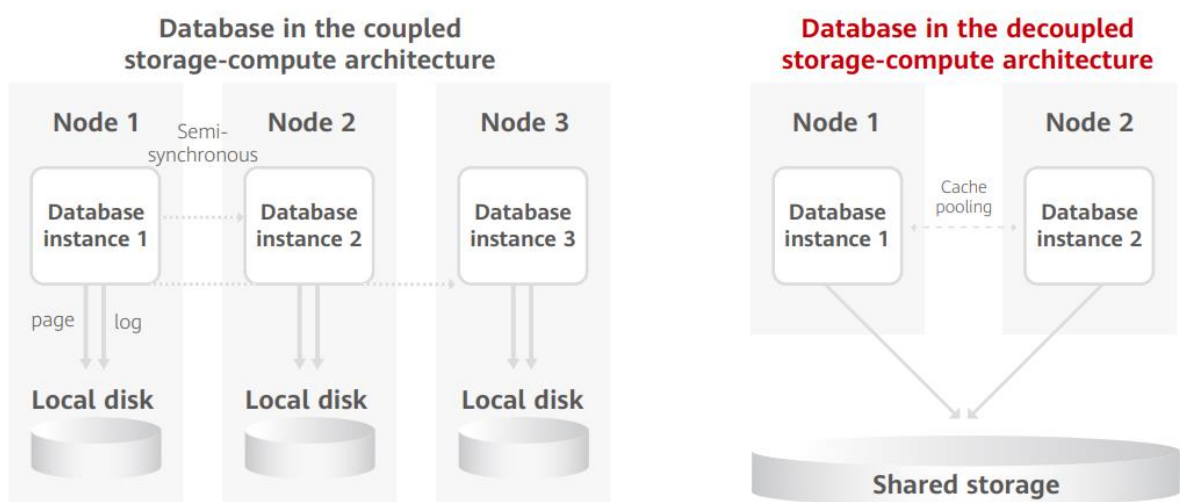
1.2.1 Tổng quan kiến trúc Medallion

Khi xây dựng hệ thống data lakehouse, nhóm báo cáo quyết định áp dụng kiến trúc medallion để tổ chức dữ liệu thành các tầng logic với chất lượng và cấu trúc được cải thiện dần theo từng bước. Thay vì đổ dồn mọi thứ vào một nơi duy nhất gây ra tình

trạng data swamp, kiến trúc này chia luồng xử lý thành ba lớp cơ bản là bronze, silver và gold.

Tầng bronze đóng vai trò là nơi hạ cánh của dữ liệu, lưu trữ nguyên bản mọi bản ghi từ hệ thống OLTP nguồn mà không qua bất kỳ sự can thiệp hay chỉnh sửa nào. Việc giữ lại dữ liệu thô giúp nhóm báo cáo luôn có một bản sao lưu an toàn để truy xuất nguồn gốc hoặc chạy lại pipeline nếu các bước xử lý phía sau gặp sự cố. Tiến sâu hơn vào hệ thống, dữ liệu từ bronze sẽ được kéo lên tầng silver. Đây là khu vực nhóm báo cáo thực hiện các tác vụ data cleansing, ép kiểu dữ liệu ngày tháng, loại bỏ các giá trị rỗng và thực hiện join các bảng rời rạc lại với nhau. Kết quả của tầng silver là một single source of truth, nơi dữ liệu bệnh nhân, lịch sử khám bệnh và các chẩn đoán lâm sàng được liên kết chặt chẽ, tạo nền tảng vững chắc cho việc phân tích. Cuối cùng, dữ liệu sạch từ silver sẽ được aggregate để đưa vào tầng gold. Tại đây, nhóm báo cáo xây dựng các bảng mang tính chất nghiệp vụ phục vụ trực tiếp cho bài toán y tế, giúp hệ thống dashboard và API có thể truy vấn ngay lập tức mà không cần tính toán lại từ đầu. Xuyên suốt ba tầng này, nhóm đồng bộ sử dụng định dạng Apache Iceberg để đảm bảo tính ACID, giúp quá trình đọc ghi đồng thời diễn ra an toàn.

1.2.2 Thiết kế kiến trúc Decoupled Storage & Compute



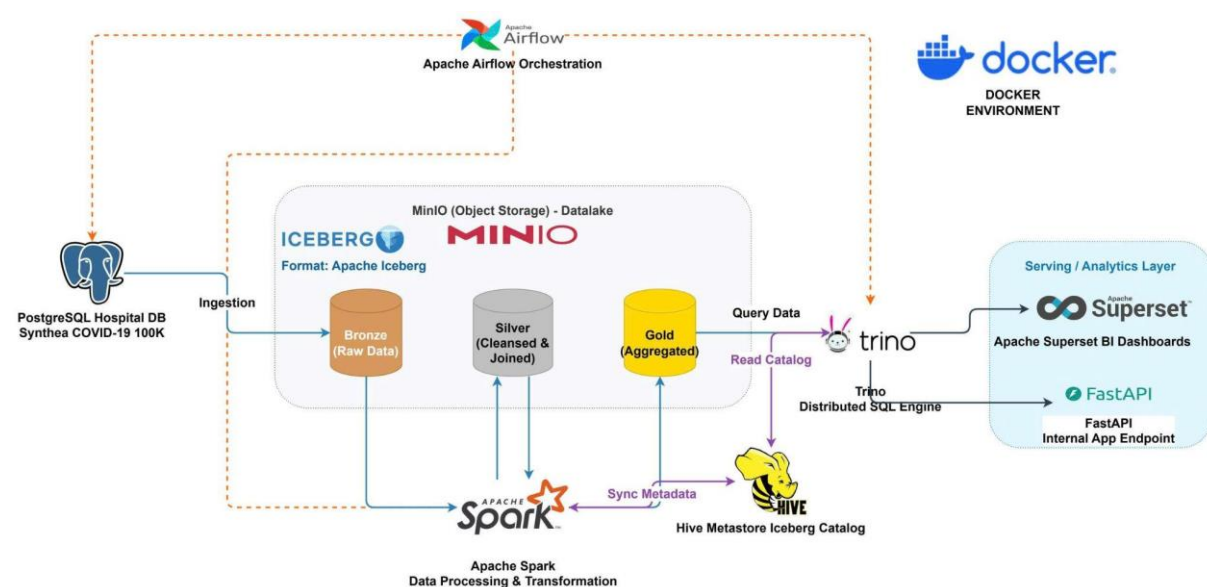
Hình: Kiến trúc Decoupled Storage & Compute (nguồn: [Decoupled Storage-Compute Architecture: The New De facto Standard for Distributed Databases](#))

Để vượt qua những hạn chế của các hệ thống data warehouse truyền thống nơi ổ cứng và vi xử lý bị gắn chặt với nhau, nhóm báo cáo đã triển khai mô hình decoupled storage and compute. Giải pháp này tách biệt hoàn toàn nơi lưu trữ dữ liệu vật lý khỏi các engine chịu trách nhiệm tính toán và truy vấn.

Trong dự án này, nhóm sử dụng MinIO làm object storage để chứa toàn bộ data files của cả ba tầng medallion. Đứng hoàn toàn độc lập với MinIO là các cụm compute engine bao gồm Apache Spark dành cho việc xử lý ETL nặng và Trino dành riêng cho việc chạy truy vấn SQL tốc độ cao[5]. Để các engine này biết được dữ liệu đang nằm ở đâu trên MinIO, nhóm thiết lập một Hive Metastore đóng vai trò làm catalog trung tâm lưu trữ metadata.

Việc thiết kế tách rời như vậy mang lại lợi thế cực lớn về mặt mở rộng hệ thống. Trong quá trình thực hành trên môi trường Docker local, nhóm báo cáo nhận thấy các tác vụ join hàng trăm ngàn dòng dữ liệu y tế tiêu tốn rất nhiều RAM. Nhờ cơ chế decoupled, nhóm dễ dàng giới hạn bộ nhớ cho Spark worker hoặc cấu hình tăng giảm số lượng node xử lý một cách độc lập mà không cần phải can thiệp vào cụm server chứa MinIO. Thiết kế này vừa giải quyết được triệt để bài toán tràn RAM cục bộ, vừa chứng minh được khả năng scale up linh hoạt của hệ thống nếu triển khai trên môi trường cloud thực tế.

1.2.3 Quy trình dòng chảy dữ liệu



Hình: Kiến trúc hệ thống triển khai

Để luồng dữ liệu di chuyển trơn tru qua các tầng kiến trúc, nhóm báo cáo đã xây dựng một pipeline tự động hóa hoàn toàn. Quá trình bắt đầu khi Airflow kích hoạt các DAG để thực hiện data ingestion, trích xuất dữ liệu từ cơ sở dữ liệu PostgreSQL giả lập của bệnh viện và nạp thẳng vào bucket raw trên MinIO.

Ngay sau khi dữ liệu thô đáp xuống, Airflow tiếp tục điều phối cụm Apache Spark khởi chạy job đầu tiên. Spark sẽ đọc các file từ thư mục raw và định dạng lại thành bảng Iceberg, chính thức đưa dữ liệu vào tầng bronze. Kế tiếp, một job Spark khác được trigger để kéo dữ liệu từ bronze lên. Tại bước này, nhóm báo cáo lập trình cho Spark thực hiện các phép lọc điều kiện, chuẩn hóa chuỗi và join các bảng thực thể y tế để ghi xuống tầng silver. Dựa trên nguồn dữ liệu sạch này, job Spark cuối cùng sẽ tính toán các chỉ số phức tạp như tỷ lệ tử vong hay số ngày nằm viện trung bình và lưu kết quả vào tầng gold.

Mỗi khi Spark thực hiện ghi dữ liệu ở bất kỳ tầng nào, nó đều đồng bộ cấu trúc bảng và vị trí file mới nhất vào Hive Metastore. Ở điểm cuối của dòng chảy, engine Trino sẽ đọc thông tin từ catalog này để biết chính xác cần quét những file nào trên MinIO. Nhờ đó, công cụ Apache Superset và hệ thống FastAPI do nhóm phát triển có thể kết nối trực tiếp vào Trino để render biểu đồ và cung cấp dữ liệu JSON cho người dùng cuối với độ trễ cực thấp.

Hệ thống áp dụng mô hình Medallion Architecture để tinh lọc dữ liệu dần dần qua 3 lớp:

Bước 1: Airflow kích hoạt tiến trình trích xuất dữ liệu từ PostgreSQL. Dữ liệu được lưu vào lớp Bronze (Raw Data) trên MinIO dưới định dạng Apache Iceberg. Đây là dữ liệu nguyên bản, chưa qua chỉnh sửa.

Bước 2: Apache Spark thực hiện đọc dữ liệu từ lớp Bronze. Tại đây, Spark tiến hành làm sạch (loại bỏ giá trị null, chuẩn hóa kiểu dữ liệu) và thực hiện các phép Join phức tạp giữa các bảng (Bệnh nhân, Hóa đơn, Chẩn đoán). Dữ liệu sau khi xử lý được ghi vào lớp Silver (Cleansed & Joined).

Bước 3: Spark tiếp tục đọc dữ liệu từ lớp Silver để tính toán các chỉ số kinh doanh (KPIs) như tổng doanh thu theo tháng, tỷ lệ mắc bệnh theo khu vực. Kết quả được lưu vào lớp Gold (Aggregated), sẵn sàng cho việc khai thác.

Bước 4: Khi Spark ghi dữ liệu (Bronze/Silver/Gold), nó đồng thời thực hiện Sync Metadata với Hive Metastore. Trino thực hiện Read Catalog từ Hive Metastore để biết cấu trúc bảng hiện tại, sau đó thực hiện Query Data trực tiếp từ lớp Gold trên MinIO để phục vụ cho Superset.

Về những ưu điểm nổi bật của kiến trúc,

1. Nhờ Apache Iceberg, hệ thống đảm bảo dữ liệu luôn chính xác ngay cả khi có nhiều tiến trình đọc/ghi diễn ra đồng thời.
2. Vì tách biệt giữa lưu trữ (MinIO) và tính toán (Spark/Trino), hệ thống có thể mở rộng tài nguyên tính toán độc lập khi khối lượng dữ liệu tăng lên mà không làm ảnh hưởng đến cấu trúc lưu trữ.
3. Sử dụng 100% các công cụ mã nguồn mở mạnh mẽ nhất, loại bỏ sự phụ thuộc vào các nhà cung cấp dịch vụ đám mây (Cloud Vendor Lock-in).
4. Trino kết hợp với định dạng Iceberg giúp việc truy vấn dữ liệu Big Data đạt tốc độ tương đương với các giải pháp thương mại đắt tiền.

1.2.4 Giới thiệu tập dữ liệu Synthea COVID-19

Dữ liệu được nhóm báo cáo sử dụng trong toàn bộ đề án là bộ Synthea COVID-19 100K do tổ chức MITRE phát triển. Đây là một tập dữ liệu synthetic mô phỏng lại lịch sử chăm sóc sức khỏe điện tử của hàng trăm ngàn bệnh nhân mắc COVID-19.

Bộ dữ liệu này được cộng đồng khoa học dữ liệu đánh giá rất cao vì nó tái hiện chính xác các quy luật thống kê y tế thực tế nhưng lại không chứa thông tin định danh thật, giúp nhóm báo cáo hoàn toàn tuân thủ các nguyên tắc bảo mật thông tin cá nhân. Mặc dù file tải về bao gồm hàng chục bảng khác nhau phản ánh mọi khía cạnh của một hệ thống bệnh viện, nhóm quyết định chỉ trích xuất những bảng mang lại giá trị lâm sàng cao nhất gồm patients, encounters, conditions và procedures.

Việc giới hạn phạm vi bảng này là một quyết định có chủ đích sau khi nhóm nghiên cứu kỹ tài liệu cấu trúc của MITRE. Cách làm này giúp giảm tải đáng kể áp lực xử lý cho hệ thống Docker local nhưng vẫn đảm bảo cung cấp đủ các trường dữ liệu thiết yếu. Dựa vào bộ khung dữ liệu này, nhóm báo cáo có đủ cơ sở để giải quyết trọn vẹn các bài toán phân tích tỷ lệ bệnh nhân phải vào phòng ICU, nhu cầu sử dụng máy thở, cũng như đánh giá tỷ lệ tử vong do COVID-19 phân bổ theo từng nhóm tuổi và giới tính.

CHƯƠNG 2: TRIỂN KHAI VÀ XÂY DỰNG HỆ THỐNG

2.1. Thiết lập hạ tầng và môi trường

2.1.1 Xây dựng hạ tầng container hóa với Docker Compose

Trong bối cảnh phát triển các hệ thống dữ liệu hiện đại, việc đảm bảo tính nhất quán giữa môi trường phát triển đặc biệt là khi phát triển một đồ án giữa nhiều thành viên, việc đồng bộ giữa các thiết bị và cấu hình cùng một môi trường phát triển đồng nhất là một thách thức rất lớn. Đối với dự án xây dựng hệ thống Data Lakehouse này, nhóm thực hiện quyết định lựa chọn Docker Compose làm công cụ chủ đạo để thiết lập và quản lý hạ tầng.

Lý do chính của sự lựa chọn này nằm ở khả năng đóng gói toàn bộ các thành phần phần mềm phức tạp từ hệ quản trị cơ sở dữ liệu, công cụ lập lịch đến các framework tính toán phân tán vào các container biệt lập. Điều này giúp loại bỏ hoàn toàn các lỗi phát sinh do sự khác biệt về thư viện hoặc cấu hình hệ điều hành cục bộ. Với Docker Compose, toàn bộ hệ sinh thái Lakehouse được định nghĩa dưới dạng mã, cho phép khởi tạo toàn bộ hạ tầng chỉ với một câu lệnh duy nhất, từ đó tối ưu hóa thời gian cấu hình và tăng khả năng tái sử dụng.

Hệ thống được *thiết kế theo kiến trúc multi-container*, trong đó mỗi dịch vụ đảm nhận một vai trò chuyên biệt trong vòng đời của dữ liệu. Các thành phần này không hoạt động đơn lẻ mà liên kết chặt chẽ với nhau thông qua một mạng ảo nội bộ, tạo thành một pipeline khép kín từ khâu thu thập Ingestion, lưu trữ Storage) quản lý Metadata đến xử lý Processing. Việc tách biệt các dịch vụ giúp hệ thống có tính module hóa cao, dễ dàng mở rộng hoặc thay thế từng thành phần mà không làm gián đoạn toàn bộ cấu trúc.

Các dịch vụ được nhóm tiến hành triển khai bao gồm: PostgreSQL, MinIO, MinIO Setup (mc), Hive Metastore và Metastore-DB, Apache Airflow, Spark Iceberg, Trino và Apache Superset. Để các thành phần riêng biệt có thể giao tiếp một cách ổn định, hạ tầng áp dụng ba cơ chế quản lý luồng thông tin chính gồm Docker Network, Điều phối phục thuộc và các biến môi trường Environment. Docker Network giúp toàn bộ các container được đặt chung trong một mạng bridge tùy chỉnh. Cơ chế này cho phép các dịch vụ nhận diện nhau thông qua Service Name, ví dụ Airflow kết nối đến MinIO bằng hostname minio thay vì IP, điều này giúp hệ thống linh hoạt và không phụ thuộc vào địa chỉ mạng tĩnh. Ta có thể thấy mọi dịch vụ đều được cấu hình chung một NetWork là lakehouse_net:

```
networks:
  lakehouse_net:
    name: lakehouse_net
```

Điều phối phụ thuộc giúp các dịch vụ có thứ tự khởi chạy ưu tiên. Ví dụ Hive Metastore chỉ được kích hoạt khi metastore-db đã đạt trạng thái healthy.

```
depends_on:
  metastore-db:
    condition: service_healthy
```

Việc sử dụng health check giúp kiểm soát chặt chẽ trạng thái sẵn sàng của các dịch vụ nền tảng trước khi dịch vụ phụ thuộc khởi động, tránh lỗi kết nối trong quá trình khởi động hệ thống. Và thành phần quan trọng nhất là các Environment Variables, các biến này giúp tinh chỉnh thông số cấu hình như thông tin đăng nhập, Endpoint URL và các tham số Spark được quản lý tập trung qua biến môi trường.

```
- DATABASE_HOST=metastore-db
  - DATABASE_PORT=5432
  - DATABASE_DB=${DB_METASTORE_NAME:-metastore_db}
  - DATABASE_USER=${DB_METASTORE_USER:-metastore}
  - DATABASE_PASSWORD=${DB_METASTORE_PASSWORD:-password}
  - DATABASE_TYPE=postgres
  - S3_BUCKET=hospital-lakehouse
  - S3_PREFIX=iceberg
  - S3_REGION=us-east-1
  - S3_ENDPOINT_URL=http://minio:9000
  - AWS_ACCESS_KEY_ID=${MINIO_ROOT_USER:-lakehouse_admin}
  - AWS_SECRET_ACCESS_KEY=${MINIO_ROOT_PASSWORD:-Lakehouse123!}
```

Điều này giúp tách biệt cấu hình khỏi mã nguồn, tăng cường tính bảo mật và dễ dàng điều chỉnh thông số hệ thống cho các môi trường khác nhau. Để tinh chỉnh các tham số này ta cần phải tham khảo cụ thể trên documentation của dịch vụ mà ta muốn sử dụng hoặc portal chính thức của docker hub.

Trong môi trường container, dữ liệu sẽ bị mất đi khi container bị xóa bỏ. Để giải quyết vấn đề này, dự án áp dụng cơ chế *Volume Mounting*. Ví dụ named volumes của airflow_data được sử dụng để lưu trữ trạng thái chạy và cấu hình của Airflow, đảm bảo dữ liệu vận hành không bị mất khi container được cập nhật hoặc khởi động lại. Một phần quan trọng của cơ chế này là bind mounts, giúp các thư mục cục bộ như ./dags, ./scripts, ./notebooks và ./data được gắn trực tiếp vào bên trong container. Cơ chế này cho phép chỉnh sửa mã nguồn hoặc nạp dữ liệu từ máy host và các thay đổi sẽ có hiệu lực ngay lập tức bên trong môi trường container, tạo điều kiện thuận lợi cho quá trình phát triển và thử nghiệm nhanh.

volumes:

- airflow_data:/opt/airflow
- ./dags:/opt/airflow/dags
- ./scripts:/opt/spark/scripts
- ../jars:/opt/spark/jars
- ../data:/data_source

Tóm lại việc áp dụng Docker Compose mang lại những lợi ích chiến lược cho hệ thống Data Lakehouse mang lại ba lợi ích chính. Đầu tiên là tính đồng nhất khi đảm bảo mọi thành viên trong nhóm dự án đều làm việc trên một môi trường giống hệt nhau, loại bỏ xung đột về phiên bản phần mềm. Thứ hai là dễ dàng triển khai vì hệ thống có thể được di chuyển và triển khai trên các máy chủ khác hoặc trên môi trường một cách nhanh chóng mà không cần cài đặt lại từng thành phần thủ công. Cuối cùng là khả năng mở rộng và cô lập: Mỗi dịch vụ chạy trong không gian riêng, lỗi xảy ra ở một container ví dụ Spark sẽ không làm ảnh hưởng trực tiếp đến trạng thái của các container khác như PostgreSQL, giúp tăng tính ổn định của toàn hệ thống.

Trong hệ thống Data Lakehouse của nhóm, mỗi container được cấu hình với các tham số chuyên biệt như trình bày ở trên nhằm đảm bảo tính toàn vẹn của dữ liệu và

hiệu suất xử lý. Dưới đây là phân tích chi tiết cho từng dịch vụ trong tệp cấu hình hạ tầng.

Đầu tiên, PostgreSQL cơ sở dữ liệu nguồn Dịch vụ này đóng vai trò là điểm khởi đầu của mọi luồng dữ liệu. Các biến `POSTGRES_USER`, `POSTGRES_PASSWORD` và `POSTGRES_DB` được thiết lập để khởi tạo một schema mặc định tên là `hospital_db`. Việc sử dụng các giá trị mặc định có khả năng ghi đè qua tệp `.env` giúp linh hoạt trong việc thay đổi định danh mà không cần build lại image. Cổng kết nối: Ánh xạ cổng `5432:5432` cho phép các công cụ quản trị từ máy host như DBeaver hoặc pgAdmin truy cập trực tiếp để kiểm tra dữ liệu nguồn trong quá trình phát triển. Việc mount tệp `postgres_init.sql` vào thư mục `/docker-entrypoint-initdb.d/` là một bước quan trọng, giúp tự động hóa việc khởi tạo bảng và dữ liệu mẫu ngay khi container vừa chạy, đảm bảo tính nhất quán cho các bài thử nghiệm. Cuối cùng, Container này cung cấp dữ liệu cho Airflow thông qua mạng nội bộ `lakehouse_net`.

```
postgres-source:
  image: postgres:15-alpine
  container_name: postgres-source
  environment:
    - POSTGRES_USER=${DB_SOURCE_USER:-admin}
    - POSTGRES_PASSWORD=${DB_SOURCE_PASSWORD:-password}
    - POSTGRES_DB=${DB_SOURCE_NAME:-hospital_db}
  networks:
    - lakehouse_net
  ports:
    - "5432:5432"
  volumes:
    - ./postgres_init.sql:/docker-entrypoint-initdb.d/init.sql
    - ../data:/data_source
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_SOURCE_USER:-admin} -d ${DB_SOURCE_NAME:-hospital_db}"]
    interval: 10s
```

```
timeout: 5s
retries: 5
```

Thứ hai, MinIO là hạt nhân của tầng lưu trữ ở tầng Storage Layer, đóng vai trò thay thế cho các dịch vụ Cloud Storage như AWS S3. Các biến MINIO_ROOT_USER và PASSWORD thiết lập quyền quản trị cao nhất. Đây là thông tin then chốt mà tất cả các dịch vụ khác (mc, Hive, Spark, Airflow) đều phải sử dụng để truy cập dữ liệu. Hệ thống mở hai cổng riêng biệt: 9000 dành cho các yêu cầu API (S3 API) từ Spark/Airflow và 9001 dành cho giao diện quản trị Web (Console). Tương tác: Nó đóng vai trò là "nhà kho" trung tâm. Mọi dịch vụ trong hệ thống đều phải cấu hình Endpoint URL trở về `http://minio:9000` để thực hiện các thao tác đọc/ghi dữ liệu.

```
minio:
  image: minio/minio:latest
  container_name: minio
  environment:
    - MINIO_ROOT_USER=${MINIO_ROOT_USER:-lakehouse_admin}
    - MINIO_ROOT_PASSWORD=${MINIO_ROOT_PASSWORD:-Lakehouse123!}
  networks:
    - lakehouse_net
  ports:
    - "9000:9000"
    - "9001:9001"
  command: server /data --console-address ":9001"
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
    interval: 10s
    timeout: 5s
    retries: 5
```


Thứ ba, MinIO Client tức mc sẽ khởi tạo hạ tầng lưu trữ. Đây không phải là một dịch vụ chạy nền mà là một tác vụ chạy một lần để cấu hình cho MinIO. Cấu hình condition: service_healthy đối với minio là bắt buộc. Điều này đảm bảo rằng các lệnh tạo bucket chỉ thực hiện khi server MinIO đã sẵn sàng nhận kết nối, tránh các lỗi "Connection Refused". Vai trò trong hệ thống: Thông qua các lệnh trong entryptpoint, dịch vụ này tự động xây dựng cấu trúc thư mục chuẩn cho Lakehouse bao gồm dữ liệu thô và warehouse dữ liệu đã qua xử lý theo định dạng Iceberg, giúp quy trình triển khai hoàn toàn tự động.

```
mc:
  image: minio/mc:latest
  container_name: mc
  depends_on:
    minio:
      condition: service_healthy
  networks:
    - lakehouse_net
  environment:
    - MINIO_ROOT_USER=${MINIO_ROOT_USER:-lakehouse_admin}
    - MINIO_ROOT_PASSWORD=${MINIO_ROOT_PASSWORD:-Lakehouse123!}
  entrypoint: >
    /bin/sh -c "
      /usr/bin/mc      alias      set      local      http://minio:9000
      ${MINIO_ROOT_USER} ${MINIO_ROOT_PASSWORD};
      /usr/bin/mc mb -p local/hospital-lakehouse;
      /usr/bin/mc mb -p local/hospital-lakehouse/raw;
      /usr/bin/mc mb -p local/hospital-lakehouse/warehouse;
      exit 0;
    "
```

Thứ tư, Hive Metastore & Metastore-DB giải quyết bài toán Schema-on-read của kiến trúc Lakehouse bằng cách quản lý thông tin về các bảng dữ liệu. Một loạt các biến S3_ENDPOINT_URL, S3_BUCKET, và thông tin AWS Credentials được thiết lập để Hive Metastore biết nơi lưu trữ metadata của các bảng Apache Iceberg trên MinIO. Phụ thuộc: hive-metastore phụ thuộc trực tiếp vào metastore-db (PostgreSQL). Đây là mối quan hệ mật thiết vì Metastore không tự lưu dữ liệu mà cần một cơ sở dữ liệu quan hệ để duy trì các bản ghi về bảng và schema. Nó mở cổng 9083 sử dụng giao thức Thrift. Đây là cửa ngõ để Spark Iceberg có thể tra cứu thông tin về bảng trước khi thực hiện các truy vấn dữ liệu thực tế.

4. Hive Metastore - Quản lý Metadata cho Iceberg

```
metastore-db:
  image: postgres:15-alpine
  container_name: metastore-db
  environment:
    - POSTGRES_USER=${DB_METASTORE_USER:-metastore}
    - POSTGRES_PASSWORD=${DB_METASTORE_PASSWORD:-password}
    - POSTGRES_DB=${DB_METASTORE_NAME:-metastore_db}
  networks:
    - lakehouse_net
  healthcheck:
    test: ["CMD", "pg_isready", "-U", "metastore", "-d",
"metastore_db"]
    interval: 5s
    timeout: 5s
    retries: 5

hive-metastore:
  image: tabulario/hive-metastore:latest
```

```

container_name: hive-metastore
depends_on:
  metastore-db:
    condition: service_healthy
environment:
  - DATABASE_HOST=metastore-db
  - DATABASE_PORT=5432
  - DATABASE_DB=${DB_METASTORE_NAME:-metastore_db}
  - DATABASE_USER=${DB_METASTORE_USER:-metastore}
  - DATABASE_PASSWORD=${DB_METASTORE_PASSWORD:-password}
  - DATABASE_TYPE=postgres
  - S3_BUCKET=hospital-lakehouse
  - S3_PREFIX=iceberg
  - S3_REGION=us-east-1
  - S3_ENDPOINT_URL=http://minio:9000
  - AWS_ACCESS_KEY_ID=${MINIO_ROOT_USER:-lakehouse_admin}
  - AWS_SECRET_ACCESS_KEY=${MINIO_ROOT_PASSWORD:-Lakehouse123!}
networks:
  - lakehouse_net
ports:
  - "9083:9083"

```

Thứ năm, Apache Airflow là bộ não điều khiển toàn bộ quy trình ETL/ELT trong hệ thống. Cấu hình Biến môi trường: Điểm đặc biệt là các biến AIRFLOW_CONN_.... Chúng được cấu hình sẵn dưới dạng chuỗi kết nối chuẩn, giúp Airflow tự động nhận diện các kết nối tới Postgres và S3 ngay khi khởi động mà không cần cấu hình thủ công qua giao diện UI. Việc mount thư mục ./dags và ./scripts cho phép đồng bộ hóa mã nguồn pipeline từ máy host vào container. Điều này cực kỳ quan trọng cho phép viết mã và kiểm thử DAGs một cách liên tục. Airflow đóng vai trò điều phối, nó kết nối tới postgres-source để lấy dữ liệu, sử dụng Spark để xử lý và ghi kết quả cuối cùng vào MinIO.

```

airflow:
  build: .
  container_name: airflow
  depends_on:
    postgres-source:
      condition: service_healthy
    minio:
      condition: service_healthy
  environment:
    - AIRFLOW__CORE__EXECUTOR=SequentialExecutor
    - AIRFLOW__CORE__LOAD_EXAMPLES=False
    - AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION=True
    # Cấu hình tài khoản Admin cố định
    - _AIRFLOW_WWW_USER_USERNAME=admin
    - _AIRFLOW_WWW_USER_PASSWORD=password
    - _AIRFLOW_WWW_USER_ROLE=Admin
    - _AIRFLOW_WWW_USER_EMAIL=admin@example.com
    # Auto-connections
    - AIRFLOW_CONN_POSTGRES_DEFAULT=postgres://${DB_SOURCE_USER:-
admin}:${DB_SOURCE_PASSWORD:-password}@postgres-
source:5432/${DB_SOURCE_NAME:-hospital_db}
    -
    AIRFLOW_CONN_SPARK_DEFAULT=spark://local?master=local%5B*%5D
    - AIRFLOW_CONN_AWS_DEFAULT=aws://${MINIO_ROOT_USER:-
lakehouse_admin}:${MINIO_ROOT_PASSWORD:-
Lakehouse123!}@?endpoint_url=http%3A%2F%2Fminio%3A9000
    # Lakehouse Config
    - PG_HOST=postgres-source
    - PG_DB=${DB_SOURCE_NAME:-hospital_db}
    - PG_USER=${DB_SOURCE_USER:-admin}
    - PG_PASSWORD=${DB_SOURCE_PASSWORD:-password}
    - MINIO_ENDPOINT=http://minio:9000
    - MINIO_ACCESS_KEY=${MINIO_ROOT_USER:-lakehouse_admin}

```

```
- MINIO_SECRET_KEY=${MINIO_ROOT_PASSWORD:-Lakehouse123!}
- AWS_ACCESS_KEY_ID=${MINIO_ROOT_USER:-lakehouse_admin}
- AWS_SECRET_ACCESS_KEY=${MINIO_ROOT_PASSWORD:-Lakehouse123!}
networks:
- lakehouse_net
volumes:
- airflow_data:/opt/airflow
- ./dags:/opt/airflow/dags
- ./scripts:/opt/spark/scripts
- ../jars:/opt/spark/jars
- ../data:/data_source
ports:
- "8080:8080"
command: standalone
```

Thứ sáu, Spark Iceberg cung cấp sức mạnh tính toán và khả năng quản lý bảng dữ liệu theo chuẩn Iceberg. Cấu hình Biến môi trường: Tập trung vào các tham số SPARK_CONF_.... Các cấu hình này cực kỳ chi tiết, chỉ định rõ ràng việc sử dụng HiveCatalog, địa chỉ của Hive Metastore (thrift://hive-metastore:9083) và cấu hình S3A để tương tác với MinIO. Tại sao cần cấu hình như vậy: Việc thiết lập spark_sql_catalog_hospital cho phép người dùng thực hiện các câu lệnh SQL trực tiếp trên các bảng Iceberg như thể đang làm việc với một database quan hệ truyền thống, mang lại tính năng ACID và Time Travel cho Data Lake. Cổng 8888 mở ra giao diện Jupyter Notebook, tạo môi trường tương tác để thực hiện phân tích dữ liệu hoặc viết code xử lý dữ liệu Spark một cách trực quan. Spark là dịch vụ tiêu thụ nhiều metadata nhất từ Hive Metastore và thực hiện các thao tác I/O nặng nhất trên MinIO.

```
spark-iceberg:
  image: tabulario/spark-iceberg:latest
  container_name: spark-iceberg
```

```

depends_on:
  - minio
  - hive-metastore
environment:
  - AWS_ACCESS_KEY_ID=${MINIO_ROOT_USER:-lakehouse_admin}
  - AWS_SECRET_ACCESS_KEY=${MINIO_ROOT_PASSWORD:-Lakehouse123!}
  - AWS_REGION=us-east-1
  -
SPARK_CONF_spark_sql_catalog_hospital=org.apache.iceberg.spark.SparkCatalog
  - SPARK_CONF_spark_sql_catalog_hospital_catalog-impl=org.apache.iceberg.hive.HiveCatalog
  - SPARK_CONF_spark_sql_catalog_hospital_uri=thrift://hive-metastore:9083
  -
SPARK_CONF_spark_sql_catalog_hospital_warehouse=s3a://hospital-lakehouse/warehouse
  - SPARK_CONF_spark_hadoop_fs_s3a_endpoint=http://minio:9000
  - SPARK_CONF_spark_hadoop_fs_s3a_path_style_access=true
networks:
  - lakehouse_net
ports:
  - "8888:8888"
  - "4040:4040"
volumes:
  - ./notebooks:/home/iceberg/notebooks

```

Kết luận lại, sự phối hợp giữa các cấu hình này tạo nên một hệ sinh thái đồng nhất. Mạng lưới lakehouse_net gắn kết các dịch vụ, trong khi cơ chế volumes và environment variables đảm bảo tính linh hoạt và khả năng bảo toàn dữ liệu cho toàn bộ kiến trúc Data Lakehouse.

2.1.2 Cài đặt và cấu hình PostgreSQL Source Database

Việc thiết lập cơ sở dữ liệu nguồn PostgreSQL được thực hiện qua hai giai đoạn: Giai đoạn thiết lập thủ công để kiểm chứng mô hình dữ liệu (DDL) - ở bước này nhóm em thực hiện thủ công các việc từ tạo cấu trúc của các bảng và đẩy các dữ liệu lên và Giai đoạn tự động hóa để chuẩn bị cho việc tích hợp vào đường ống dữ liệu (Data Pipeline). Nhóm thực hiện 2 cách như trên vì, ở cách 1 thì có thể giúp làm quen hơn với các thao tác trên PostgreSQL và hiểu hơn về cách sử dụng nó và cách 2 được thực hiện vì nhu cầu thực tế cần để cho việc tự động, thực thi các dags của airflow

Link chi tiết step-by-step thực hiện từng bước như đính kèm:

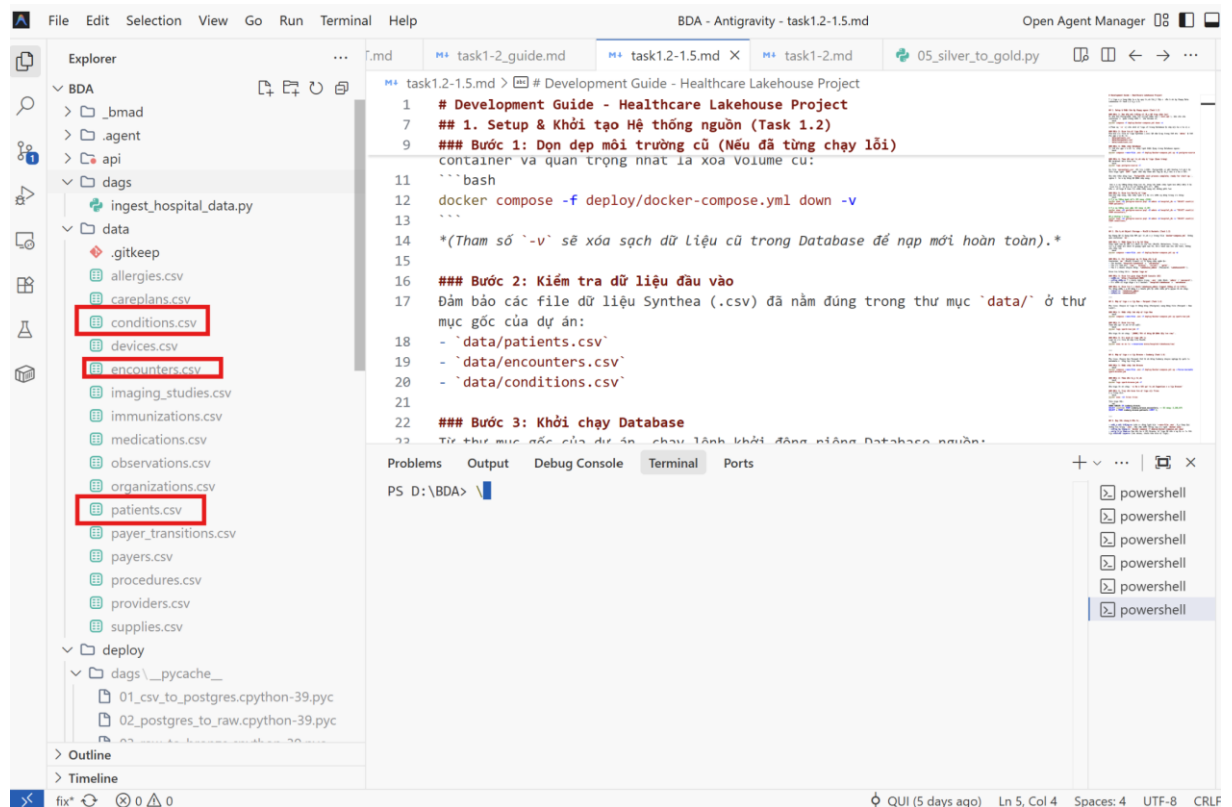
https://github.com/darktheDE/healthcare-lakehouse-covid19/blob/develop/docs/task1-2_guide.md

Nội dung thực hiện như sau:

Phương án 1: Thiết lập và nạp dữ liệu thủ công (Manual Setup)

Phương án này giúp kiểm soát chặt chẽ kiểu dữ liệu và thứ tự nạp của các bảng có ràng buộc khóa ngoại.

- Kiểm tra các file data source



- Thực hiện login vào PostgreSQL terminal

```
docker exec -it postgres-source psql -U admin -d hospital_db
```

```
PS D:\BDA> docker exec -it postgres-source psql -U admin -d hospital_db
psql (15.17)
Type "help" for help.

hospital_db=# docker exec -it postgres-source psql -U admin -d hospital_db
hospital_db=#
```

- Khởi tạo cấu trúc bảng (DDL): Truy cập vào container và thực thi mã SQL để tạo các bảng với kiểu dữ liệu chuẩn xác nhằm tương thích với file CSV từ hệ thống Synthea (ví dụ: dùng UUID cho các mã định danh, TIMESTAMPTZ cho thời gian và NUMERIC cho chi phí y tế).

(Thực thi các lệnh CREATE TABLE cho patients, encounters, conditions).

```
CREATE TABLE patients (
    id UUID PRIMARY KEY,
    birthdate DATE NOT NULL,
    deathdate DATE,
    ssn VARCHAR(20),
    drivers VARCHAR(20),
    passport VARCHAR(20),
    prefix VARCHAR(20),
    first VARCHAR(100),
    last VARCHAR(100),
    suffix VARCHAR(20),
    maiden VARCHAR(100),
    marital VARCHAR(1),
    race VARCHAR(50),
    ethnicity VARCHAR(50),
    gender VARCHAR(1),
    birthplace VARCHAR(255),
    address VARCHAR(255),
    city VARCHAR(100),
    state VARCHAR(100),
    county VARCHAR(100),
```



```

        zip VARCHAR(20),
        lat NUMERIC(10, 6),
        lon NUMERIC(10, 6),
        healthcare_expenses NUMERIC(12, 2),
        healthcare_coverage NUMERIC(12, 2)
    );

-- Tạo bảng encounters
CREATE TABLE encounters (
    id UUID PRIMARY KEY,
    start_time TIMESTAMPTZ NOT NULL,
    stop_time TIMESTAMPTZ,
    patient UUID REFERENCES patients(id),
    organization UUID,
    provider UUID,
    payer UUID,
    encounterclass VARCHAR(50),
    code VARCHAR(50),
    description VARCHAR(255),
    base_encounter_cost NUMERIC(12, 2),
    total_claim_cost NUMERIC(12, 2),
    payer_coverage NUMERIC(12, 2),
    reasoncode VARCHAR(50),
    reasondescription VARCHAR(255)
);

-- Tạo bảng conditions
CREATE TABLE conditions (
    start_date DATE NOT NULL,
    stop_date DATE,
    patient UUID REFERENCES patients(id),
    encounter UUID REFERENCES encounters(id),
    code VARCHAR(50),

```

description VARCHAR(255)

);

Problems Output Debug Console Terminal Ports

PS D:\BDA> docker exec -it postgres-source psql -U admin -d hospital_db

```
hospital_db=# CREATE TABLE patients (  
    id UUID PRIMARY KEY,  
    birthdate DATE NOT NULL,  
    deathdate DATE,  
    ssn VARCHAR(20),  
    drivers VARCHAR(20),  
    passport VARCHAR(20),  
    prefix VARCHAR(20),  
    first VARCHAR(100),  
    last VARCHAR(100),  
    suffix VARCHAR(20),  
    maiden VARCHAR(100),  
    marital VARCHAR(1),  
    race VARCHAR(50),  
    ethnicity VARCHAR(50),  
    gender VARCHAR(1),  
    birthplace VARCHAR(255),  
); healthcare_coverage NUMERIC(12, 2),  
CREATE TABLE  
hospital_db=#
```

```
hospital_db=# CREATE TABLE encounters (  
    id UUID PRIMARY KEY,  
    start_time TIMESTAMPTZ NOT NULL,  
    stop_time TIMESTAMPTZ,  
    patient UUID REFERENCES patients(id),  
    organization UUID,  
    provider UUID,  
    payer UUID,  
    encounterclass VARCHAR(50),  
    code VARCHAR(50),  
    description VARCHAR(255),  
    base_encounter_cost NUMERIC(12, 2),  
    total_claim_cost NUMERIC(12, 2),  
    payer_coverage NUMERIC(12, 2),  
    reasoncode VARCHAR(50),  
    reasondescription VARCHAR(255)  
);  
CREATE TABLE
```

```

        reasondescription VARCHAR(255)
    );
CREATE TABLE
hospital_db=# CREATE TABLE conditions (
        start_date DATE NOT NULL,
        stop_date DATE,
        patient UUID REFERENCES patients(id),
        encounter UUID REFERENCES encounters(id),
        code VARCHAR(50),
        description VARCHAR(255)
    );
CREATE TABLE
hospital_db=# 

```

- Đưa dữ liệu vào Container: Sử dụng lệnh docker cp để sao chép các file CSV từ máy vật lý vào thư mục /tmp của container PostgreSQL:

```

docker cp data/patients.csv postgres-source:/tmp/patients.csv
docker cp data/encounters.csv postgres-source:/tmp/encounters.csv
docker cp data/conditions.csv postgres-source:/tmp/conditions.csv

```

```

Problems  Output  Debug Console  Terminal  Ports

● PS D:\BDA> docker cp data/patients.csv postgres-source:/tmp/patients.csv
  Successfully copied 35.4MB to postgres-source:/tmp/patients.csv
  PS D:\BDA> docker cp data/encounters.csv postgres-source:/tmp/encounters.csv
● Successfully copied 989MB to postgres-source:/tmp/encounters.csv
● PS D:\BDA> docker cp data/conditions.csv postgres-source:/tmp/conditions.csv
  Successfully copied 141MB to postgres-source:/tmp/conditions.csv
○ PS D:\BDA> 

```

- Nạp dữ liệu bằng lệnh COPY: Sử dụng lệnh COPY của PostgreSQL để nạp dữ liệu từ file CSV vào các bảng đã tạo. Quy trình này đòi hỏi phải nạp bảng patients trước, sau đó mới đến các bảng phụ thuộc như encounters và conditions để không vi phạm ràng buộc khóa ngoại.

```

COPY patients FROM '/tmp/patients.csv' DELIMITER ',' CSV HEADER;
COPY encounters FROM '/tmp/encounters.csv' DELIMITER ',' CSV HEADER;
COPY conditions FROM '/tmp/conditions.csv' DELIMITER ',' CSV HEADER;

```

```

hospital_db=#
hospital_db=# COPY patients FROM '/tmp/patients.csv' DELIMITER ',' CSV HEADER;
COPY 124150
hospital_db=# COPY encounters FROM '/tmp/encounters.csv' DELIMITER ',' CSV HEADER;
COPY 3188675
hospital_db=# COPY conditions FROM '/tmp/conditions.csv' DELIMITER ',' CSV HEADER;
COPY 1143900
hospital_db=# 

```

Kiểm tra nếu hiện có số dòng là thành công

```
PS D:\BDA\deploy> docker exec -it postgres-source psql -U admin -d hospital_db -c "SELECT count(*) AS total_patients FROM patients;"
total_patients
-----
124150
(1 row)
```

Phương án 2: Tự động hóa với Docker Compose và Init Script

Để tối ưu hóa thời gian và đảm bảo tính nhất quán khi triển khai lại hệ thống, toàn bộ quy trình trên được tự động hóa.

- Sử dụng Script khởi tạo (postgres_init.sql): Tất cả các lệnh DDL và COPY được tập hợp vào file postgres_init.sql. File này được ánh xạ (volume mapping) vào thư mục /docker-entrypoint-initdb.d/ trong container. Khi container khởi động lần đầu, Docker sẽ tự động thực thi toàn bộ script này.
- Ánh xạ dữ liệu trực tiếp: Thay vì dùng docker cp, thư mục dữ liệu nguồn được ánh xạ trực tiếp thông qua Docker Compose. Dịch vụ PostgreSQL được định nghĩa trong file docker-compose.yml sử dụng hình ảnh (image) chính thức postgres:15-alpine để tối ưu hóa dung lượng.

```
postgres-source:
  image: postgres:15-alpine
  container_name: postgres-source
  environment:
    - POSTGRES_USER=${DB_SOURCE_USER:-admin}
    - POSTGRES_PASSWORD=${DB_SOURCE_PASSWORD:-password}
    - POSTGRES_DB=${DB_SOURCE_NAME:-hospital_db}
  networks:
    - lakehouse_net
  ports:
    - "5432:5432"
  volumes:
    - ./postgres_init.sql:/docker-entrypoint-initdb.d/init.sql
    - ../data:/data_source
```

Giải thích cấu hình:

Environment: Thiết lập các thông tin đăng nhập và tên database mặc định (admin/password/hospital_db).

Ports: Ánh xạ cổng 5432 từ container ra máy chủ để các công cụ bên ngoài (như DBeaver) có thể truy cập.

Volumes:

Ánh xạ file postgres_init.sql vào thư mục đặc biệt /docker-entrypoint-initdb.d/. Docker sẽ tự động thực thi file này khi container được tạo lần đầu tiên.

Ánh xạ thư mục dữ liệu nguồn (../data) để phục vụ quy trình Ingestion.

Cấu hình Healthcheck

Để đảm bảo trật tự khởi động trong hệ thống pipeline, PostgreSQL được cấu hình cơ chế tự kiểm tra trạng thái (Healthcheck). Các dịch vụ phía sau (như Airflow hay Spark) sẽ chỉ được khởi động khi PostgreSQL đã sẵn sàng chấp nhận kết nối.

healthcheck:

test: ["CMD-SHELL", "pg_isready -U admin -d hospital_db"]

interval: 10s

timeout: 5s

retries: 5

Xây dựng nền tảng cho Pipeline tự động (Ingestion Scripts)

- Để chuẩn bị cho việc điều phối bằng Apache Airflow, em đã phát triển các file script Python/PySpark chuyên dụng (ingest_patients.py, ingest_conditions.py,...).
- Nhiệm vụ của Ingest Scripts:

Mapping & Transform: Tự động đổi tên các cột từ định dạng CSV (BIRTHDATE, FIRST, LAST) sang định dạng chuẩn trong database (birthdate, first, last).

Sử dụng thư viện psycopg2 & JDBC: Dùng psycopg2 để thực hiện các lệnh dọn dẹp bảng (TRUNCATE CASCADE) và dùng Spark JDBC để đẩy dữ liệu lớn vào Postgres một cách nhanh chóng.

Mục tiêu phối hợp: Các file script này được thiết kế để Airflow có thể gọi thực thi thông qua BashOperator hoặc SparkSubmitOperator. Việc tách rời script ingestion

giúp hệ thống linh hoạt hơn: người sau chỉ cần viết file DAG để gọi các script này mà không cần quan tâm đến logic xử lý dữ liệu bên trong.

2.1.3 Cấu hình Datalake với MinIO và Hive Metastore

Quy trình thiết lập hệ thống lưu trữ đối tượng và quản lý danh mục dữ liệu được thực hiện qua hai giai đoạn: thiết lập thủ công (để kiểm duyệt giao diện và quyền truy cập) và thiết lập tự động hóa (để triển khai nhanh chóng và đồng bộ).

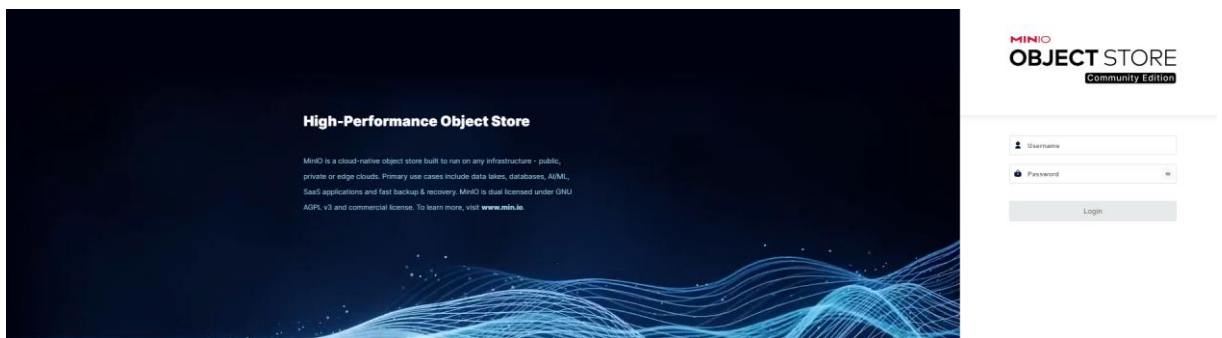
A. Thiết lập MinIO Object Storage (Lớp lưu trữ)

Hệ thống cung cấp hai phương thức để khởi tạo vùng chứa dữ liệu:

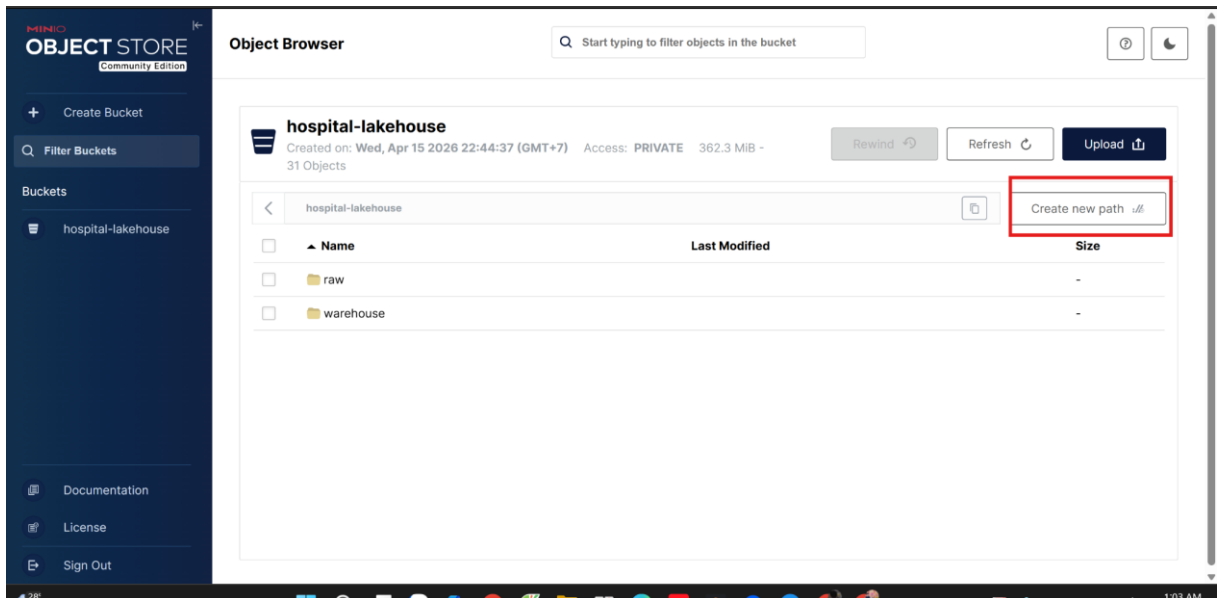
Phương thức thiết lập thủ công (Manual Setup):

Phù hợp để làm quen với giao diện quản trị và kiểm tra các tính năng trực quan.

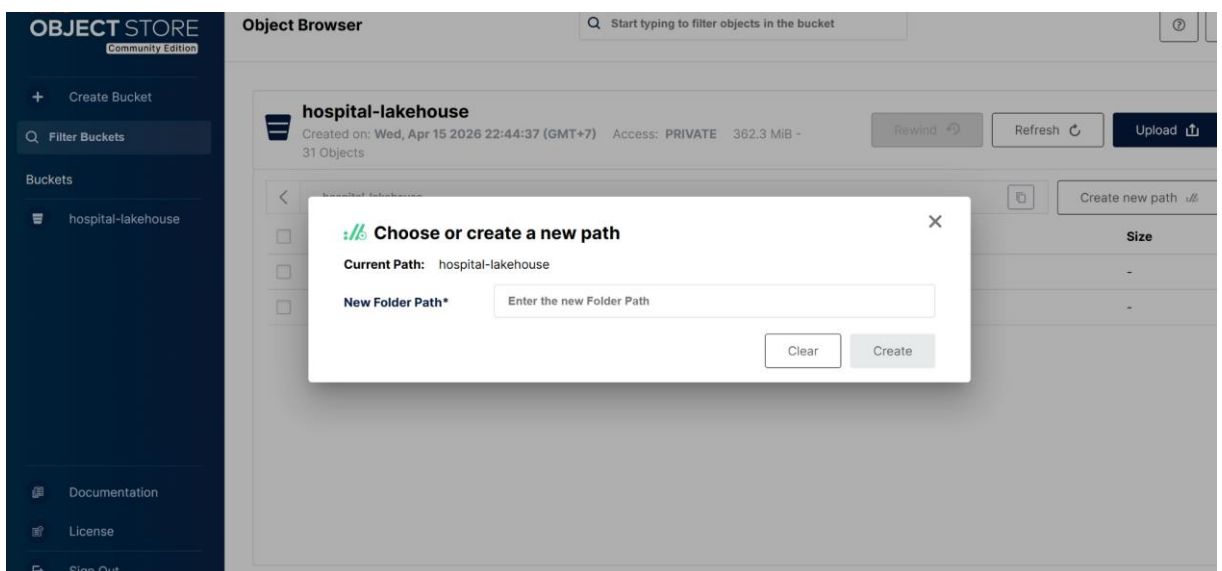
- Truy cập giao diện: Sử dụng trình duyệt tới địa chỉ <http://localhost:9001>, đăng nhập bằng tài khoản Admin (mặc định trong bài lab là admin/password).



- Khởi tạo Bucket: Tạo Bucket chính với tên hospital-lakehouse.
- Tạo cấu trúc thư mục (Prefix): Bên trong bucket, tạo lần lượt các thư mục ảo tương ứng với kiến trúc Medallion: raw/, bronze/, silver/, và gold/.
- Các bước thực hiện chi tiết:
 - 1. Bấm “Create Bucket”, nhập tên là ‘hospital-lakehouse’ rồi khởi tạo.



- 2. Tại Object Browser, điều hướng vào `hospital-lakehouse`. Nhấn nút “Create new path” để tạo lần lượt các folders (prefix) ảo: `raw/`, `bronze/`, `silver/`, `gold/`.



- Khi Prefix của MinIO chưa chứa tệp nào, nó có thể bị UI tự động ẩn đi. Bạn có thể up 1 tệp nào đó thử vào trong thư mục đó để chúng hiện rõ. Còn không thì khi thực thi các task nó cũng sẽ hiển thị lên

Phương thức tự động hóa (Automated via Docker):

Dự án được cấu hình để tự động hóa toàn bộ phần trên thông qua container mc (MinIO Client) trong file docker-compose.yml. Khi gọi lệnh docker compose up -d, container này sẽ tự thực thi:

- Kết nối và kiểm tra trạng thái dịch vụ MinIO.

- Tự động tạo bucket hospital-lakehouse và các phân vùng dữ liệu.
- Tạo tài khoản chuyên dụng với định danh cố định để sử dụng cho toàn hệ thống:
Access Key: lakehouse_admin
Secret Key: Lakehouse123!
Default Region: us-east-1

Thiết lập Hive Metastore và Catalog (Lớp danh mục)

Sau khi lớp lưu trữ đã sẵn sàng, quy trình cấu hình Hive Metastore (HMS) được thực hiện để quản lý Schema của Apache Iceberg:

- Khởi chạy Database cho Metastore: Hệ thống khởi chạy một container PostgreSQL riêng biệt (metastore-db) đóng vai trò là "nơi ghi nhớ" toàn bộ cấu trúc bảng. HMS sẽ kết nối vào database này thông qua driver JDBC để lưu trữ thông tin về Snapshot và vị trí file trên MinIO.
- Cấu hình HMS Service: Container Hive Metastore được thiết lập với các biến môi trường trọng yếu để kết nối 2 thành phần:

DATABASE_HOST: Kết nối về metastore-db.

S3_ENDPOINT_URL: Trỏ về http://minio:9000.

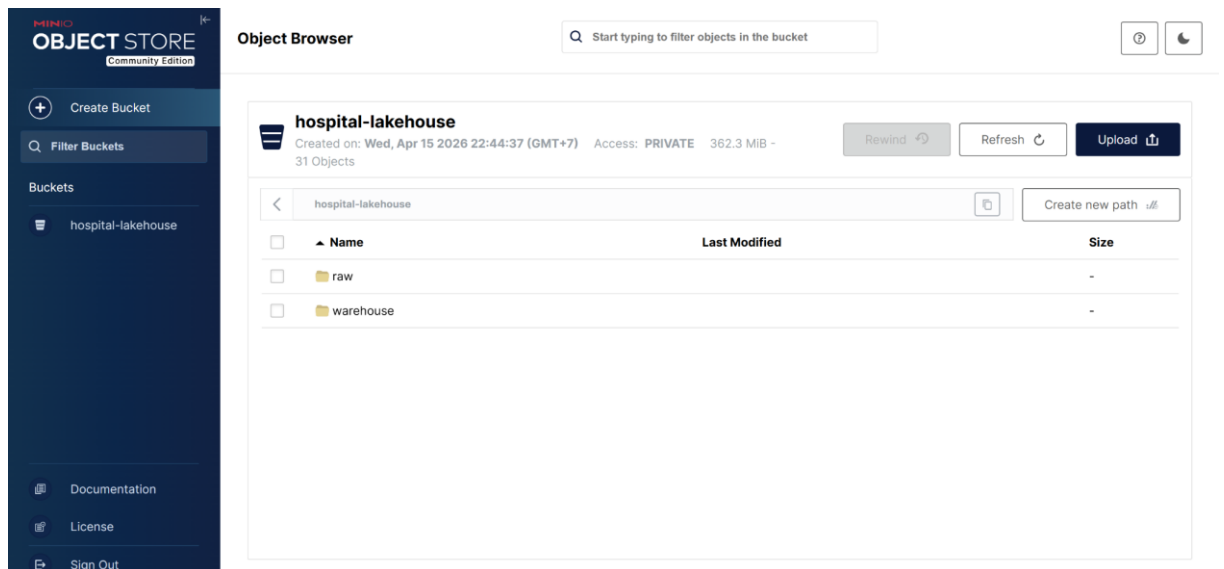
Mở cổng giao thức Thrift (9083) để làm điểm kết nối chung cho Spark và Trino.

Các bước Xác minh và Kiểm tra (Verification)

Sau khi triển khai, quy trình xác minh tính sẵn sàng được thực hiện từng bước:

- Kiểm tra Logs: Sử dụng lệnh `docker logs postgres-source -f` để theo dõi tiến độ nạp dữ liệu ngẫu nhiên của PostgreSQL.
- Xác minh quyền truy cập: Thử nghiệm đăng nhập vào MinIO Console và kiểm tra xem bucket hospital-lakehouse đã được tạo cùng các Key tương ứng chưa.
- Kiểm tra kết nối HMS: Đảm bảo container Hive Metastore đã khởi động thành công và tìm thấy các thông điệp "Metastore Server is up" trong log của container.

Kết quả đạt được



2.1.4 Cài đặt và tích hợp Trino Query Engine

2.1.4.1. Cấu hình docker-compose

Trong file docker-compose.yml, service Trino được định nghĩa như sau:

```
trino:
  image: trinodb/trino:480
  container_name: trino
  depends_on:
    minio:
      condition: service_healthy
    hive-metastore:
      condition: service_started
  networks:
    - lakehouse_net
  ports:
    - "8081:8080"
  volumes:
    - ./trino/catalog:/etc/trino/catalog
```

Service này sử dụng image trinodb/trino:480, một phiên bản ổn định của Trino. Trino phụ thuộc vào hai service khác: MinIO (để truy cập Object Storage) và Hive Metastore (để lấy metadata). Port 8081 trên host được mapping đến port 8080 bên trong container, cho phép truy cập giao diện web Trino từ máy chủ. Thư mục ./trino/catalog được mount vào /etc/trino/catalog trong container, nơi Trino tìm kiếm các file cấu hình catalog. Trino được đặt trong mạng lakehouse_net chung, đảm bảo nó có thể giao tiếp

với các service khác như Hive Metastore (host: hive-metastore:9083) và MinIO (host: minio:9000).

2.1.4.2. Cấu hình Catalog Iceberg

File iceberg.properties chứa toàn bộ thông số cần thiết để Trino kết nối với Iceberg tables trên MinIO thông qua Hive Metastore:

```
connector.name=iceberg
iceberg.catalog.type=HIVE_METASTORE
hive.metastore.uri=thrift://hive-metastore:9083
fs.native-s3.enabled=true
s3.endpoint=http://minio:9000
s3.aws-access-key=lakehouse_admin
s3.aws-secret-key=Lakehouse123!
s3.path-style-access=true
s3.region=us-east-1
```

Dòng connector.name=iceberg khai báo rằng catalog này sử dụng Iceberg connector. Dòng iceberg.catalog.type=HIVE_METASTORE yêu cầu Trino sử dụng Hive Metastore làm backend để quản lý metadata Iceberg. URI thrift://hive-metastore:9083 chỉ định địa chỉ và cổng của Hive Metastore service (cổng 9083 là cổng giao thức Thrift tiêu chuẩn). Vì dữ liệu được lưu trên MinIO (S3-compatible object storage), các tham số s3.endpoint, s3.aws-access-key, s3.aws-secret-key, và s3.path-style-access=true được cấu hình để Trino có thể truy cập MinIO. Tham số fs.native-s3.enabled=true kích hoạt S3 filesystem native support, tối ưu hóa hiệu suất khi đọc dữ liệu từ MinIO.

Ngoài ra, nếu ta muốn mở rộng để Trino có thể đọc được dữ liệu từ nhiều nguồn hơn thì có thể tạo thêm catalog khác, ví dụ như mysql.properties,...

2.1.4.3. Thực thi trên Coordinator

Sau khi đã cài xong dịch vụ Trino trong file docker-compose. Ta có thể truy cập vào Trino CLI bằng lệnh sau để vào Coordinator

```
docker exec -it trino trino
```

Xem các catalog được khởi tạo

```
trino> SHOW CATALOGS;
```

```

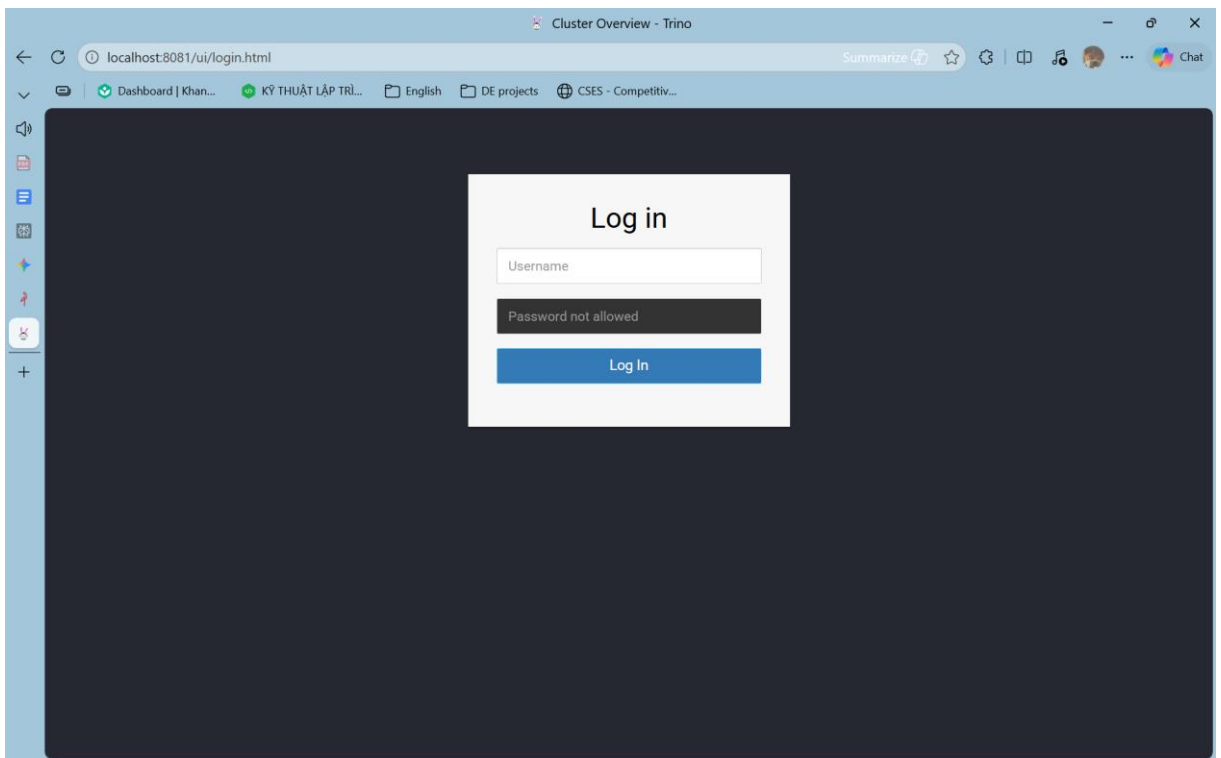
PS D:\UTE\S6-P1\Big Data Analysis\healthcare-lakehouse-covid19> docker exec -it trino trino
trino> SHOW CATALOGS;
Catalog
-----
iceberg
system
(2 rows)

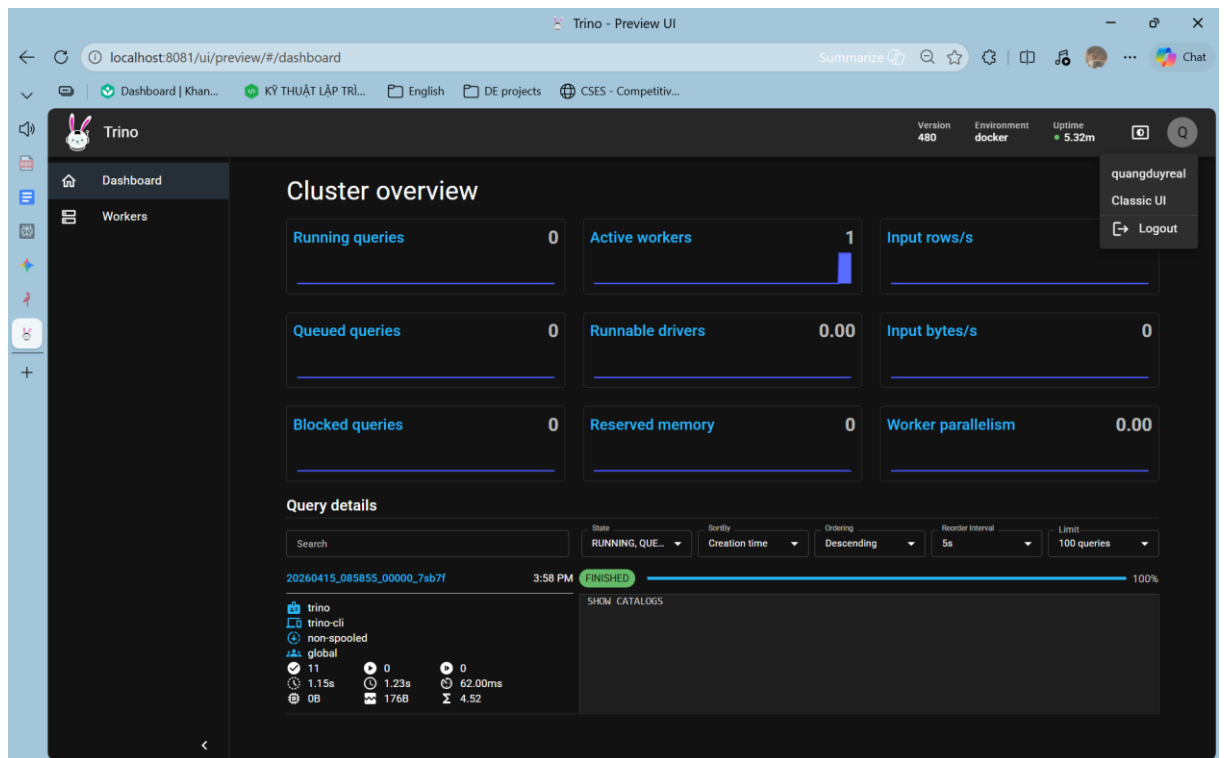
Query 20260415_180721_00031_xndfb, FINISHED, 1 node
Splits: 11 total, 11 done (100.00%)
0.22 [0 rows, 128B] [0 rows/s, 577B/s]

trino> █

```

Trino còn cung cấp cho người dùng giao diện để giám sát việc thực thi các câu truy vấn. Truy cập vào localhost:8081 theo config của file docker-compose. Ở trang login có thể nhập bất cứ tên username nào cũng được.





Sau khi đăng nhập vào thì Trino sẽ hiển thị một dashboard với từng mục riêng biệt để quản lý truy vấn và các Worker. Ở đây thì người không thể viết câu truy vấn thay cho Trino CLI mà chỉ có thể xem được các câu truy vấn lịch sử mà Trino đã được thực thi bởi ai, vào thời gian nào... Ta có thể quan sát được hệ thống đang dành bao nhiêu tài nguyên để tải các câu truy vấn. Rất hữu ích cho việc giám sát thực thi trong quy mô lớn

2.1.5 Thiết lập môi trường lập lịch với Apache Airflow

Trong kiến trúc Data Lakehouse, khi số lượng các tiến trình xử lý dữ liệu trở nên phức tạp và phụ thuộc lẫn nhau, việc quản lý thủ công các kịch bản chạy kịch bản scripts là không khả thi. Do đó, Apache Airflow được lựa chọn làm hệ thống điều phối Orchestration trung tâm, đóng vai trò như "bộ não" điều khiển toàn bộ dòng chảy dữ liệu. Airflow không trực tiếp thực hiện việc tính toán dữ liệu nặng nề mà thay vào đó, nó lập lịch, kích hoạt và giám sát các tác vụ xử lý trên Spark, đảm bảo mọi công đoạn từ thu thập dữ liệu thô đến phân tích chuyên sâu được diễn ra theo đúng trình tự và thời điểm định trước.

Trong môi trường container hóa của dự án, Apache Airflow được triển khai với cấu hình tối ưu cho việc thử nghiệm và phát triển. Hệ thống sử dụng SequentialExecutor, một cơ chế thực thi tuần tự phù hợp cho các môi trường Lab hoặc đơn máy chủ. Lựa

chọn này giúp giảm thiểu độ phức tạp về hạ tầng khi không yêu cầu các hàng đợi tin nhắn bổ sung như Redis hay RabbitMQ, nhưng vẫn đảm bảo tính chính xác tuyệt đối trong việc thực hiện các tác vụ theo chuỗi. Để Airflow có thể tương tác mật thiết với các thành phần khác trong hệ sinh thái, một loạt các biến môi trường đã được thiết lập sẵn trong Docker Compose. Các biến này tự động cấu hình các kết nối tới PostgreSQL nguồn, dịch vụ lưu trữ MinIO và cụm tính toán Spark thông qua các tiền tố như `AIRFLOW_CONN_...`. Việc cấu hình sẵn này giúp hệ thống đạt trạng thái sẵn sàng ngay khi khởi động, cho phép các tiến trình `SparkSubmitOperator` có thể gọi đến các dịch vụ bên ngoài mà không cần cấu hình thủ công trên giao diện quản trị.

Cơ chế vận hành của Airflow dựa trên khái niệm DAG. Mỗi DAG đại diện cho một quy trình công việc cụ thể, trong đó các nút là các Task (Tác vụ) và các cạnh biểu thị sự phụ thuộc giữa chúng. Scheduler của Airflow liên tục giám sát trạng thái của các DAG và các điều kiện kích hoạt để đưa các Task vào hàng đợi thực thi khi các điều kiện tiên quyết đã được thỏa mãn. Trong dự án này, sự kết nối giữa Airflow và mã nguồn được duy trì thông qua cơ chế Volume Mount, ánh xạ thư mục `./dags` và `./scripts` từ máy chủ vật lý vào container Airflow. Điều này đảm bảo rằng mọi thay đổi trong logic xử lý dữ liệu hoặc cấu hình pipeline đều được cập nhật tức thì, tạo điều kiện thuận lợi cho việc phát triển và tinh chỉnh các kịch bản ETL.

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions
01_csv_to_postgres	lakehouse_admin	3	None	2026-04-13, 02:17:34		3	
02_postgres_to_raw	lakehouse_admin	2	None	2026-04-13, 02:34:24		1	
03_raw_to_bronze	lakehouse_admin	1	None	2026-04-13, 02:44:05		1	
04_bronze_to_silver	lakehouse_admin	1	None	2026-04-11, 21:30:08		1	
05_silver_to_gold	lakehouse_admin	1	None	2026-04-11, 21:35:06		1	

Hình 2.1.5.1 Các DAG đang có trong hệ thống

Vai trò điều phối của Airflow được thể hiện rõ nét qua chuỗi 5 giai đoạn cốt lõi của pipeline Data Lakehouse. Đầu tiên, DAG `01_csv_to_postgres` quản lý việc nạp dữ liệu thô từ định dạng CSV vào cơ sở dữ liệu nguồn PostgreSQL, tuân thủ nghiêm ngặt

các ràng buộc khóa ngoại thông qua việc thiết lập thứ tự chạy tuần tự giữa các bảng Patients, Encounters và Conditions. Tiếp theo, giai đoạn 02_postgres_to_raw thực hiện nhiệm vụ trích xuất dữ liệu từ Postgres để đẩy lên MinIO dưới định dạng Parquet nhằm tối ưu dung lượng lưu trữ. Tại giai đoạn 03_raw_to_bronze, Airflow kích hoạt Spark để nạp dữ liệu Parquet vào các bảng Iceberg tại tầng Bronze, bắt đầu quy trình quản lý Metadata theo chuẩn Lakehouse. Quy trình tiếp tục với DAG 04_bronze_to_silver, nơi thực hiện các tác vụ làm sạch và chuẩn hóa dữ liệu. Cuối cùng, DAG 05_silver_to_gold điều phối các truy vấn phân tích lâm sàng chuyên sâu để tạo ra các tập dữ liệu giá trị cao phục vụ báo cáo. Nhờ khả năng quản lý lỗi, tự động thực hiện lại (retry) khi gặp sự cố và giao diện giám sát trực quan, Airflow giúp toàn bộ quy trình từ dữ liệu thô đến dữ liệu vàng (Gold layer) trở nên bền bỉ, có thể dự đoán và dễ dàng kiểm soát. Thông tin cụ thể về các DAG sẽ được trình bày trong mục 2.2.4.

2.1.6. Cài đặt Apache Superset để trực quan hóa dữ liệu

Superset được xây dựng từ image gốc của Apache Superset và được tùy biến thêm để hỗ trợ kết nối với Trino. Trong Dockerfile, dự án cài đặt thêm gói driver sqlalchemy-trino vào môi trường Python nội bộ của Superset, nhằm bảo đảm hệ thống có thể nhận diện và sử dụng đúng dialect của Trino khi thiết lập kết nối dữ liệu. Bên cạnh đó, file khởi động superset_init.sh đảm nhiệm quá trình tự động hóa ban đầu, bao gồm nâng cấp cơ sở dữ liệu metadata của Superset, tạo tài khoản quản trị mặc định, khởi tạo hệ thống nội bộ và cuối cùng là khởi chạy web server trên cổng 8088. Cách tổ chức này giúp Superset có thể được triển khai nhất quán bằng Docker mà không cần thao tác thủ công từng bước sau mỗi lần khởi động lại môi trường.

superset/docker/Dockerfile

```
FROM apache/superset:latest
```

```
USER root
```

```
RUN pip3 install --target /app/.venv/lib/python3.10/site-packages -  
-no-cache-dir sqlalchemy-trino
```

```
USER superset
```

superset/docker/superset_init.sh

```
#!/bin/sh
set -e

superset db upgrade
superset fab create-admin \
  --username "${SUPERSET_ADMIN_USERNAME:-admin}" \
  --firstname "${SUPERSET_ADMIN_FIRSTNAME:-Admin}" \
  --lastname "${SUPERSET_ADMIN_LASTNAME:-User}" \
  --email "${SUPERSET_ADMIN_EMAIL:-admin@example.com}" \
  --password "${SUPERSET_ADMIN_PASSWORD:-admin}" || true
superset init
exec superset run -p 8088 -h 0.0.0.0 --with-threads --reload --
debugger
```

Khai báo container cho superset trong file docker-compose

```
superset:
  build:
    context: ./superset/docker
    dockerfile: Dockerfile
  container_name: superset
  depends_on:
    trino:
      condition: service_started
  environment:
    - SUPERSET_CONFIG_PATH=/app/pythonpath/superset_config.py
    - SUPERSET_ADMIN_USERNAME=${SUPERSET_ADMIN_USERNAME:-admin}
    - SUPERSET_ADMIN_PASSWORD=${SUPERSET_ADMIN_PASSWORD:-admin}
    - SUPERSET_ADMIN_FIRSTNAME=${SUPERSET_ADMIN_FIRSTNAME:-Admin}
    - SUPERSET_ADMIN_LASTNAME=${SUPERSET_ADMIN_LASTNAME:-User}
    - SUPERSET_ADMIN_EMAIL=${SUPERSET_ADMIN_EMAIL:-
admin@example.com}
  networks:
    - lakehouse_net
```

ports:

- "8088:8088"

volumes:

-

./superset/config/superset_config.py:/app/pythonpath/superset_config.py

- **./superset/docker/superset_init.sh:/app/superset_init.sh**

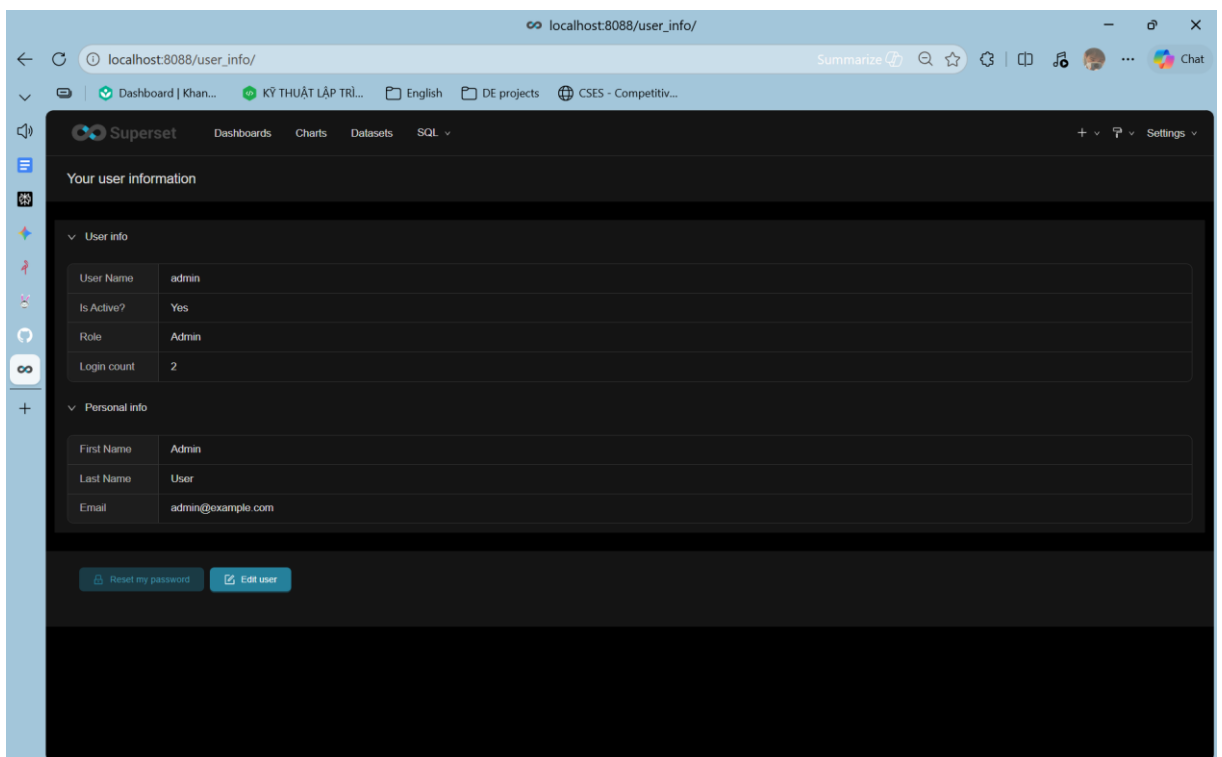
- **superset_home:/app/superset_home**

command: ["/bin/sh", "/app/superset_init.sh"]

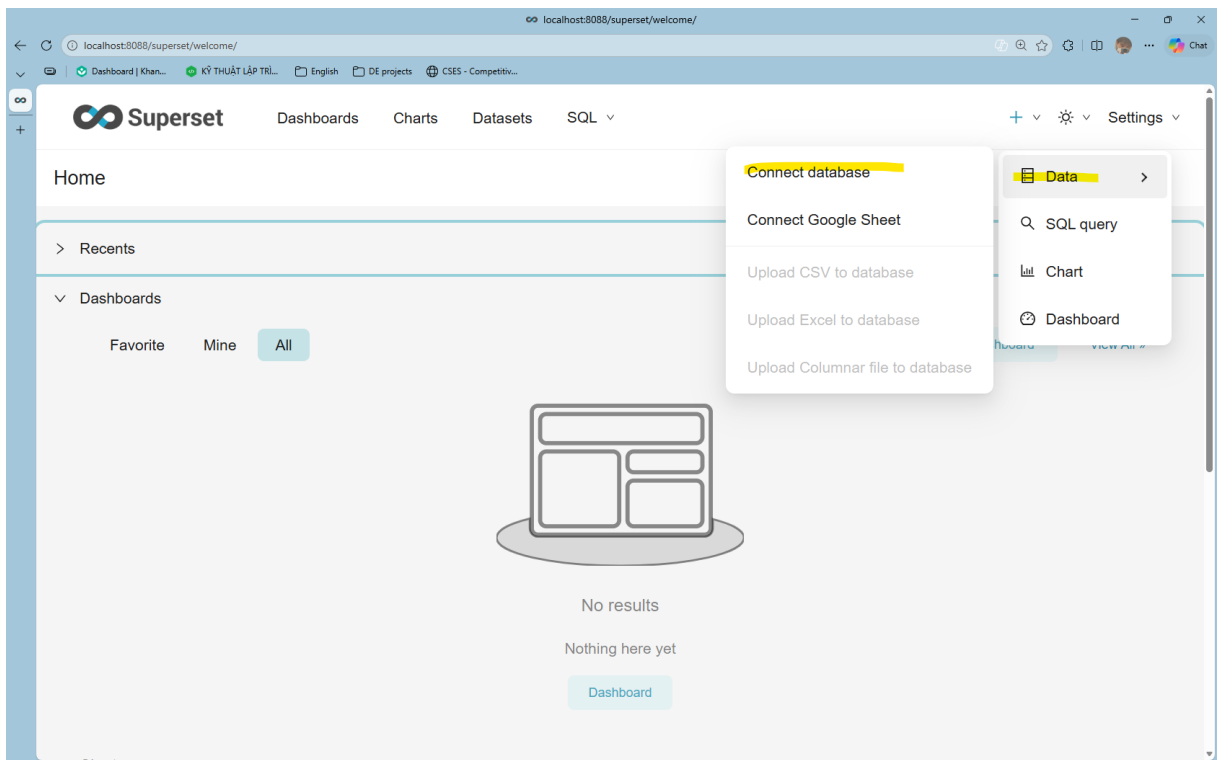
Build image Superset

```
docker compose --env-file .env -f deploy/docker-compose.yml build  
superset --no-cache
```

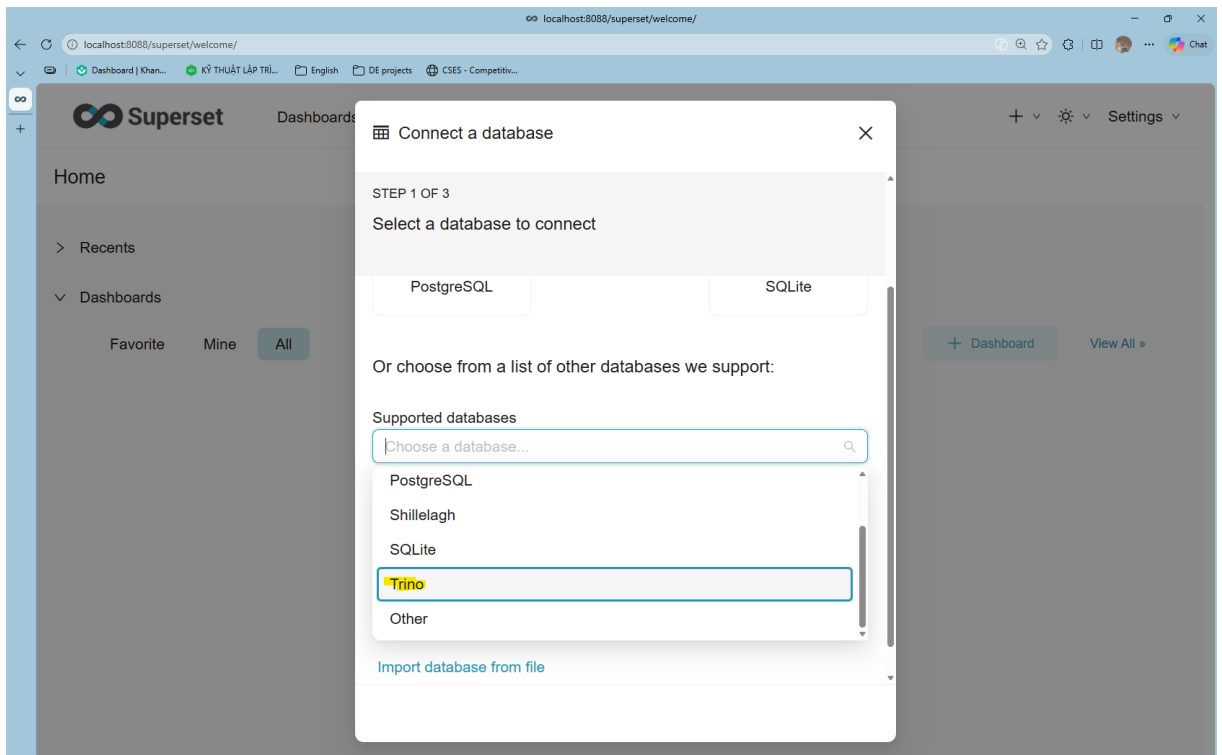
Sau khi đã cài đặt xong, ta có thể truy cập vào Superset Web UI ở localhost:8088, tài khoản và mật khẩu mặc định là “admin”.



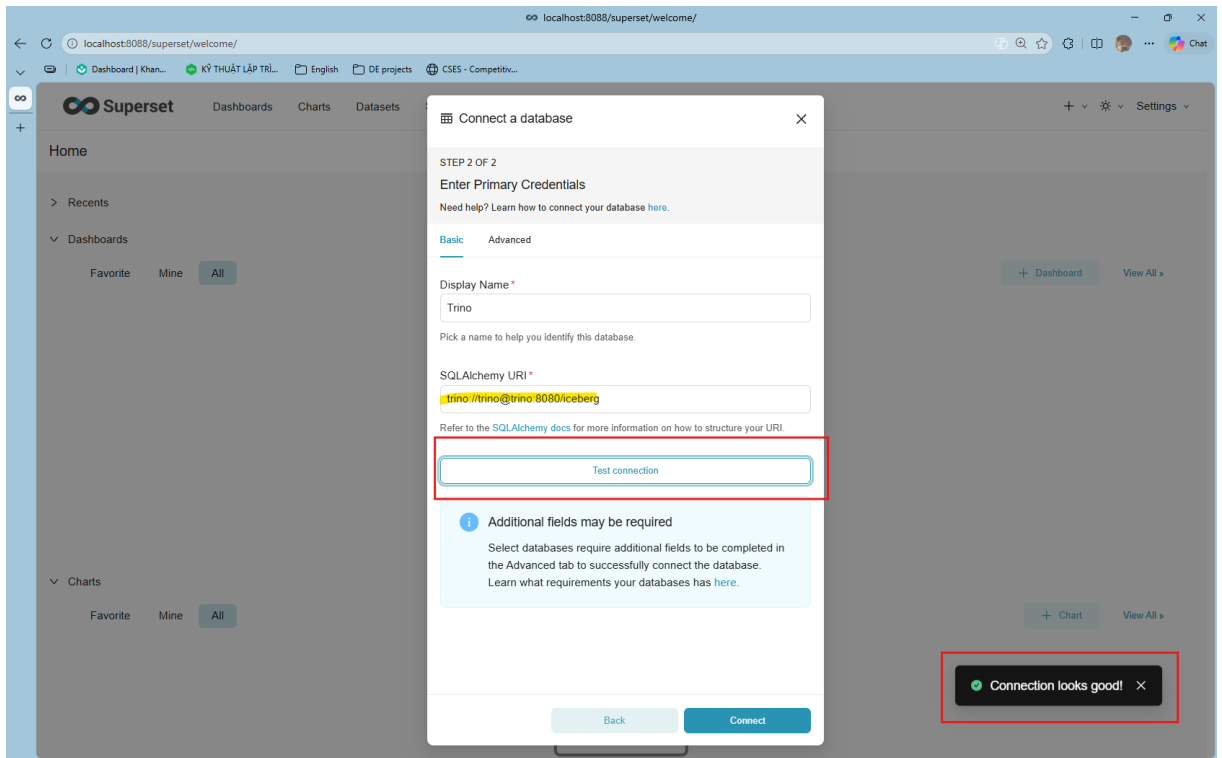
Bây giờ ta thực hiện kết nối Trino với Superset thông qua SQLAlchemy. Ở giao diện chính ta nhập vào Data > Connect database



Chọn Trino



Nhập vào SQLAlchemy URI: `trino://trino@trino:8080/iceberg` và test connection



Chọn Connect > Done.

2.2. Xây dựng Pipeline xử lý dữ liệu

2.2.1 Ingestion: Trích xuất dữ liệu từ PostgreSQL sang lớp Bronze

Quy trình Ingestion (Nạp dữ liệu) đóng vai trò là "cửa ngõ" đầu tiên trong kiến trúc Medallion. Mục tiêu của giai đoạn này là di chuyển dữ liệu từ cơ sở dữ liệu quan hệ PostgreSQL sang lớp Bronze trên Data Lakehouse, chuyển đổi dữ liệu từ định dạng hàng/cột truyền thống sang định dạng bảng Apache Iceberg hiện đại.

Mục tiêu và Chiến lược nạp dữ liệu

- Mục tiêu: Lưu trữ toàn bộ dữ liệu lịch sử từ nguồn một cách vẹn toàn để phục vụ cho việc truy vết và tái xử lý khi cần thiết.
- Chiến lược: Sử dụng phương pháp Full Load (nạp toàn bộ) kết hợp với cơ chế Overwrite của Iceberg. Việc này giúp lớp Bronze luôn phản ánh đúng trạng thái mới nhất của lớp Source nhưng vẫn giữ được lịch sử thông qua các Snapshot của Iceberg.

Quy trình kỹ thuật (Hai giai đoạn)

Trong dự án này, quy trình Ingestion được chia nhỏ thành hai bước kỹ thuật thông qua các script PySpark chuyên dụng để tối ưu hóa hiệu năng:

Giai đoạn 1: Trích xuất từ Postgres sang Raw Zone (postgres_to_raw.py)

Công nghệ: Sử dụng PySpark kết nối qua JDBC Driver.

Tối ưu hóa: Cấu hình tham số fetchsize=10000 để tăng tốc độ đọc cho các bảng lớn (như bảng encounters với hơn 3 triệu bản ghi).

Kết quả: Dữ liệu được ghi vào Bucket hospital-lakehouse/raw/ dưới định dạng Parquet. Đây là định dạng lưu trữ dạng cột (columnar) giúp tiết kiệm dung lượng và tăng tốc độ đọc cho Spark ở các bước sau.

Giai đoạn 2: Chuyển đổi sang bảng Bronze Iceberg (bronze_ingestion.py)

Công nghệ: Apache Spark kết hợp với Iceberg Library và Hive Metastore Catalog.

Quy trình: Spark đọc dữ liệu từ vùng Raw, thực hiện khởi tạo Namespace hospital.bronze và ghi dữ liệu vào các bảng Iceberg.

Tại sao dùng Iceberg ở lớp Bronze?

- ACID Transactions: Đảm bảo quá trình nạp dữ liệu không bị lỗi nửa chừng gây hỏng dữ liệu.
- Snapshot Management: Cho phép quay ngược thời gian (Time Travel) để kiểm tra dữ liệu tại các thời điểm nạp khác nhau.
- Schema Enforcement: Đảm bảo cấu trúc dữ liệu luôn tuân thủ đúng định dạng đã thiết kế tại lớp Source.

Cấu hình Spark trọng yếu

- Để quy trình Ingestion hoạt động trơn tru với MinIO và Hive Metastore, Spark được cấu hình với các tham số quan trọng:

spark.sql.catalog.hospital: Sử dụng HiveCatalog để quản lý các bảng Iceberg.

spark.hadoop.fs.s3a.endpoint: Trỏ về http://minio:9000.

spark.sql.extensions: Kích hoạt các tính năng mở rộng của Iceberg để hỗ trợ các câu lệnh SQL nâng cao.

Kết quả vận hành

- Dữ liệu từ 3 bảng nguồn (patients, encounters, conditions) đã được nạp thành công vào lớp Bronze.

MINIO

OBJECT STORE

Community Edition

Create Bucket

Filter Buckets

Buckets

hospital-lakehouse

Documentation

License

Sign Out

Object Browser

Q Start typing to filter objects in the bucket

?

🌙

hospital-lakehouse

Created on: Wed, Apr 15 2026 22:44:37 (GMT+7) Access: PRIVATE 362.3 MiB - 31 Objects

Rewind ↺ Refresh ↻ Upload ↗

hospital-lakehouse / warehouse

Create new path ↗

☐	Name	Last Modified	Size
☐	bronze.db		-
☐	gold.db		-
☐	silver.db		-

hospital-lakehouse

Created on: Wed, Apr 15 2026 22:44:37 (GMT+7) Access: PRIVATE 362.3 MiB - 31 Objects

Rewind ↺ Refresh ↻ Upload ↗

hospital-lakehouse / warehouse / bronze.db

Create new path ↗

☐	Name	Last Modified	Size
☐	conditions		-
☐	encounters		-
☐	patients		-

- Cấu trúc dữ liệu trên MinIO lúc này được quản lý tự động bởi Iceberg, bao gồm thư mục data (chứa các file Parquet) và thư mục metadata (chứa các file JSON mô tả lịch sử bảng).

hospital-lakehouse

Created on: Wed, Apr 15 2026 22:44:37 (GMT+7) Access: PRIVATE 362.3 MiB - 31 Objects

Rewind ↺ Refresh ↻ Upload ↗

hospital-lakehouse / warehouse / bronze.db / patients

Create new pat

☐	Name	Last Modified	Size
☐	data		-
☐	metadata		-

hospital-lakehouse

Created on: Wed, Apr 15 2026 22:44:37 (GMT+7) Access: PRIVATE 362.3 MiB - 31 Objects

Rewind ↺ Refresh ↻ Upload ↗

hospital-lakehouse / warehouse / bronze.db / patients / data

Create new path ↗

☐	Name	Last Modified	Size
☐	00000-10-26e3783b-4843-495a-9f4a-04b1a3b18ea0-00001...	Wed, Apr 15 2026 23:05 (GMT+7)	8.1 MiB

- Dữ liệu tại lớp Bronze lúc này đã sẵn sàng để các công cụ như Trino truy vấn hoặc tiếp tục quy trình làm sạch dữ liệu chuyển sang lớp Silver.

2.2.2 Silver Layer: Làm sạch dữ liệu và chuẩn hóa kiểu dữ liệu

Trước khi đi vào phân xử lý dữ liệu, ta cần nhìn dữ liệu ở trạng thái ban đầu như một bộ hồ sơ y tế bán cấu trúc hơn là một bảng phân tích hoàn chỉnh. Khi truy vấn trực tiếp lớp Bronze qua Trino, có thể thấy mức độ phân tách dữ liệu rất rõ: bảng patients chỉ có 124,150 dòng, trong khi bảng encounters có 3,188,675 dòng và bảng conditions có 1,143,900 dòng. Điều đó cho thấy dữ liệu không nằm trong một bảng đơn nhất mà bị chia thành nhiều thực thể, trong đó mỗi thực thể chỉ mô tả một lát cắt của toàn bộ hành trình điều trị. EDA trong data/Synthea COVID-19 Analysis.html cũng xác nhận đúng cách tiếp cận này rằng notebook không phân tích theo một bảng tổng hợp duy nhất mà phải nạp đồng thời nhiều file CSV như patients, conditions, careplans, encounters, devices, supplies, procedures, medications, sau đó lọc theo các quy tắc nghiệp vụ cụ thể như mã COVID ‘840539006’, kết quả xét nghiệm âm tính ‘94531-1’, ngày tử vong DEATHDATE.notna(), hay trạng thái hoàn thành care plan. Như vậy, dữ liệu đầu vào không đồng nhất về ngữ nghĩa và chưa ở trạng thái sẵn sàng để khai thác trực tiếp ở mức phân tích tổng hợp.

Các câu truy vấn sau được dùng để kiểm chứng trực tiếp trạng thái dữ liệu Bronze trong Trino:

1. Kiểm tra các bảng hiện có trong lớp Bronze:

```
SHOW TABLES IN iceberg.bronze;
```

```
PS D:\UTE\S6-P1\Big Data Analysis\healthcare-lakehouse-covid19> docker exec -it trino trino
trino> SHOW TABLES IN iceberg.bronze;
Table
-----
conditions
encounters
patients
(3 rows)

Query 20260415_175710_00022_xndfb, FINISHED, 1 node
Splits: 11 total, 11 done (100.00%)
0.23 [3 rows, 242B] [13 rows/s, 1.03KiB/s]

trino> █
```

2. Đếm số dòng của các bảng chính

```
SELECT 'patients' AS table_name, count(*) AS rows FROM
iceberg.bronze.patients
UNION ALL
```

```

SELECT 'encounters' AS table_name, count(*) AS rows FROM
iceberg.bronze.encounters
UNION ALL
SELECT 'conditions' AS table_name, count(*) AS rows FROM
iceberg.bronze.conditions;

```

```

PS D:\UTE\S6-P1\Big Data Analysis\healthcare-lakehouse-covid19> docker exec -it trino trino
trino> SELECT 'patients' AS table_name, count(*) AS rows FROM iceberg.bronze.patients
-> UNION ALL
-> SELECT 'encounters' AS table_name, count(*) AS rows FROM iceberg.bronze.encounters
-> UNION ALL
-> SELECT 'conditions' AS table_name, count(*) AS rows FROM iceberg.bronze.conditions;
->
table_name | rows
-----+-----
encounters | 3188675
patients   | 124150
conditions | 1143900
(3 rows)

Query 20260415_175733_00024_xndfb, FINISHED, 1 node
Splits: 39 total, 39 done (100.00%)
0.35 [4.46M rows, 172B] [12.7M rows/s, 491B/s]

trino> █

```

3. Kiểm tra mức độ thiếu dữ liệu trong patients

```

SELECT
    count(*) AS rows,
    count_if(deathdate IS NULL) AS null_deathdate,
    count_if(gender IS NULL OR trim(gender) = '') AS empty_gender
FROM iceberg.bronze.patients;

```

```

PS D:\UTE\S6-P1\Big Data Analysis\healthcare-lakehouse-covid19> docker exec -it trino trino
trino> SELECT
-> count(*) AS rows,
-> count_if(deathdate IS NULL) AS null_deathdate,
-> count_if(gender IS NULL OR trim(gender) = '') AS empty_gender
-> FROM iceberg.bronze.patients;
->
rows | null_deathdate | empty_gender
-----+-----+-----
124150 | 100000 | 0
(1 row)

Query 20260415_175747_00025_xndfb, FINISHED, 1 node
Splits: 10 total, 10 done (100.00%)
0.34 [124K rows, 138KiB] [365K rows/s, 407KiB/s]

trino> █

```

4. Phân bố giới tính trong patients

```

SELECT gender, count(*) AS cnt
FROM iceberg.bronze.patients
GROUP BY gender

```

```
ORDER BY cnt DESC;
```

```
PS D:\UTE\S6-P1\Big Data Analysis\healthcare-lakehouse-covid19> docker exec -it trino trino
trino> SELECT gender, count(*) AS cnt
  -> FROM iceberg.bronze.patients
  -> GROUP BY gender
  -> ORDER BY cnt DESC;
  ->
gender | cnt
-----+-----
M      | 62810
F      | 61340
(2 rows)

Query 20260415_175805_00026_xndfb, FINISHED, 1 node
Splits: 27 total, 27 done (100.00%)
0.22 [124K rows, 64.3KiB] [559K rows/s, 290KiB/s]

trino> █
```

5. Phân bố các loại encounter trong encounters

```
SELECT encounterclass, count(*) AS cnt
FROM iceberg.bronze.encounters
GROUP BY encounterclass
ORDER BY cnt DESC;
```

```
PS D:\UTE\S6-P1\Big Data Analysis\healthcare-lakehouse-covid19> docker exec -it trino trino
trino> SELECT encounterclass, count(*) AS cnt
  -> FROM iceberg.bronze.encounters
  -> GROUP BY encounterclass
  -> ORDER BY cnt DESC;
  ->
encounterclass | cnt
-----+-----
wellness       | 1569498
ambulatory     | 835566
outpatient     | 574583
inpatient      | 95273
emergency      | 73484
urgentcare     | 40271
(6 rows)

Query 20260415_175816_00027_xndfb, FINISHED, 1 node
Splits: 28 total, 28 done (100.00%)
0.32 [3.19M rows, 845KiB] [9.96M rows/s, 2.58MiB/s]

trino> █
```

Khi đi sâu hơn vào chất lượng dữ liệu, ta thấy vấn đề không phải là thiếu toàn bộ bản ghi, mà là cấu trúc dữ liệu còn thiên về mô tả sự kiện và còn thiếu tính phân tích. Ở bảng patients, cột deathdate có tới 100,000 giá trị NULL trên tổng 124,150 dòng, tức phần lớn bệnh nhân chưa ghi nhận tử vong trong mô phỏng; ngược lại, cột giới tính lại được điền đầy đủ với hai giá trị chủ đạo là M và F (lần lượt 62,810 và 61,340 bản ghi). Điều này cho thấy một số thuộc tính nhân khẩu học có độ phủ tốt, nhưng các thuộc tính kết cục lâm sàng lại thưa hơn và cần được xử lý cẩn trọng khi diễn giải. Ở bảng encounters, dữ liệu không có lỗi thiếu khóa chính, nhưng lại có khối lượng giao dịch rất

lớn và phân bố theo nhiều loại encounter khác nhau, trong đó wellness, ambulatory, outpatient, inpatient, emergency, và urgentcare chiếm các tỷ trọng khác nhau. Nói cách khác, dữ liệu không bị hỏng theo nghĩa kỹ thuật, nhưng đang ở dạng event-level dày đặc, nhiều lớp, và nếu đưa thẳng vào phân tích sẽ làm tăng chi phí join, group by, cũng như làm mờ trọng tâm bệnh nhân.

Chính vì vậy, nhóm chọn ba bảng patients, encounters, conditions để transform sang Silver. Lựa chọn này có cơ sở nghiệp vụ rõ ràng. patients là thực thể gốc, cung cấp danh tính và đặc điểm nhân khẩu học; encounters phản ánh các lần tương tác y tế và chi phí liên quan; conditions là nguồn xác định bệnh lý và cohort COVID. Ba bảng này tạo thành bộ dữ liệu đủ bao quát để xây một bảng Silver theo hướng patient-centric, tức là mỗi bệnh nhân là một đơn vị phân tích chính thay vì phải truy vấn rời rạc từng sự kiện. Đây là tiền đề để lớp Gold có thể tổng hợp tiếp thành các bảng thống kê phục vụ việc ra quyết định.

Về mặt cài đặt, toàn bộ logic được triển khai trong file `deploy/scripts/silver_cleansing.py`. Script này khởi tạo Spark theo catalog Hive, đọc dữ liệu từ Bronze, làm sạch từng bảng riêng lẻ, nhận diện cohort COVID, rồi tổng hợp về mức bệnh nhân trước khi ghi xuống Iceberg. Đoạn cấu hình Spark cho thấy rõ pipeline không làm việc trên file cục bộ mà gắn chặt với Hive Metastore và MinIO, nhờ đó bảng Silver sau khi ghi có thể được Trino truy cập ngay:

```
SparkSession.builder.appName("Iceberg Silver Cleansing")
    .config("spark.sql.catalog.hospital",
"org.apache.iceberg.spark.SparkCatalog")
    .config("spark.sql.catalog.hospital.type", "hive")
    .config("spark.sql.catalog.hospital.uri",          "thrift://hive-
metastore:9083")
    .config("spark.sql.catalog.hospital.warehouse",   "s3a://hospital-
lakehouse/warehouse")
```

Trong bước làm sạch, patients là bảng được ưu tiên xử lý đầu tiên vì đây là nguồn định danh gốc của toàn bộ mô hình phân tích. Code `clean_patients()` thực hiện chuẩn hóa patient_id bằng cách loại bỏ khoảng trắng và chuyển các giá trị rỗng, NULL, null thành None. Sau đó, các cột ngày tháng được ép sang date, còn các trường thông tin nhân khẩu học, địa chỉ và chi phí được ép về kiểu tương ứng. Đặc biệt, trường giới tính

được chuẩn hóa từ nhiều biểu diễn khác nhau về hai giá trị nghiệp vụ phổ biến là Male và Female, nhờ hàm `normalize_gender()`. Cách làm này giúp giảm sai khác do encoding đầu vào và tạo điều kiện cho phân tích nhóm dân số ở các tầng sau.

```
def clean_patients(patients: DataFrame) -> DataFrame:
    return (
        patients.select(
            normalize_identifiser(F.col("id")).alias("patient_id"),
            F.to_date(F.col("birthdate")).alias("birthdate"),
            F.to_date(F.col("deathdate")).alias("deathdate"),
            F.col("ssn").cast("string").alias("ssn"),
            F.col("drivers").cast("string").alias("drivers"),
            F.col("passport").cast("string").alias("passport"),
            F.col("prefix").cast("string").alias("prefix"),
            F.col("first").cast("string").alias("first_name"),
            F.col("last").cast("string").alias("last_name"),
            F.col("suffix").cast("string").alias("suffix"),
            F.col("maiden").cast("string").alias("maiden_name"),

F.col("marital").cast("string").alias("marital_status"),
            F.col("race").cast("string").alias("race"),
            F.col("ethnicity").cast("string").alias("ethnicity"),
            normalize_gender(F.col("gender")).alias("gender"),
            F.col("birthplace").cast("string").alias("birthplace"),
            F.col("address").cast("string").alias("address"),
            F.col("city").cast("string").alias("city"),
            F.col("state").cast("string").alias("state"),
            F.col("county").cast("string").alias("county"),
            F.col("zip").cast("string").alias("zip_code"),
            F.col("lat").cast("decimal(10, 6)").alias("lat"),
            F.col("lon").cast("decimal(10, 6)").alias("lon"),
            F.col("healthcare_expenses").cast("decimal(12,
2)").alias("healthcare_expenses"),
            F.col("healthcare_coverage").cast("decimal(12,
```

```
2)").alias("healthcare_coverage"),
    )
    .filter(F.col("patient_id").isNotNull())
    .dropDuplicates(["patient_id"])
    )
```

Ở bảng encounters, nhóm thực hiện chuẩn hóa các trường thời gian sang timestamp, đồng thời ép kiểu các cột chi phí sang decimal(12, 2). Đây là bước quan trọng vì dữ liệu encounter không chỉ dùng để đếm số lượt khám mà còn dùng để đánh giá gánh nặng chi phí điều trị. Hai khóa liên kết quan trọng nhất là encounter_id và patient_id cũng được làm sạch ngay từ đầu để tránh lỗi join ở bước tổng hợp. Tương tự, conditions được làm sạch bằng cách chuẩn hóa patient_id, encounter_id, chuyển ngày bắt đầu và ngày kết thúc sang kiểu date, sau đó loại bỏ bản ghi trùng theo tổ hợp khóa nghiệp vụ.

Một điểm cốt lõi của lớp Silver là chuyển dữ liệu từ mức sự kiện sang mức bệnh nhân. Sau khi chuẩn hóa xong các bảng nguồn, script xác định cohort COVID từ conditions với mã chẩn đoán ‘840539006’. Sau đó, các bảng conditions và encounters được lọc theo cohort này, rồi tổng hợp theo patient_id để tạo ra các chỉ số như số lần gặp, thời điểm gần nhất, và tổng chi phí điều trị. Đoạn code dưới đây minh họa cách pipeline thực hiện bước tổng hợp theo bệnh nhân:

```
condition_summary = (
    conditions_covid.repartition("patient_id")
    .groupBy("patient_id")
    .agg(
        F.count(F.lit(1)).alias("condition_count"),

        F.max(F.col("condition_start_date")).alias("last_condition_date"),
    )
)

encounter_summary = (
    encounters_covid.repartition("patient_id")
    .groupBy("patient_id")
```

```

    .agg(
        F.count(F.lit(1)).alias("encounter_count"),
        F.max(F.col("encounter_start")).alias("last_encounter_time"),
        F.sum(F.coalesce(F.col("total_claim_cost"),
F.lit(0))).cast("decimal(18,2)").alias(
            "total_claim_cost_sum"
        ),
    )
)

```

Việc bổ sung `repartition("patient_id")` trước `groupBy()` là để giảm áp lực shuffle trong Spark, đặc biệt hữu ích khi khối lượng dữ liệu lớn. Ở đây, dữ liệu được gom theo cùng một khóa logic trước khi tổng hợp, nhờ vậy job hạn chế nguy cơ phát sinh lỗi Out of Memory. Sau cùng, bảng Silver được ghi bằng `writeTo(...).createOrReplace()`, nghĩa là output không chỉ là một file dữ liệu rời rạc mà là một bảng Iceberg hoàn chỉnh, có metadata rõ ràng và sẵn sàng cho Trino khai thác ngay sau khi job kết thúc.

Nếu so sánh dữ liệu trước và sau xử lý, khác biệt thể hiện khá rõ. Trước xử lý, các bảng Bronze có nhiều trường dạng chuỗi cần chuyển kiểu, định danh chưa được chuẩn hóa hoàn toàn, ngày tháng có thể tồn tại ở nhiều định dạng khác nhau, và dữ liệu rời rạc theo từng sự kiện nên khó dùng trực tiếp cho phân tích bệnh nhân. Sau xử lý, dữ liệu Silver trở thành một bảng trung tâm `covid_clinical_master` với cấu trúc nhất quán hơn, các khóa định danh đã được làm sạch, thời gian và chi phí đã có kiểu dữ liệu đúng, cohort COVID đã được xác định, và các chỉ số tổng hợp theo bệnh nhân đã được tính sẵn. Nhờ đó, dữ liệu sau xử lý có tính giải thích tốt hơn, truy vấn nhanh hơn và phù hợp hơn cho phân tích lâm sàng cũng như trực quan hóa ở lớp BI.

```

PS D:\UTE\S6-P1\Big Data Analysis\healthcare-lakehouse-covid19> docker exec -it trino trino
trino> DESCRIBE iceberg.silver.covid_clinical_master;

```

Column	Type	Extra	Comment
patient_id	varchar		
patient_birthdate	date		
patient_deathdate	date		
ssn	varchar		
drivers	varchar		
passport	varchar		
prefix	varchar		
first_name	varchar		
last_name	varchar		
suffix	varchar		
maiden_name	varchar		
marital_status	varchar		
race	varchar		
ethnicity	varchar		
gender	varchar		
birthplace	varchar		
address	varchar		
city	varchar		
state	varchar		
county	varchar		
zip_code	varchar		
lat	decimal(10,6)		
lon	decimal(10,6)		
healthcare_expenses	decimal(12,2)		
healthcare_coverage	decimal(12,2)		
encounter_count	bigint		
last_encounter_time	timestamp(6) with time zone		
total_claim_cost_sum	decimal(18,2)		
condition_count	bigint		
last_condition_date	date		
observation_count	integer		
last_observation_time	timestamp(6) with time zone		

```

(32 rows)

Query 20260415_181304_00035_xndfb, FINISHED, 1 node
Splits: 11 total, 11 done (100.00%)
0.26 [32 rows, 5.69KiB] [121 rows/s, 21.6KiB/s]

trino> █

```

Schema của bảng silver.covid_clinical_master

Lớp Silver trong đã thực hiện việc làm sạch dữ liệu và chuẩn hóa ngữ nghĩa để chuyển dữ liệu thô thành một tập dữ liệu nghiên cứu có giá trị. Output cuối cùng của quá trình này là bảng silver.covid_clinical_master, với số dòng ổn định, metadata Iceberg đầy đủ, và khả năng truy vấn trực tiếp từ Trino. Sẵn sàng dùng để xây dựng các bảng Gold và dashboard Superset trong các bước tiếp theo.

2.2.3 Gold Layer: Tổng hợp dữ liệu và phân tích các chỉ số COVID-19

Lớp Vàng là tầng cuối cùng trong kiến trúc Medallion, đóng vai trò là tầng trình diễn và phục vụ phân tích. Tại đây, dữ liệu không còn tồn tại dưới dạng các bảng kỹ thuật rời rạc mà được chuyển đổi thành các tập dữ liệu mang tính chiến lược, phục vụ trực tiếp cho việc ra quyết định lâm sàng và lập báo cáo dịch tễ học. Mục tiêu trọng tâm của giai đoạn này là tổng hợp thông tin từ tầng Silver và Bronze để trích xuất các chỉ số KPIs quan trọng về tình hình điều trị bệnh nhân COVID-19. Trong phạm vi của đồ án,

tiến trình gold_clinical_analysis.py thực hiện hai bài toán phân tích trọng điểm nhằm cung cấp cái nhìn toàn diện về tác động của đại dịch và hiệu quả của hệ thống y tế. Phân tích tỷ lệ tử vong theo nhân khẩu học. Các thông số trong spark ta cấu hình như sau

```
def build_spark() -> SparkSession:
    return (
        SparkSession.builder.appName("Iceberg Gold Clinical
Analytics")
        .config("spark.sql.catalog.hospital",
"org.apache.iceberg.spark.SparkCatalog")
        .config("spark.sql.catalog.hospital.type", "hadoop")
        .config("spark.sql.catalog.hospital.warehouse",
"s3a://hospital-lakehouse/warehouse")
        .config("spark.hadoop.fs.s3a.endpoint", MINIO_ENDPOINT)
        .config("spark.hadoop.fs.s3a.access.key", MINIO_ACCESS_KEY)
        .config("spark.hadoop.fs.s3a.secret.key", MINIO_SECRET_KEY)
        .config("spark.hadoop.fs.s3a.path.style.access", "true")
        .config("spark.hadoop.fs.s3a.connection.ssl.enabled",
"false")
        .config("spark.hadoop.fs.s3a.impl",
"org.apache.hadoop.fs.s3a.S3AFileSystem")
        .config(
            "spark.hadoop.fs.s3a.aws.credentials.provider",
"org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider",
        )
        .config("spark.sql.defaultCatalog", CATALOG_NAME)
        .getOrCreate()
    )
```

Bài toán thứ nhất tập trung vào việc đánh giá mức độ rủi ro dựa trên các đặc điểm sinh học của bệnh nhân. Dữ liệu đầu vào được trích xuất từ bảng covid_clinical_master

tại tầng Silver, nơi lưu trữ thông tin bệnh nhân đã được làm sạch và xác định nhiễm COVID-19.

```
def create_demographics_mortality(spark: SparkSession):
    print("Đang phân tích Bảng 1: Phân tích tử vong theo Nhân Khẩu Học...")
    df_silver = spark.table(SILVER_MASTER)

    # 1. Tính tuổi (lấy năm hiện tại trừ năm sinh, hoặc năm mất trừ năm sinh)
    cur_year = F.year(F.current_date())
    age_col = F.when(F.col("patient_deathdate").isNotNull(),
F.year("patient_deathdate") - F.year("patient_birthdate")) \
        .otherwise(cur_year - F.year("patient_birthdate"))

    df_with_age = df_silver.withColumn("age", age_col)

    # 2. Phân loại nhóm tuổi
    df_with_group = df_with_age.withColumn("age_group",
        F.when(F.col("age") <= 18, "0-18")
        .when((F.col("age") > 18) & (F.col("age") <= 34), "19-34")
        .when((F.col("age") > 34) & (F.col("age") <= 49), "35-49")
        .when((F.col("age") > 49) & (F.col("age") <= 64), "50-64")
        .when(F.col("age") >= 65, "65+")
        .otherwise("Unknown")
    )

    # 3. Aggregate
    df_agg = df_with_group.groupBy("age_group", "gender").agg(
        F.count("patient_id").alias("total_covid_patients"),
        F.sum(F.when(F.col("patient_deathdate").isNotNull(),
1).otherwise(0)).alias("total_deaths")
    )
```

```

# 4. Tính tỷ lệ %
df_final = df_agg.withColumn(
    "mortality_rate_percent",
    F.round((F.col("total_deaths") /
F.col("total_covid_patients")) * 100, 2)
)

# Ghi ra bảng Gold
df_final.writeTo(TABLE_DEMOGRAPHICS).createOrReplace()
print("Đã tạo thành công bảng: " + TABLE_DEMOGRAPHICS)
df_final.show(10)

```

Logic xử lý được thực hiện qua nhiều bước. Đầu tiên là tính toán tuổi thực tế, hệ thống sử dụng hàm xử lý thời gian để tính toán tuổi của bệnh nhân tại thời điểm phân tích. Đối với những bệnh nhân đã tử vong, tuổi được xác định bằng khoảng cách giữa năm sinh và năm mất. Với những bệnh nhân còn sống, tuổi được tính dựa trên năm hiện tại trừ đi năm sinh. Tiếp theo là phân nhóm độ tuổi nhằm tạo ra các báo cáo có ý nghĩa về mặt thống kê y tế, bệnh nhân được chia vào 5 nhóm tuổi chiến lược: nhi khoa (0-18), thanh niên (19-34), trung niên (35-49, 50-64) và nhóm người cao tuổi (65+). Cuối cùng là tổng hợp chỉ số tử vong, với mỗi nhóm tuổi và giới tính, hệ thống thực hiện tổng hợp để đếm tổng số ca nhiễm và số ca tử vong. Từ đó, công thức tính tỷ lệ tử vong được áp dụng nhằm chỉ ra nhóm đối tượng có nguy cơ cao nhất, giúp các cơ quan y tế có phương án bảo vệ ưu tiên. Kết quả thu được như hình bên dưới

Parquet Viewer

SQL Query | Chart | Schema | Export

Enter your SQL query here
Example: SELECT * FROM data LIMIT 4

▶ EXECUTE ◀ CLEAR QUERY

◀ PREVIOUS ▶ NEXT

Showing row #1 to #10 of 10 total

age_group - str	gender - str	total_covid_patients - i64	total_deaths - i64	mortality_rate_percent - f64
65+	Female	11484	1065	9.27
50-64	Female	9889	322	3.26
50-64	Male	9287	497	5.35
35-49	Female	8540	52	0.61
35-49	Male	8008	154	1.92
65+	Male	10970	1379	12.57

Hình 2.2.3.1 Kết quả của bài toán thứ nhất

Bài toán thứ hai đi sâu vào việc tái hiện lộ trình điều trị của bệnh nhân từ khi phát hiện bệnh cho đến khi kết thúc quá trình lâm sàng tức là phân tích luồng điều trị. Điểm đặc biệt của quy trình này là sự kết hợp dữ liệu liên tầng và hệ thống sử dụng danh sách bệnh nhân nhiễm bệnh từ tầng Silver, sau đó truy vấn ngược lại tầng Bronze tại các bảng encounters để lấy lịch sử thăm khám và conditions tức tình trạng bệnh lý để tìm kiếm các dấu hiệu lâm sàng chi tiết.

```
def create_pathway_summary(spark: SparkSession):
    print("Đang phân tích Bảng 2: Thống kê Phân luồng điều trị (Pathways)...")
    df_silver = spark.table(SILVER_MASTER)
    df_encounters = spark.table(BRONZE_ENCOUNTERS)
    df_conditions = spark.table(BRONZE_CONDITIONS)

    # Danh sách tập bệnh nhân COVID
    covid_patients = df_silver.select("patient_id",
    "patient_deathdate")

    # Gắn flag tử vong / hồi phục
    covid_patients = covid_patients.withColumn("is_dead",
    F.when(F.col("patient_deathdate").isNotNull(), 1).otherwise(0))
```



```

covid_patients = covid_patients.withColumn("is_recovered",
F.when(F.col("patient_deathdate").isNull(), 1).otherwise(0))

# Xét các luồng nhập viện / ICU dựa trên lịch sử Khám (Encounters)
# Bronze encounters dùng cột 'patient', Silver dùng 'patient_id'
encounters_flagged = df_encounters.join(
    covid_patients,
    df_encounters["patient"] == covid_patients["patient_id"],
    how="inner"
).withColumn(
    "is_hospitalized",
F.when(F.lower(F.col("encounterclass")).like("%inpatient%"),
1).otherwise(0)
    ).withColumn(
        "is_icu",
        F.when(F.lower(F.col("encounterclass")).like("%icu%")
F.lower(F.col("encounterclass")).like("%intensive%"),
1).otherwise(0)
        )

    patient_enc_agg = encounters_flagged.groupBy(covid_patients["patient_id"]).agg(
        F.max("is_hospitalized").alias("hospitalized_flag"),
        F.max("is_icu").alias("icu_flag")
    )

# Nội suy thở máy (Mechanical Ventilation) từ Conditions /
Encounters description
# Bronze conditions dùng cột 'patient', không phải 'patient_id'
vent_cond = df_conditions.join(
    covid_patients,
    df_conditions["patient"] == covid_patients["patient_id"],

```

```

        how="inner"
    ).withColumn(
        "is_vent",
        F.when(F.lower(F.col("description")).like("%ventilator%") |
            F.lower(F.col("description")).like("%ventilation%")
        |
            F.lower(F.col("description")).like("%hypoxemia%"),
        1).otherwise(0)
    )
    patient_vent_agg =
vent_cond.groupBy(covid_patients["patient_id"]).agg(F.max("is_vent"
).alias("vent_flag"))

    # Tổng hợp tất cả về 1 bảng Master Passway
    master_pathway = covid_patients.join(patient_enc_agg,
on="patient_id", how="left") \
        .join(patient_vent_agg,
on="patient_id", how="left")

    master_pathway = master_pathway.fillna(0,
subset=["hospitalized_flag", "icu_flag", "vent_flag"])

    # Cách ly tại nhà = Không có nhập viện
    master_pathway =
master_pathway.withColumn("home_isolation_flag",
F.when(F.col("hospitalized_flag") == 0, 1).otherwise(0))

    # Tóm tắt số liệu
    df_summary = master_pathway.agg(
        F.count("patient_id").alias("total_covid_cases"),
        F.sum("home_isolation_flag").alias("home_isolation_cases"),
        F.sum("hospitalized_flag").alias("hospitalized_cases"),
        F.sum("icu_flag").alias("icu_cases"),

```

```

        F.sum("vent_flag").alias("ventilation_cases"),
        F.sum("is_recovered").alias("recovered_cases"),
        F.sum("is_dead").alias("death_cases")
    )

    # Ghi ra bảng Gold
    df_summary.writeTo(TABLE_PATHWAYS).createOrReplace()
    print("Đã tạo thành công bảng: " + TABLE_PATHWAYS)
    df_summary.show()

```

Logic phân tích luồng điều trị bao gồm: Xác định mức độ can thiệp y tế: Dựa trên lịch sử thăm khám, hệ thống lọc ra các bản ghi có loại hình là inpatient để xác định bệnh nhân có phải nhập viện hay không. Các từ khóa chuyên môn như icu hoặc intensive được sử dụng để đánh dấu những ca bệnh nặng cần chăm sóc đặc biệt. Nội suy nhu cầu hỗ trợ hô hấp: Thông qua việc phân tích mô tả bệnh lý trong bảng conditions, hệ thống xác định các bệnh nhân có triệu chứng suy hô hấp hoặc cần can thiệp máy thở. Đây là chỉ số quan trọng để đánh giá mức độ nghiêm trọng của ca bệnh. Phân loại kết quả lâm sàng: Bệnh nhân được phân luồng vào các trạng thái cuối cùng bao gồm cách ly tại nhà, nhập viện điều trị, điều trị tích cực hay ICU, hồi phục hoặc tử vong. Lưu trữ và Quản trị dữ liệu Gold Layer Kết quả của các phép phân tích trên không chỉ được hiển thị ra màn hình mà được ghi trực tiếp vào Namespace gold dưới dạng các bảng Apache Iceberg. Việc sử dụng định dạng Iceberg tại tầng này mang lại lợi thế vượt trội về khả năng quản lý phiên bản, cho phép các nhà phân tích dữ liệu so sánh các chỉ số dịch tễ theo từng mốc thời gian khác nhau của đại dịch. Toàn bộ quy trình này chuyển hóa các dòng dữ liệu thô ban đầu thành những tri thức y khoa có giá trị, giúp trả lời các câu hỏi thực tế như: "Nhóm tuổi nào dễ bị tổn thương nhất?" hay "Tỷ lệ bệnh nhân cần dùng máy thở chiếm bao nhiêu phần trăm trong số các ca nhập viện?", từ đó hoàn thiện giá trị của hệ thống Data Lakehouse trong lĩnh vực y tế. Kết quả thu được sẽ như sau

Parquet Viewer

Back

Limited view mode - please purchase more credit to unlock all features.

SQL Query Chart Schema Export

Enter your SQL query here
Example: SELECT * FROM data LIMIT 4

▶ EXECUTE ◀ CLEAR QUERY

◀ PREVIOUS ▶ NEXT

Showing row #1 to #1 of 1 total

total_covid_cases - i64	home_isolation_cases - i64	hospitalized_cases - i64	icu_cases - i64	ventilation_cases - i64	recovered_cases - i64	death_cases - i64
88166	61188	26978	0	18175	84525	3641

◀ PREVIOUS ▶ NEXT

Hình 2.2.3.2 Kết quả của bài toán thứ 2

Tiếp nối các phân tích trên, quy trình xử lý tại `gold_aggregation.py` được triển khai để mở rộng khả năng khai thác dữ liệu, tập trung vào mối quan hệ giữa các triệu chứng lâm sàng và kết quả điều trị, cũng như đánh giá gánh nặng công việc tại hệ thống y tế. Các cấu hình bổ sung cho công việc này bao gồm:

```
# Cấu hình Namespace và Bảng đích cho Gold Layer
GOLD_NAMESPACE = "hospital.gold"
OUTPUT_TABLE = f"{GOLD_NAMESPACE}.symptoms_outcomes"
MORTALITY_TABLE = f"{GOLD_NAMESPACE}.mortality_demographics"
HOSPITALIZATION_TABLE = f"{GOLD_NAMESPACE}.hospitalization_workload"
```

Bài toán thứ ba: Phân tích Triệu chứng và Biến chứng lâm sàng (Symptoms & Outcomes)

Bài toán này tập trung vào việc thống kê tỷ lệ các biến chứng nghiêm trọng như Sepsis (nhiễm trùng huyết), ARDS (suy hô hấp), Heart Failure (suy tim) và các triệu chứng điển hình (Cough, Fever) xuất hiện sau khi bệnh nhân nhiễm COVID-19. Mục tiêu là so sánh sự khác biệt về mặt lâm sàng giữa nhóm Sống sót (Survivors) và nhóm Tử vong (Non-survivors).

```
# 1. Xác định Ngày nhiễm COVID-19 của từng bệnh nhân
covid_diagnosis_df = (
    conditions_df.filter(F.col("condition_code") == "840539006")
    .groupBy("patient_id")
```

```

.agg(F.min("condition_start_date").alias("covid_diagnosis_date"))
)

# 2. Lọc các tình trạng bệnh lý bắt đầu sau ngày chẩn đoán COVID
analysis_df = cohort_df.join(covid_diagnosis_df, on="patient_id",
how="inner") \
                        .join(filtered_conditions, on="patient_id",
how="inner")

post_covid_conditions = analysis_df.filter(
    F.col("condition_start_date") >= F.col("covid_diagnosis_date")
)

# 3. Tổng hợp tỷ lệ % theo từng nhóm triệu chứng và kết quả điều trị
final_df = agg_df.withColumn(
    "rate_percentage",
    F.round((F.col("patient_count") /
F.col("total_group_population")) * 100, 2)
)

```

Logic xử lý chính:

Hệ thống chỉ ghi nhận các biến chứng nếu chúng khởi phát cùng ngày hoặc sau ngày chẩn đoán COVID-19 chính thức. Điều này giúp loại bỏ các bệnh nền có sẵn (pre-existing conditions) để tập trung hoàn toàn vào tác động trực tiếp của virus.

Sử dụng biểu thức chính quy (RegEx) để chuẩn hóa các mô tả lâm sàng phức tạp thành các nhóm dễ hiểu. Kết quả cuối cùng cung cấp tỷ lệ % mắc bệnh trên tổng quy mô của từng phân khúc (Sống/Chết), giúp xác định các "dấu hiệu chỉ báo" (red flags) có liên quan chặt chẽ đến nguy cơ tử vong.

Tiến hành query kiểm tra dùng SuperSet SQL Lab (qua Trino):

```

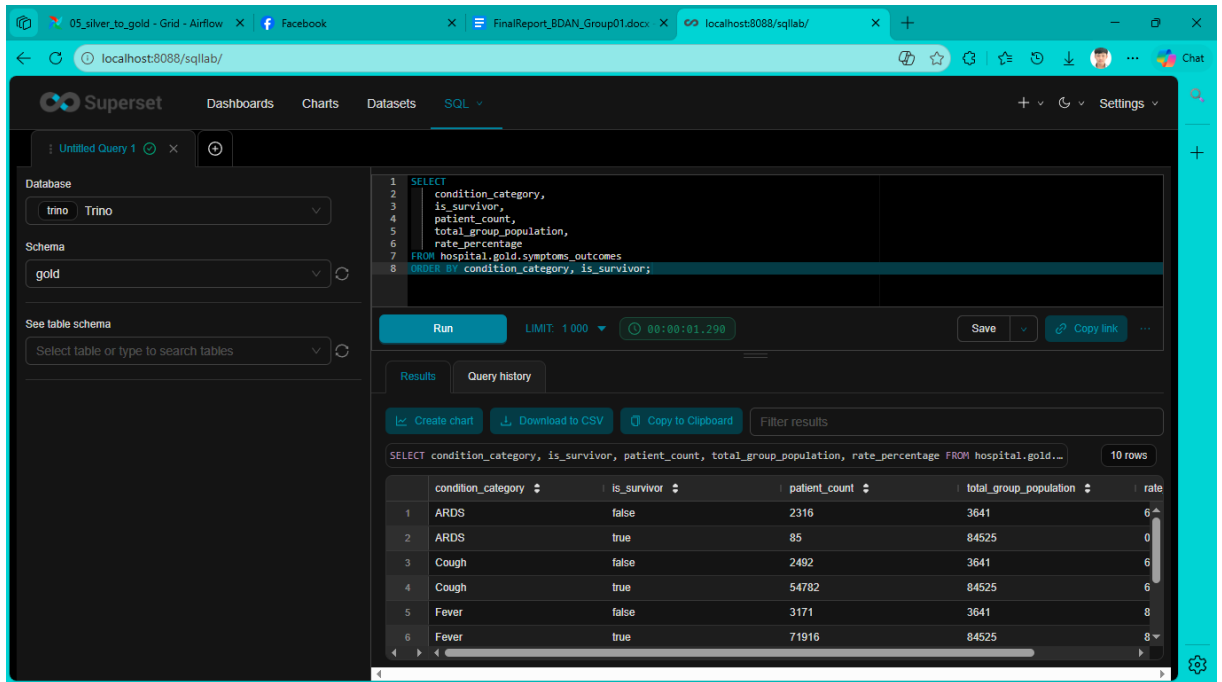
SELECT
    condition_category,
    is_survivor,
    patient_count,

```

```

total_group_population,
rate_percentage
FROM hospital.gold.symptoms_outcomes
ORDER BY condition_category, is_survivor;

```



The screenshot shows the Superset SQL Lab interface. The SQL query is: `SELECT condition_category, is_survivor, patient_count, total_group_population, rate_percentage FROM hospital.gold.symptoms_outcomes ORDER BY condition_category, is_survivor;`. The results table displays 10 rows of data.

	condition_category	is_survivor	patient_count	total_group_population	rate
1	ARDS	false	2316	3641	63.61
2	ARDS	true	85	84525	0.1
3	Cough	false	2492	3641	68.17
4	Cough	true	54782	84525	64.69
5	Fever	false	3171	3641	86.84
6	Fever	true	71916	84525	85.08

Hình: Kết quả của bài toán thứ ba

Qua kết quả, ta có thể kết luận:

Triệu chứng nền không mang tính phân cấp. Ho (Cough) và Sốt (Fever) là các triệu chứng đặc trưng của COVID-19 xuất hiện với tỷ lệ rất cao và gần như tương đồng ở cả hai nhóm (~65-68% với Ho và ~85-87% với Sốt).

Dấu hiệu cảnh báo tử vong cao (Red Flags). Có sự chênh lệch không nhỏ về sự xuất hiện của các biến chứng nghiêm trọng đối với nhóm bệnh nhân không qua khỏi:

Nhiễm trùng huyết (Sepsis) có mặt ở gần như toàn bộ ca tử vong (96.79%) nhưng chỉ có ở 4.04% ca được chữa khỏi.

Suy hô hấp (ARDS) kéo theo 63.61% ca tử vong (so với chỉ 0.1% ở ca sống).

Suy tim (Heart Failure) chiếm 35.98% ca tử vong (so với chỉ 0.21% ca sống).

=> Tóm lại: Nhiễm trùng huyết, Suy hô hấp cấp và Suy tim chính là những biến chứng quyết định và là nguyên nhân trực tiếp nhất dẫn đến nguy cơ tử vong cao ở các bệnh nhân nhiễm COVID-19. Mọi phác đồ điều trị cần đặc biệt ưu tiên theo dõi và ngăn chặn 3 biến chứng này.

Bài toán thứ tư: Phân tích Tỷ lệ tử vong chi tiết theo Thập kỷ

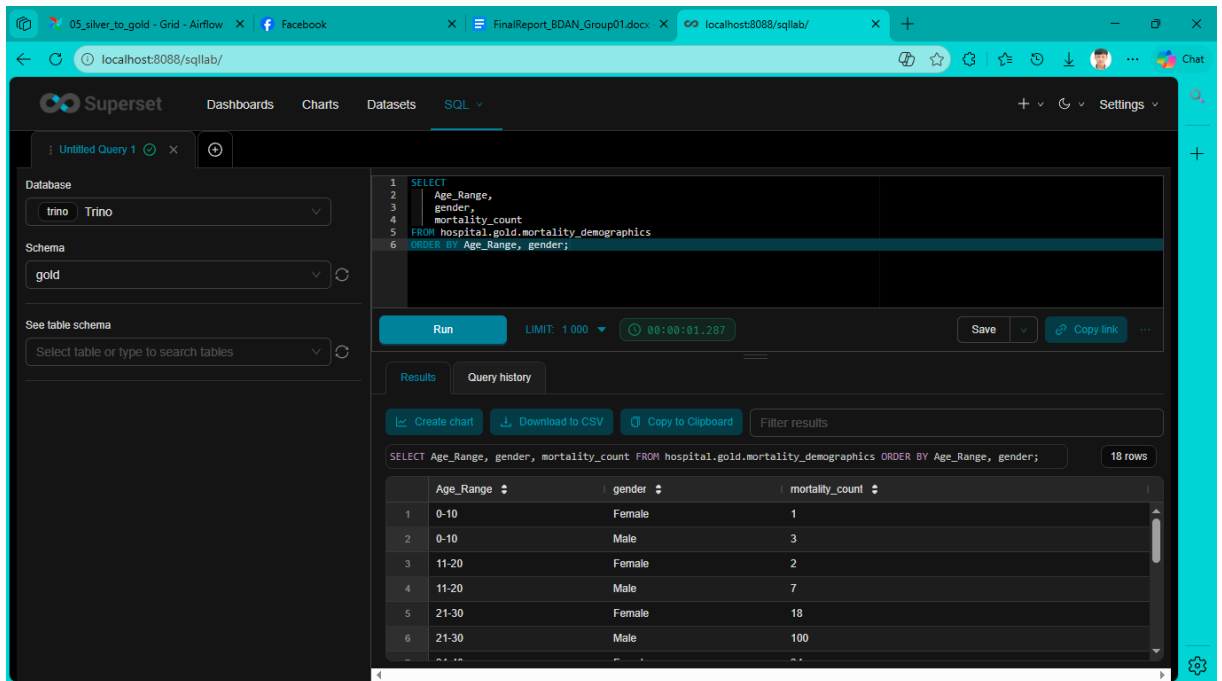
Thay vì chỉ phân nhóm rộng, bài toán này thực hiện phân tích sâu về mật độ tử vong theo dải tuổi 10 năm (Decade-based binning). Điều này cho phép quan sát sự thay đổi rủi ro sinh học theo từng giai đoạn lão hóa của con người.

```
# Phân nhóm tuổi lúc tử vong theo dải 10 năm
mortality_binned = mortality_with_age.withColumn(
    "Age_Range",
    F.when(F.col("age_at_death") <= 10, "0-10")
      .when(F.col("age_at_death") <= 20, "11-20")
      .when(F.col("age_at_death") <= 30, "21-30")
      .when(F.col("age_at_death") <= 40, "31-40")
      .when(F.col("age_at_death") <= 50, "41-50")
      .when(F.col("age_at_death") <= 60, "51-60")
      .when(F.col("age_at_death") <= 70, "61-70")
      .when(F.col("age_at_death") <= 80, "71-80")
      .otherwise("80+")
)
```

Dữ liệu hiển thị rõ ràng nhóm đối tượng nào chịu ảnh hưởng nặng nề nhất, phục vụ cho việc lập kế hoạch ưu tiên tiêm chủng và bảo vệ các nhóm dân cư yếu thế.

Tiến hành query bài toán thứ tư:

```
SELECT
    Age_Range,
    gender,
    mortality_count
FROM hospital.gold.mortality_demographics
ORDER BY Age_Range, gender;
```



Hình: Kết quả bài toán thứ tư

Qua kết quả, ta thấy được:

Độ tuổi tỷ lệ thuận với rủi ro tử vong. Số ca tử vong rất thấp ở trẻ em và người trẻ tuổi (dưới 30), bắt đầu tăng mạnh từ mốc 50 tuổi trở lên. Nhóm người cao tuổi (từ 70 tuổi trở lên) chịu ảnh hưởng nặng nề và chiếm tỷ trọng tử vong cao nhất.

Nam giới có rủi ro tử vong cao hơn nữ giới tuyệt đối. Ở mọi dải tuổi, số ca tử vong ở Nam giới luôn lớn hơn Nữ giới. Tại một số độ tuổi như 21-30 hay 41-50, tỷ lệ tử vong ở nam giới cao gấp 3 đến 5 lần so với nữ giới.

=> Tóm lại: Đặc điểm nhân khẩu học có rủi ro sinh học cao nhất trước COVID-19 là người lớn tuổi (trên 50 tuổi) và mang giới tính Nam. Đây là nhóm đối tượng yếu thế cần được ưu tiên hàng đầu trong các kế hoạch tiêm chủng, phân bổ tài nguyên y tế và bảo vệ nghiêm ngặt.

Bài toán thứ năm: Phân tích Gánh nặng Y tế và Thời gian lưu trú (ALOS)

Bài toán cuối cùng tập trung vào khía cạnh vận hành y tế, đo lường tỷ lệ nhập viện (Hospitalization), nhập ICU và tính toán Average Length of Stay (ALOS - Số ngày nằm viện trung bình).

```
# Tính số ngày nằm viện (Length of Stay)
hospitalization_df = covid_encounters.withColumn(
    "length_of_stay_days",
```



```

        F.datediff("encounter_stop", "encounter_start")
    )

# Tổng hợp ALOS và tỷ lệ sử dụng dịch vụ y tế
hospitalization_agg                                =
hospitalization_df.groupby("encounter_description").agg(
    F.countDistinct("patient_id").alias("patient_count"),
    F.round(F.avg("length_of_stay_days"),
2).alias("avg_length_of_stay_days")
)

```

Thông qua việc lọc các mã định danh y tế chuẩn hóa (1505002 cho Nhập viện thường và 305351004 cho ICU), hệ thống cung cấp bức tranh thực tế về cường độ sử dụng tài nguyên bệnh viện. Chỉ số ALOS là thước đo quan trọng để ước tính chi phí điều trị và lập kế hoạch luân chuyển giường bệnh trong các giai đoạn cao điểm của dịch bệnh.

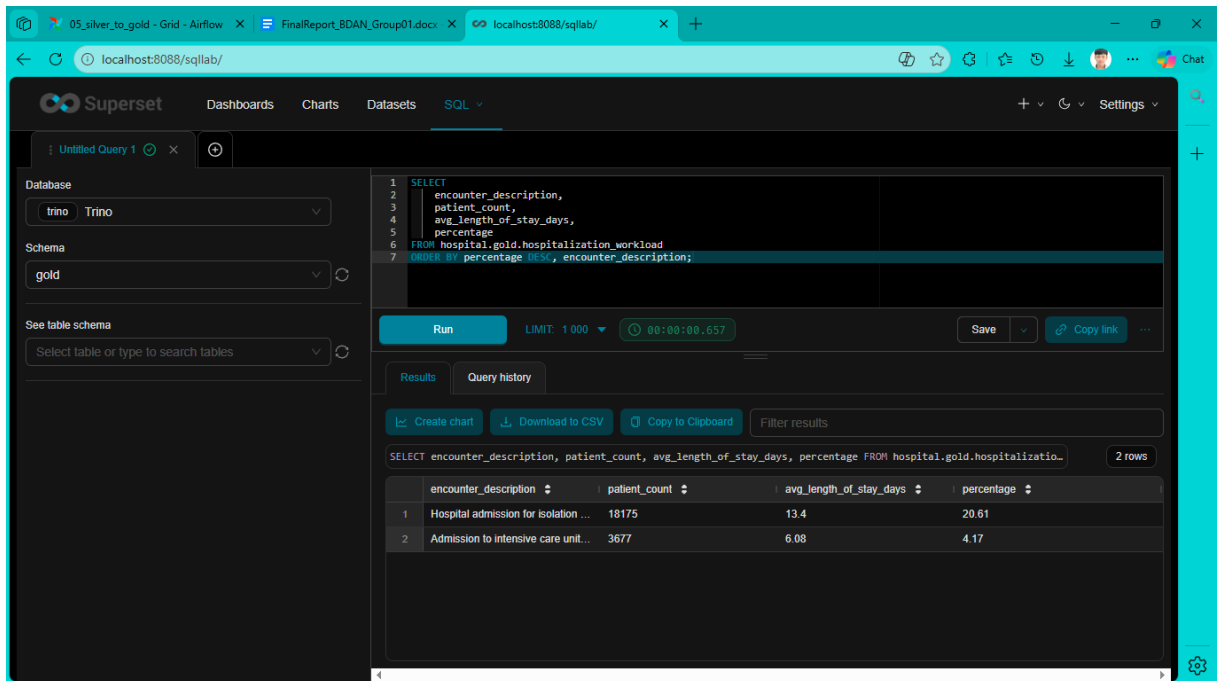
Toàn bộ kết quả từ ba bài toán bổ sung này được lưu trữ vào lớp Gold dưới định dạng Iceberg, sẵn sàng cho việc kết nối với các công cụ Superset để xây dựng các Dashboard giám sát dịch tễ học thời gian thực.

Tiến hành query bài toán thứ năm:

```

SELECT
    encounter_description,
    patient_count,
    avg_length_of_stay_days,
    percentage
FROM hospital.gold.hospitalization_workload
ORDER BY percentage DESC, encounter_description;

```



Qua kết quả, ta thấy được:

Gánh nặng lên khối phòng bệnh thông thường (Isolation). Hơn 1/5 tổng số ca nhiễm COVID-19 (20.61%) phải nhập viện cách ly. Đáng chú ý, thời gian nằm viện trung bình (ALOS) rất dài, lên tới 13.4 ngày. Điều này cho thấy tỷ lệ luân chuyển giường bệnh cực kỳ chậm (gần 2 tuần/bệnh nhân), gây sức ép khổng lồ lên hệ thống lưu trú y tế cơ sở.

Áp lực lên Hồi sức tích cực (ICU). Có khoảng 4.17% bệnh nhân chuyển nặng phải nhập ICU với thời gian nằm ICU trung bình là 6.08 ngày. Mặc dù tỷ lệ bệnh nhân không quá lớn, nhưng nguồn lực để duy trì hệ thống ICU là rất tốn kém và mang tính kịch trần.

=> Thách thức vận hành y tế lớn nhất trong đại dịch không chỉ đến từ số lượng người mắc, mà nằm ở việc thời gian điều trị nội trú quá dài (hơn 13 ngày). Việc dự báo và chuẩn bị số lượng giường bệnh cách ly hoặc đưa ra các quy định tự cách ly tại nhà là bài toán bắt buộc để giúp bệnh viện không bị vỡ trận tuyến đầu không bị "vỡ trận".

2.2.4 Tự động hóa quy trình bằng Airflow DAGs

Trong kiến trúc Data Lakehouse, việc tự động hóa các luồng dữ liệu đóng vai trò quyết định đến tính kịp thời và độ chính xác của thông tin. Một DAG đại diện cho một tập hợp các tác vụ được tổ chức theo cấu trúc đồ thị có hướng, đảm bảo dữ liệu chảy qua các layer của hệ thống một cách có kiểm soát. Sự kết hợp giữa khả năng điều phối

của Airflow và sức mạnh tính toán của Apache Spark tạo nên một hệ thống pipeline tự động hoàn chỉnh, xuyên suốt từ khâu tiếp nhận dữ liệu cho đến tầng Gold Layer.

Để hiểu rõ cách thức vận hành của pipeline này, chúng ta sẽ phân tích chi tiết file cấu hình mẫu 01_csv_to_postgres.py. Đây là điểm khởi đầu của toàn bộ hệ thống, nơi dữ liệu thô từ các tập tin CSV được nạp vào cơ sở dữ liệu PostgreSQL.

```
from airflow import DAG
from airflow.providers.apache.spark.operators.spark_submit import
SparkSubmitOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'lakehouse_admin',
    'depends_on_past': False,
    'start_date': datetime(2023, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    '01_csv_to_postgres',
    default_args=default_args,
    description='Step 1: Ingest raw CSV data from source to
PostgreSQL',
    schedule_interval=None,
    catchup=False,
    tags=['lakehouse', 'ingestion', 'step1']
) as dag:

    # File JAR Postgres cần thiết
    POSTGRES_JAR = '/opt/spark/jars/postgresql-42.7.3.jar'

    ingest_patients = SparkSubmitOperator(
```

```

        task_id='ingest_patients',
        application='/opt/spark/scripts/ingest_patients.py',
        conn_id='spark_default',
        jars=POSTGRES_JAR,
        driver_class_path=POSTGRES_JAR,
        name='ingest_patients'
    )

    ingest_encounters = SparkSubmitOperator(
        task_id='ingest_encounters',
        application='/opt/spark/scripts/ingest_encounters.py',
        conn_id='spark_default',
        jars=POSTGRES_JAR,
        driver_class_path=POSTGRES_JAR,
        name='ingest_encounters'
    )

    ingest_conditions = SparkSubmitOperator(
        task_id='ingest_conditions',
        application='/opt/spark/scripts/ingest_conditions.py',
        conn_id='spark_default',
        jars=POSTGRES_JAR,
        driver_class_path=POSTGRES_JAR,
        name='ingest_conditions'
    )

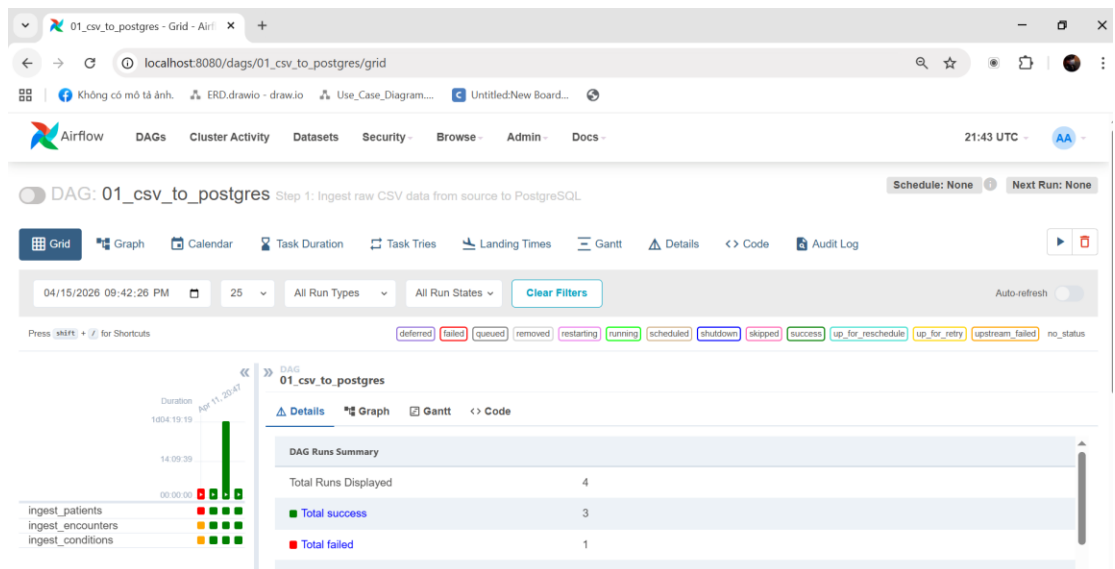
    # Chạy lần lượt (vì có Foreign Key constraints: patients ->
    encounters -> conditions)
    ingest_patients >> ingest_encounters >> ingest_conditions

```

Cấu trúc của một file DAG điển hình bao gồm ba phần chính: thiết lập tham số mặc định, định nghĩa DAG và thiết lập thứ tự thực thi. Trong phần `default_args`, các tham số như `'retries': 1` và `'retry_delay': timedelta(minutes=5)` được thiết lập. Đây là cơ chế tự động hóa quan trọng giúp Airflow có khả năng tự phục hồi (Self-healing); nếu một tiến trình Spark gặp sự cố do nghẽn mạng hoặc tài nguyên, hệ thống sẽ không dừng lại ngay lập tức mà tự động thử lại sau một khoảng thời gian định sẵn, giúp tăng tính bền bỉ cho pipeline.

Việc định nghĩa đối tượng DAG với các thuộc tính như `schedule_interval=None` cho phép người vận hành kích hoạt pipeline thủ công trong giai đoạn thử nghiệm hoặc cấu hình chạy định kỳ theo thời gian thực khi đưa vào vận hành. Trái tim của mỗi tác vụ trong đồ án này là `SparkSubmitOperator`. Công cụ này đóng vai trò là cầu nối liên lạc giữa Airflow và Spark. Ví dụ, trong tác vụ `ingest_patients`, mã nguồn chỉ định rõ tệp xử lý (`application='/opt/spark/scripts/ingest_patients.py'`) và các thư viện cần thiết (`jars=POSTGRES_JAR`). Việc tách biệt logic điều phối (trong Airflow) và logic xử lý dữ liệu (trong Spark Python script) giúp hệ thống đạt được tính module hóa cao, dễ dàng bảo trì và nâng cấp từng thành phần độc lập.

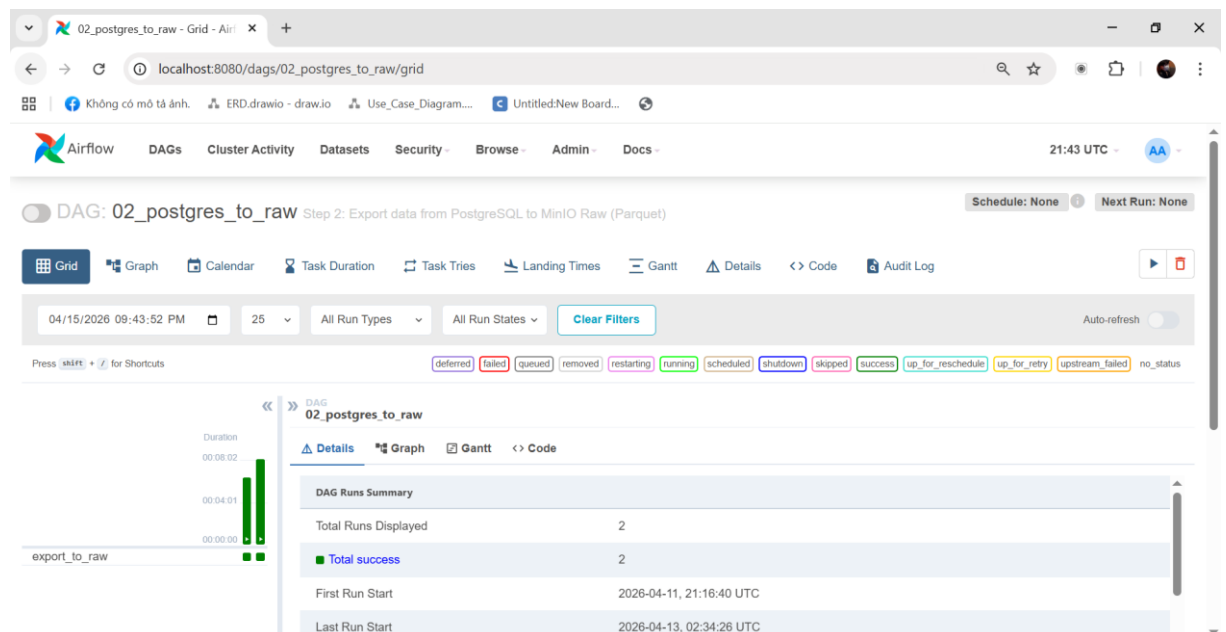
Một khía cạnh kỹ thuật quan trọng khác là việc thiết lập sự phụ thuộc giữa các tác vụ. Trong file mẫu, dòng mã `ingest_patients >> ingest_encounters >> ingest_conditions` sử dụng toán tử dịch chuyển bit để quy định thứ tự chạy. Trong nghiệp vụ y tế, thông tin về lượt khám phải luôn đi sau hồ sơ bệnh nhân để đảm bảo tính toàn vẹn của khóa ngoại. Airflow đảm bảo rằng tác vụ tiếp theo chỉ được kích hoạt khi tác vụ đứng trước nó đã hoàn thành thành công.



Hình 2.2.4.1 Tiến trình thực thi của DAG 1

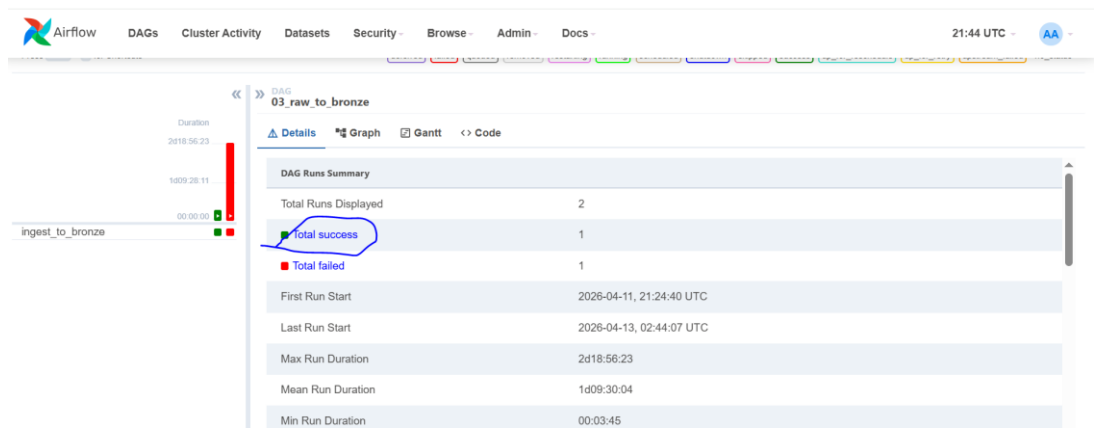
Quy trình tự động hóa này được thiết lập nhất quán qua các giai đoạn tiếp theo của mô hình Medallion:

Giai đoạn Ingestion (02_postgres_to_raw): Chuyển đổi dữ liệu sang định dạng Parquet trên MinIO.



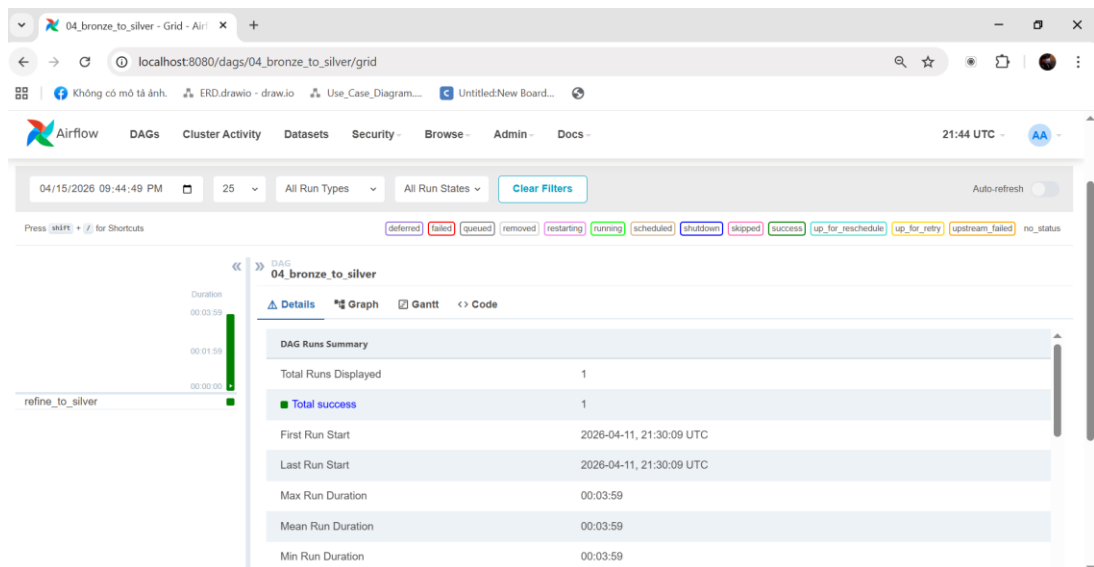
Hình 2.2.4.2 Tiến trình thực thi của DAG 2

Giai đoạn Bronze (03_raw_to_bronze): Nạp dữ liệu vào bảng Iceberg, thiết lập metadata cơ bản.



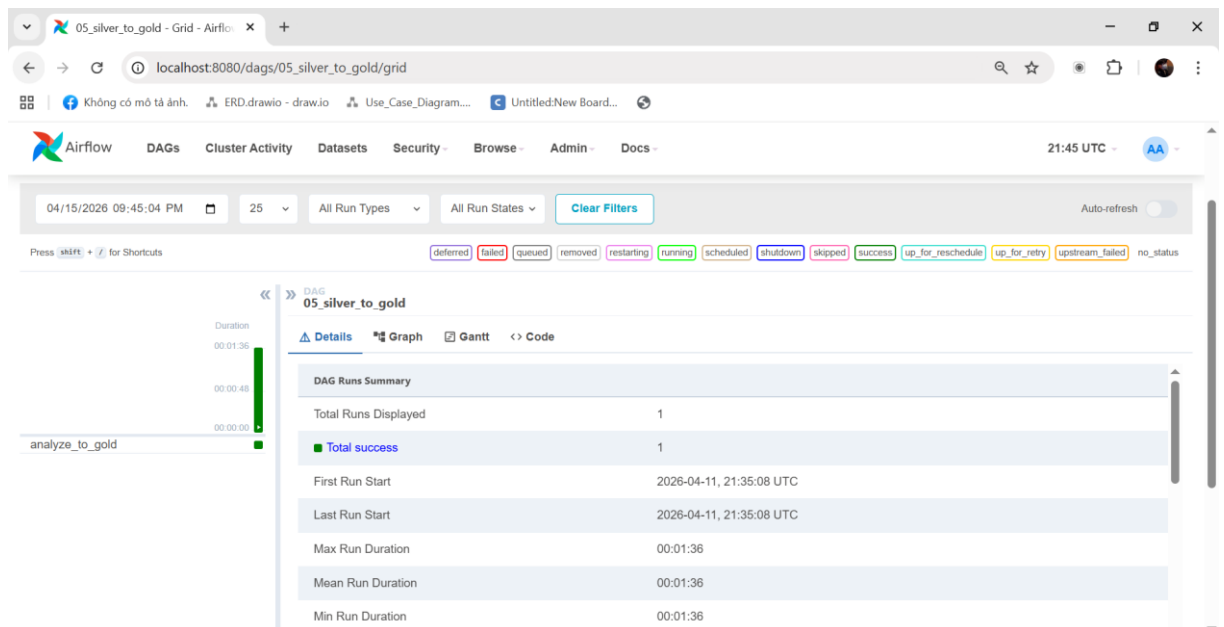
Hình 2.2.4.3 Tiến trình thực thi của DAG 3

Giai đoạn Silver (04_bronze_to_silver): Thực hiện làm sạch và chuẩn hóa dữ liệu.



Hình 2.2.4.4 Tiến trình thực thi của DAG 4

Giai đoạn Gold (05_silver_to_gold): Thực hiện các phép tính toán chỉ số lâm sàng chuyên sâu.

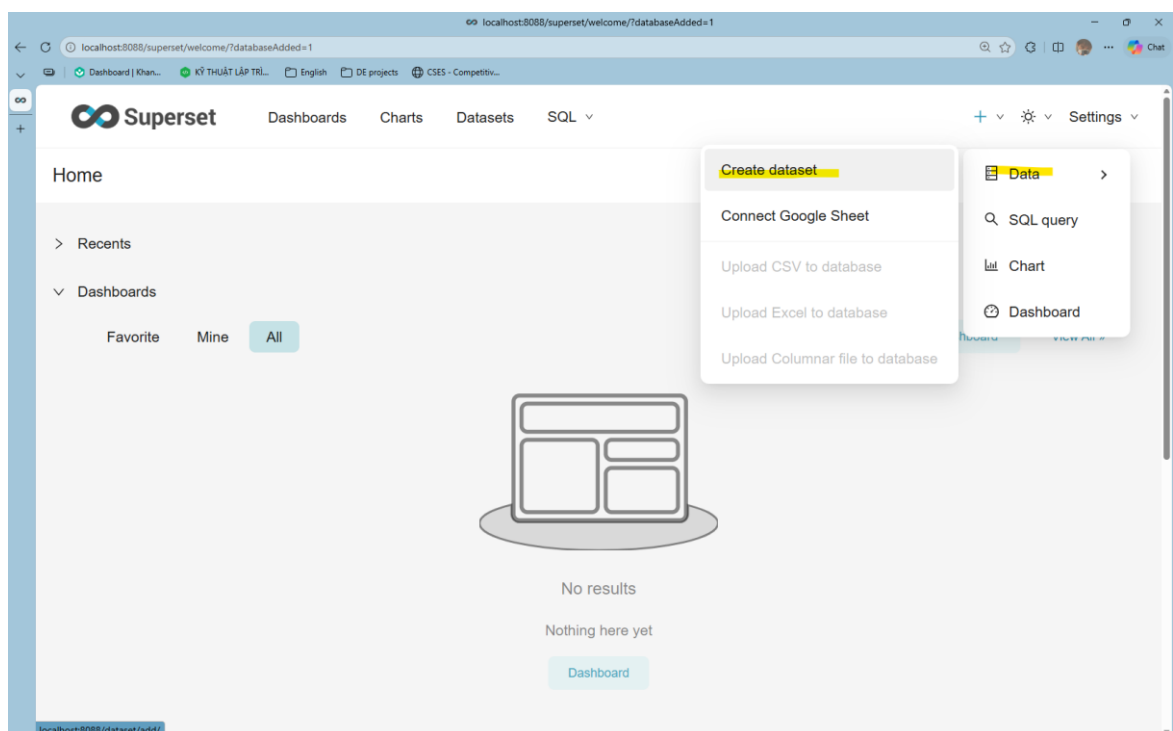


Hình 2.2.4.5 Tiến trình thực thi của DAG 5

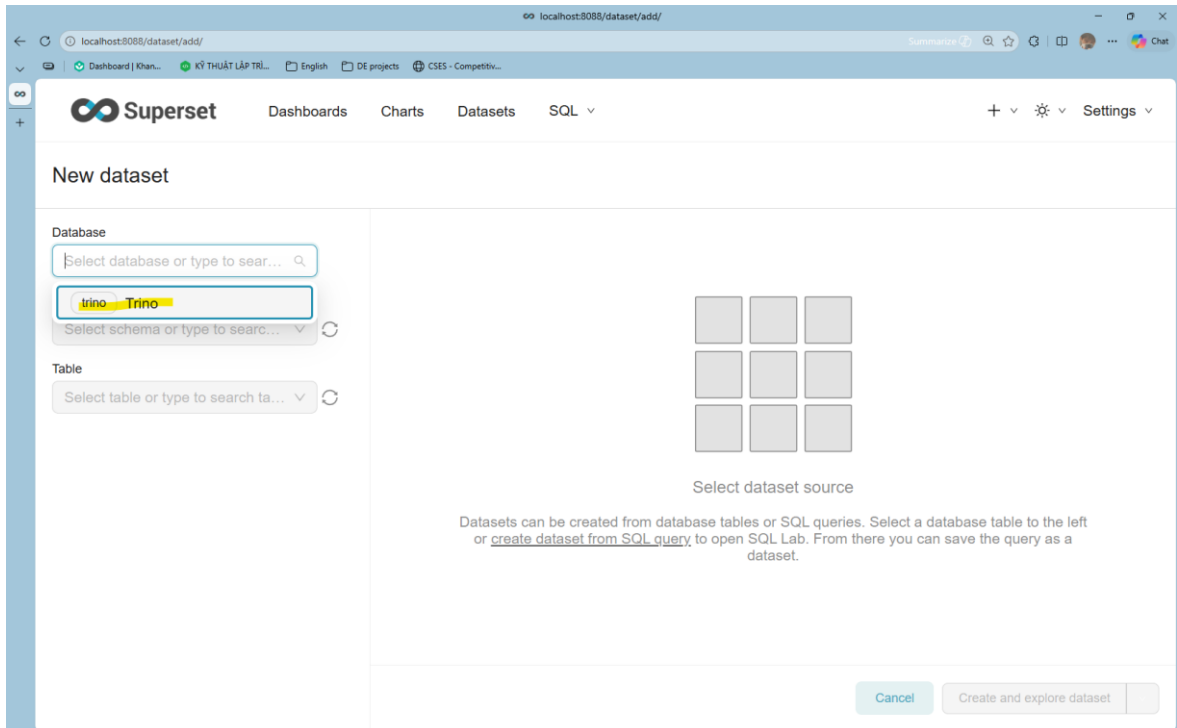
Việc chia nhỏ pipeline thành các file DAG độc lập theo từng layer thay vì một file duy nhất giúp kỹ sư dữ liệu dễ dàng giám sát và xử lý lỗi. Nếu dữ liệu tại tầng Silver không đạt yêu cầu về chất lượng, chúng ta có thể dừng pipeline tại đó để kiểm tra mà

2.3. Khai thác và trình diễn dữ liệu bằng Trino và Apache Superset

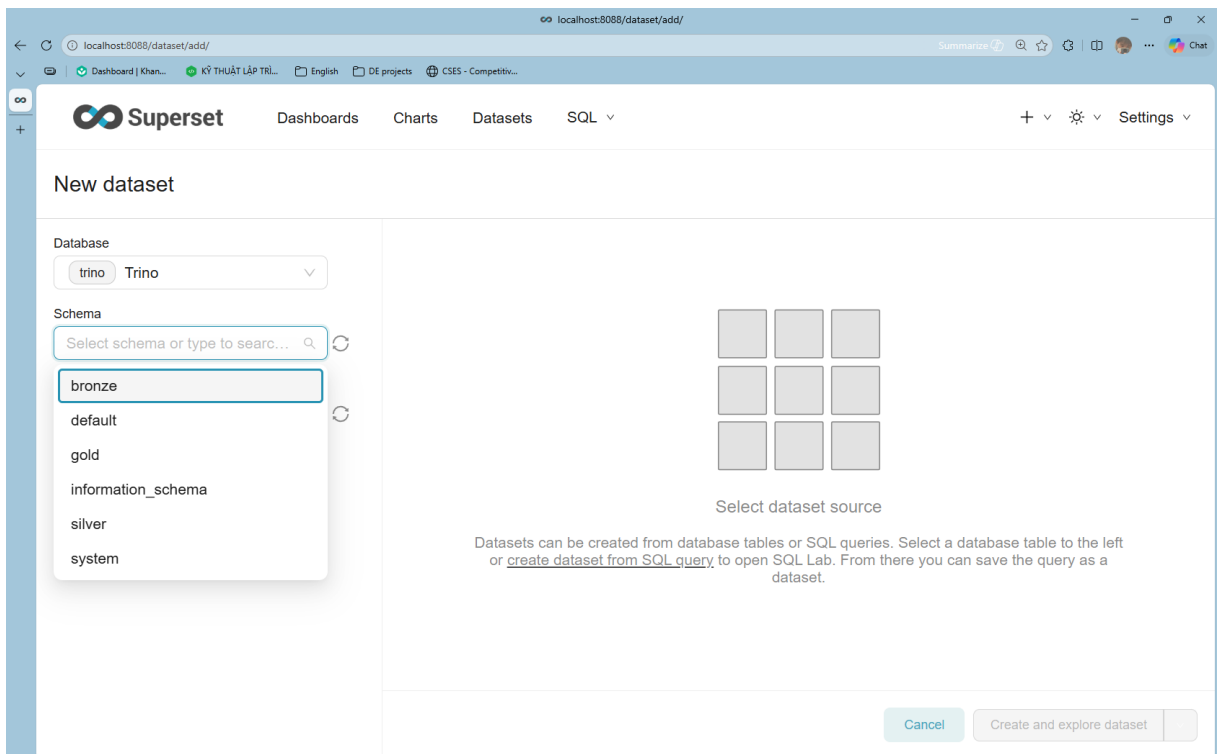
Sau khi đã connect database rồi thì ta vào lại Data > Create dataset



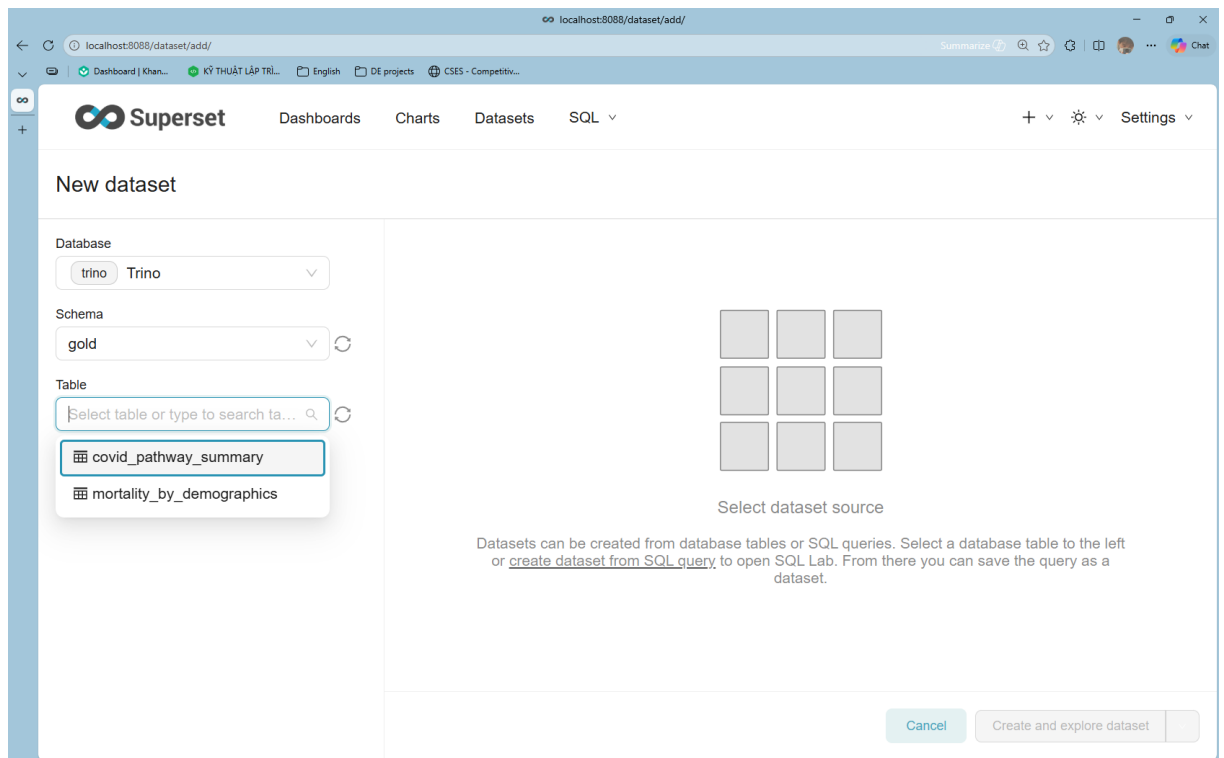
Chọn Database là Trino



Chọn Schema từ các layer. Ở đây nhóm phân tích từ Gold



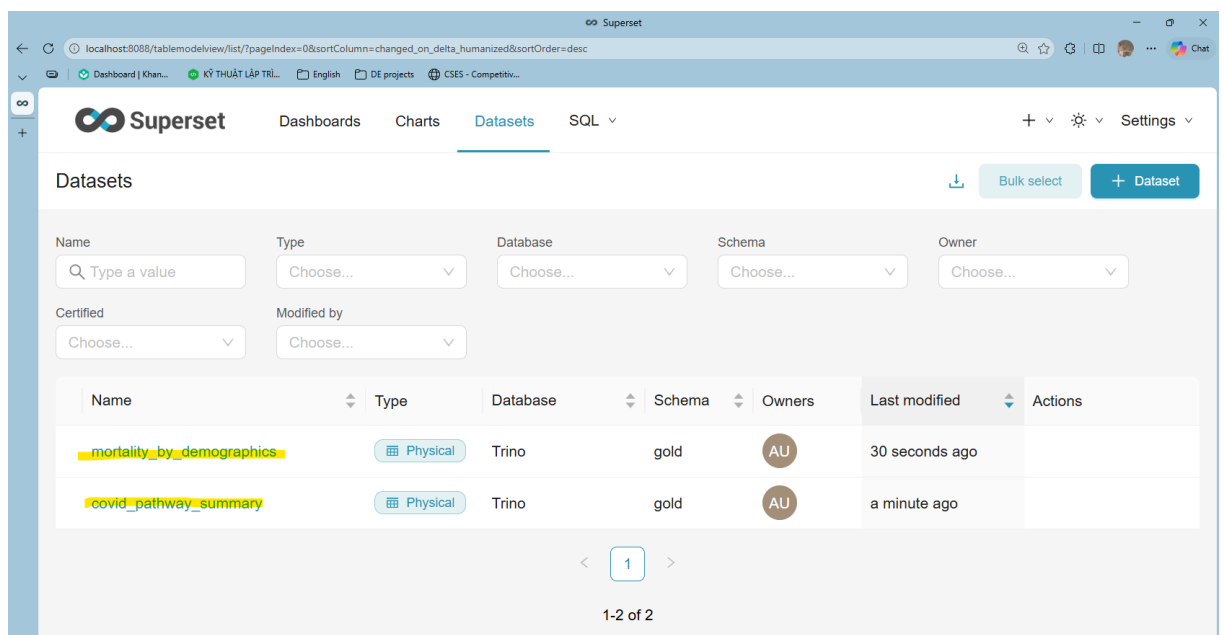
Và cuối cùng là chọn bảng để tạo datasets.



2.3.1. Tạo datasets

Nhóm tạo 2 datasets từ 2 bảng trong lớp gold, phục vụ cho mục đích khác nhau:

- gold.mortality_by_demographics
- gold.covid_pathway_summary

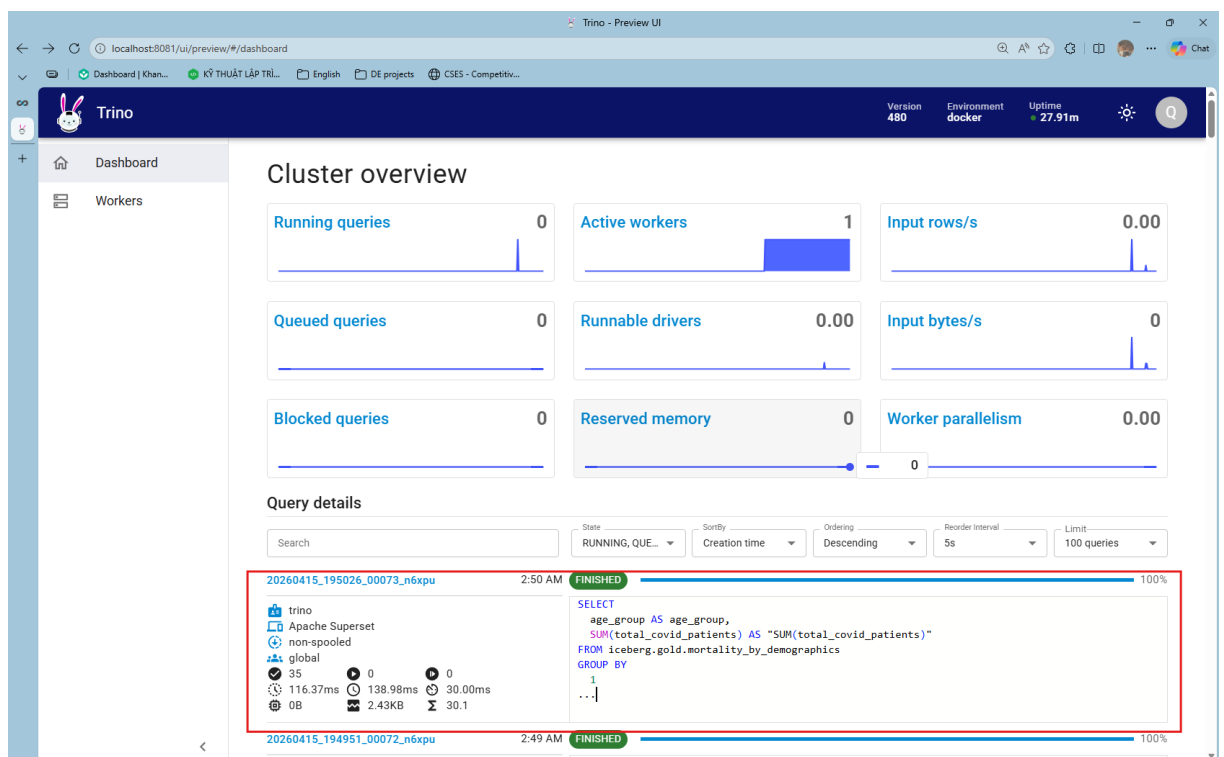
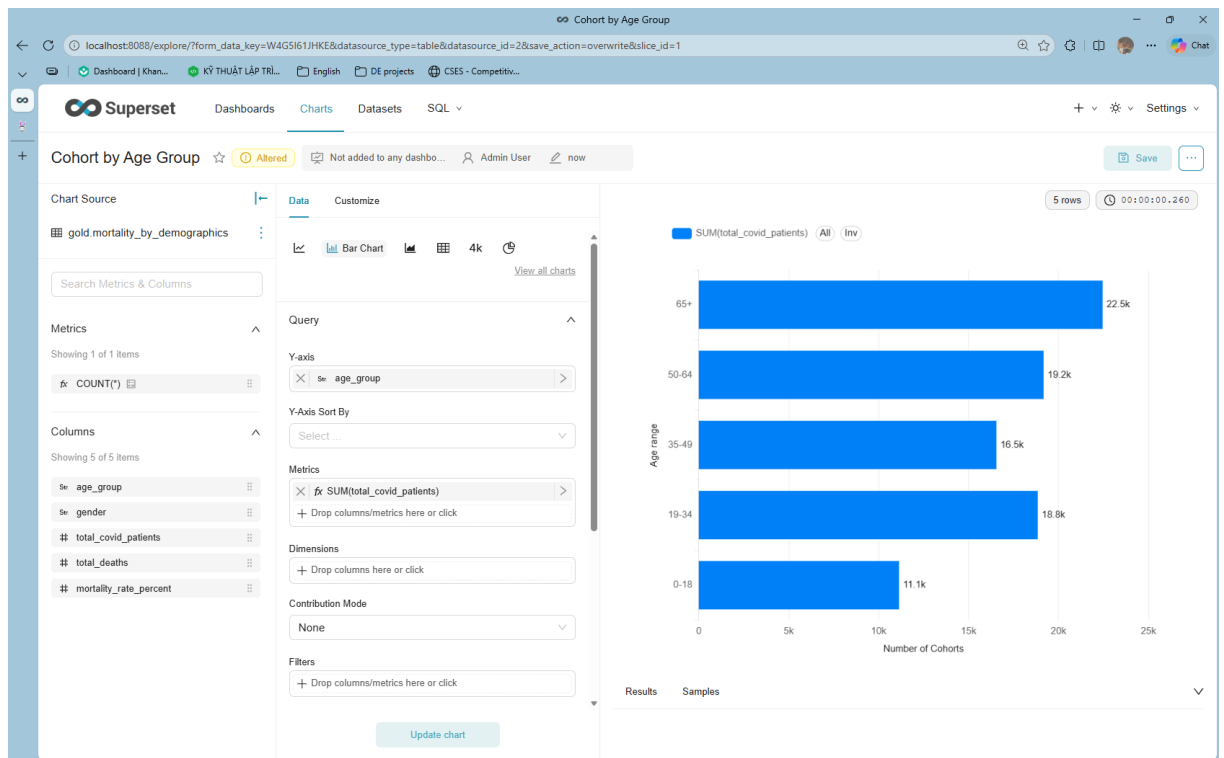


2.3.2. Biểu đồ Cohort by Age Group

Dữ liệu cho biểu đồ Cohort by Age Group được lấy từ dataset gold.mortality_by_demographics. Thuộc tính age_group được đặt ở trục tung và metrics

sử dụng là tổng các ca bệnh nhân mắc bệnh covid bao gồm cả nam và nữ. Biểu đồ này sử dụng kiểu Bar chart để thể hiện sự cao thấp đối với các mắc bệnh trong từng nhóm tuổi. Biểu đồ này có thể được dùng để trả lời các câu hỏi như:

- Nhóm tuổi nào có quy mô bệnh nhân mắc bệnh lớn nhất ?
- Chênh lệch quy mô giữa các nhóm tuổi là bao nhiêu ?
- Nhóm tuổi nào cần ưu tiên nguồn lực theo số lượng bệnh nhân ?



Khi ta thiết lập columns đặt vào data, chọn thêm các trường khác như filter hay limit và bấm Update chart thì trong Web UI của Trino sẽ chạy câu truy vấn tương ứng để xuất ra dữ liệu cho biểu đồ mà ta thấy trên Superset. Mọi câu truy vấn Update Chart hay truy vấn thông thường, Trino đều lưu lại để Coordinator có thể giám sát lịch sử.

2.3.3. Biểu đồ Mortality by Age and Gender

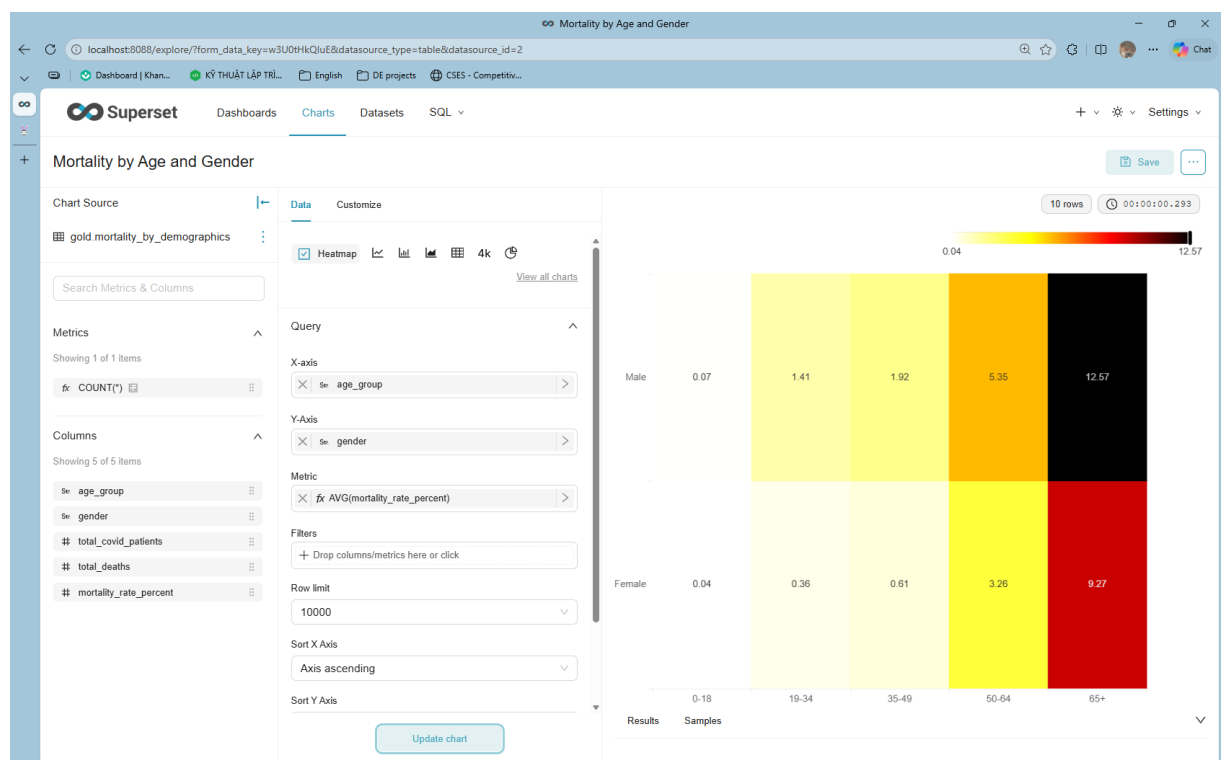
Dữ liệu cho Biểu đồ Mortality by Age and Gender được lấy từ dataset gold.mortality_by_demographics.

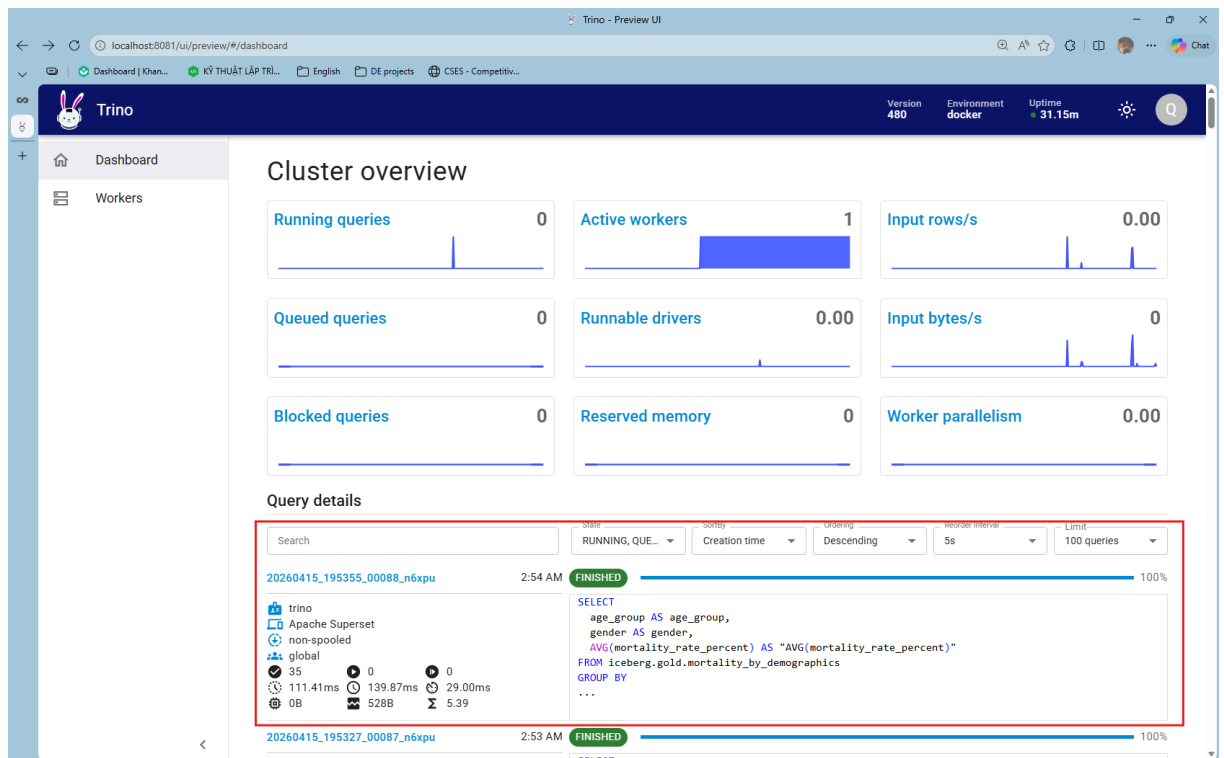
X-axis: age_group

Y-axis: gender

Metrics: AVG(mortality_by_rate).

Biểu đồ nhiệt này thể hiện mối tương quan mắc Covid giữa các nhóm tuổi theo 2 phái nam và nữ. Ta có thể biết được mức độ nghiêm trọng đối với từng nhóm tuổi theo giới tính để từ đó xác định xem nên tập trung nguồn lực nào để lập chiến dịch rà soát chữa bệnh kịp thời. Ngoài ra, biểu đồ này còn có thể dùng để trả lời các câu hỏi như “Nhóm tuổi nào có tỷ lệ tử vong cao nhất ?” hay “Trong cùng nhóm tuổi, khác biệt giữa nam và nữ ra sao ?”.



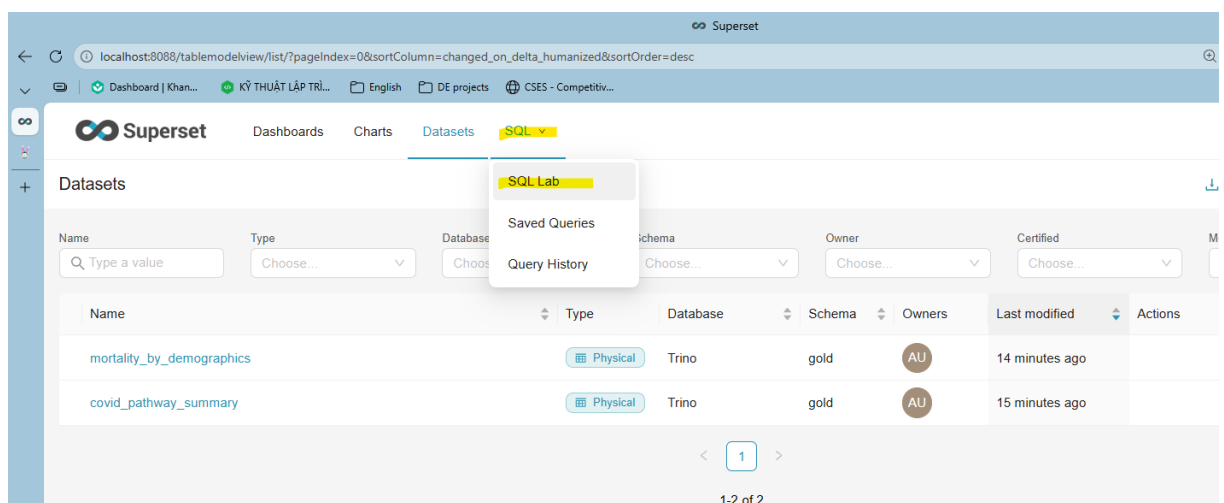


2.3.4. Biểu đồ COVID Pathway Distribution

Nhóm sử dụng gold.covid_pathway_summary để biểu đồ COVID Pathway Distribution để phân tích tỉ lệ phần trăm các ca điều trị được diễn ra như thế nào nhưng trong quá trình làm thì thấy dựa trên bảng gốc thì không thể thực hiện được do:

- Bảng gốc là dạng wide, chỉ 1 dòng và nhiều cột số đo
- Pie chart trong Superset cần dữ liệu dạng long tức là mỗi dòng ứng một nhóm và một giá trị.
- Virtual Dataset giúp chuyển wide → long để Superset hiểu đúng cấu trúc category/value.

Để tạo Virtual Dataset, ta vào SQL > SQL Lab



Truy vấn câu lệnh sau để hợp các bảng lại.

```
SELECT 'recovered' AS pathway, recovered_cases AS value
FROM iceberg.gold.covid_pathway_summary
UNION ALL
SELECT 'home_isolation' AS pathway, home_isolation_cases AS value
FROM iceberg.gold.covid_pathway_summary
UNION ALL
SELECT 'hospitalized' AS pathway, hospitalized_cases AS value
FROM iceberg.gold.covid_pathway_summary
UNION ALL
SELECT 'ventilation' AS pathway, ventilation_cases AS value
FROM iceberg.gold.covid_pathway_summary
UNION ALL
SELECT 'death' AS pathway, death_cases AS value
FROM iceberg.gold.covid_pathway_summary
UNION ALL
SELECT 'icu' AS pathway, icu_cases AS value
FROM iceberg.gold.covid_pathway_summary;
```

The screenshot shows the Superset SQL interface. On the left, the database is set to 'trino', schema to 'gold', and table to 'covid_pathway_summary'. The table schema is displayed, showing columns like 'latest_partition', 'total_covid_cases', 'home_isolation_cases', 'hospitalized_cases', 'icu_cases', 'ventilation_cases', 'recovered_cases', and 'death_cases'. The main area shows the SQL query being executed, which is a UNION ALL of SELECT statements for each pathway. The query is run, and the results are displayed in a table with 6 rows. The results table has columns 'pathway' and 'value'.

	pathway	value
1	hospitalized	26978
2	home_isolation	61188
3	ventilation	18175
4	recovered	84525
5	death	3641
6	icu	0

Ấn Save > Đặt tên là gold.covid_pathway_distribution_vds > Save dataset

Save query



Name

gold.covid_pathway_distribution_vds

Description

Cancel

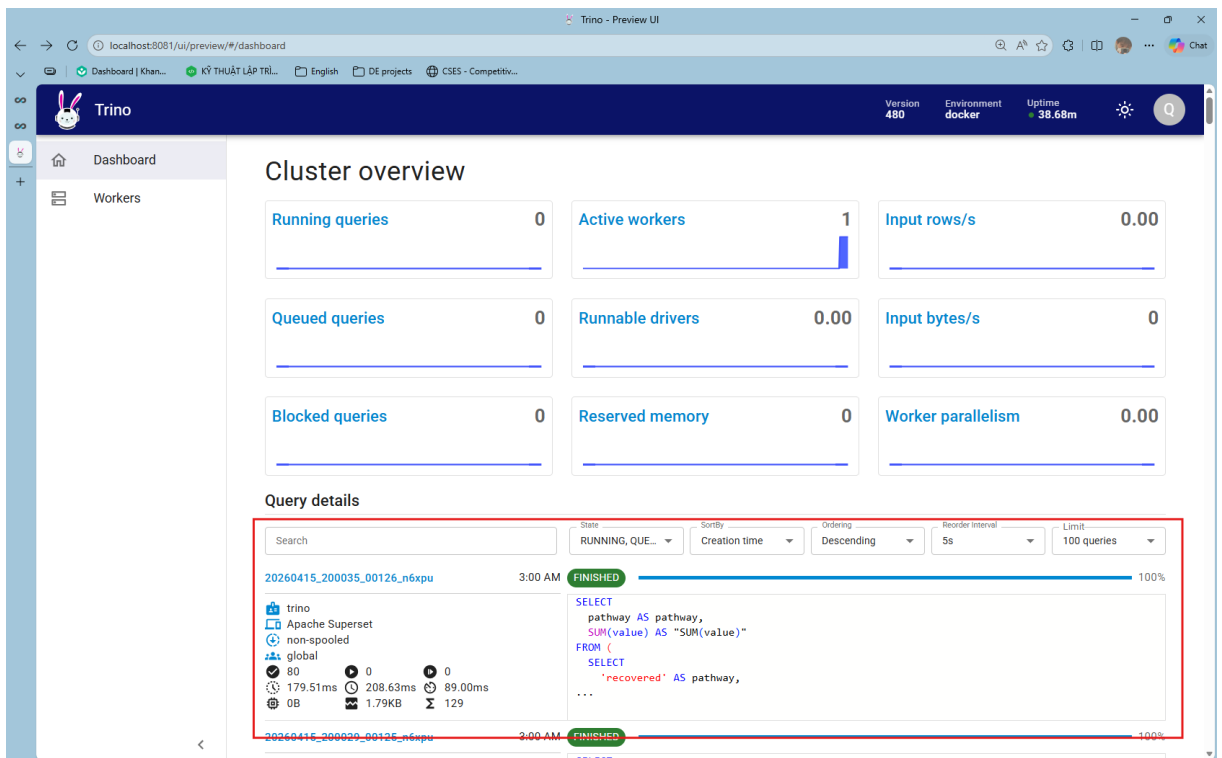
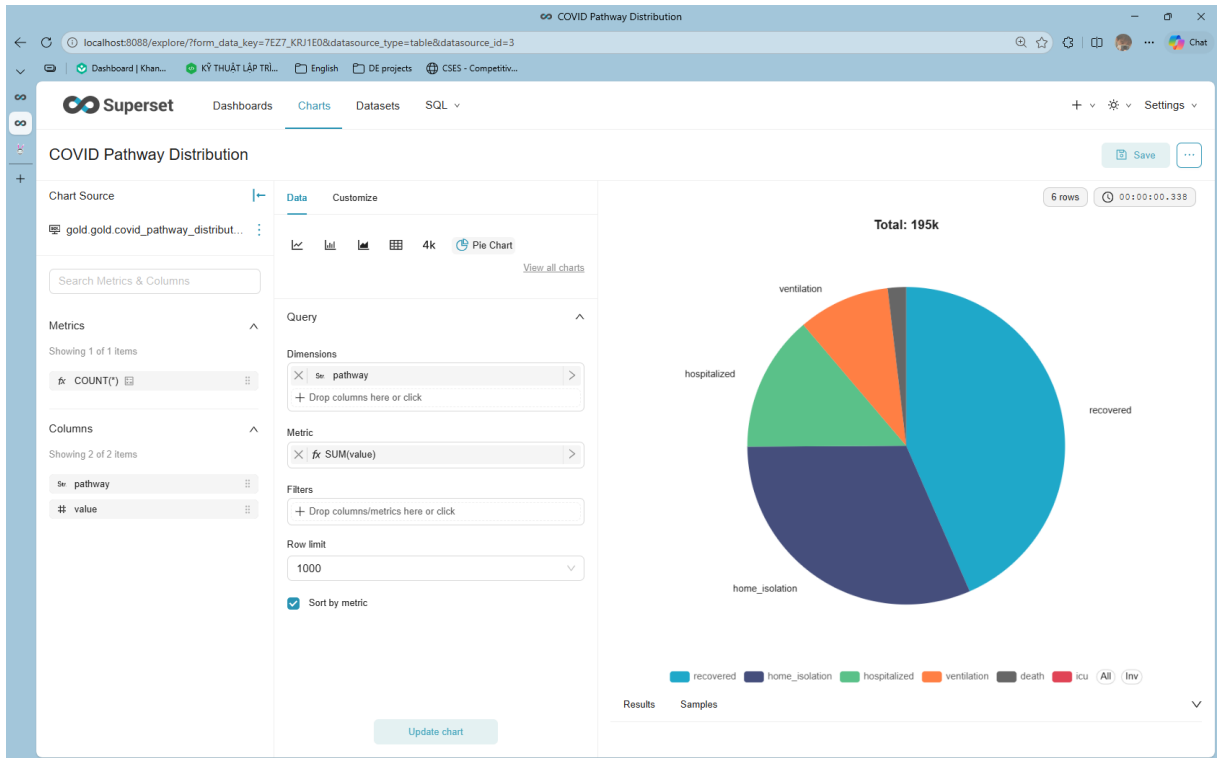
Save

The screenshot shows the Superset web interface. The 'Datasets' tab is selected. A table lists three datasets. The first dataset, 'gold.covid_pathway_distribution_vds', is highlighted and its type is 'Virtual'. The other two datasets, 'mortality_by_demographics' and 'covid_pathway_summary', are of type 'Physical'. All datasets are from the 'Trino' database and 'gold' schema, owned by 'AU'.

Name	Type	Database	Schema	Owners	Last modified	Actions
gold.covid_pathway_distribution_vds	Virtual	Trino	gold	AU	5 seconds ago	
mortality_by_demographics	Physical	Trino	gold	AU	18 minutes ago	
covid_pathway_summary	Physical	Trino	gold	AU	19 minutes ago	

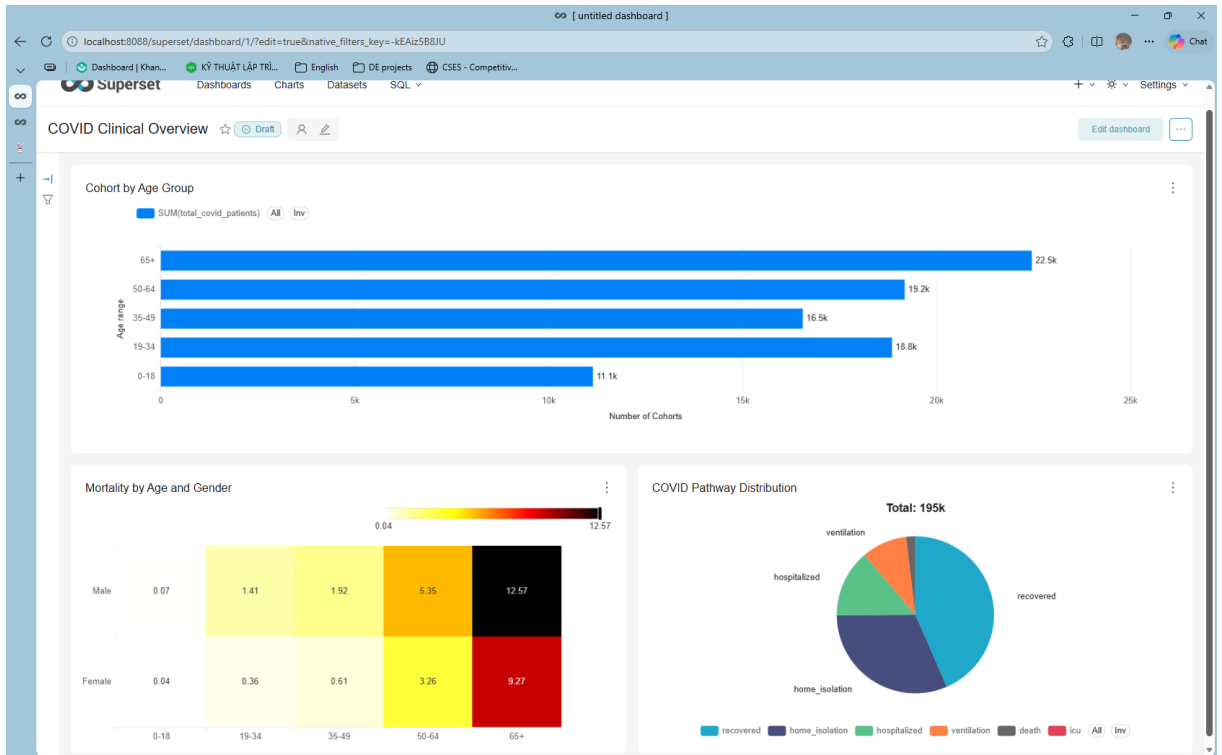
Đã tạo xong Virtual Dataset.

Biểu đồ tròn này dùng pathway làm dimension và metric là SUM(value). Dataset được sử dụng là Virtual dataset vừa tạo. Biểu đồ này có thể dùng để trả lời các câu hỏi tỷ trọng các trạng thái điều trị/kết cục hiện tại là như thế nào, Tỷ trọng các trạng thái nặng (icu, ventilation, death) đang ở mức nào.



2.3.5. Dashboard hoàn chỉnh

Sau khi đã trực quan các biểu đồ trên xong, nhóm đã ghép lại chung vào một dashboard để có một cái nhìn tổng thể. Đó là đối với những bệnh nhân thuộc nhóm tuổi càng cao thì sẽ có tỉ lệ tử vong cao hơn, điều này tăng một cách tuyến tính khi xét trên trục tuổi. Ngoài ra, khoảng 43% trong tổng số 195.000 bệnh nhân được chữa khỏi và khoảng 31% bệnh nhân thực hiện cách ly tại gia. Nhờ vào đó ta có thể tập trung vào chữa trị các ca bệnh nặng hơn thuộc về những người có độ tuổi cao.



CHƯƠNG 3: KẾT LUẬN

3.1. Kết quả đạt được

Dự án đã hoàn thành các mục tiêu đề ra ban đầu, xây dựng được một hệ thống Data Lakehouse hiện đại phục vụ phân tích dữ liệu y tế. Các kết quả cụ thể bao gồm:

Xây dựng thành công kiến trúc Medallion: Triển khai luồng dữ liệu chuẩn hóa qua 3 tầng: Bronze (lưu trữ dữ liệu thô), Silver (làm sạch và chuẩn hóa), và Gold (tổng hợp chỉ số phân tích) dựa trên sự kết hợp giữa Apache Spark và định dạng bảng Apache Iceberg.

Tự động hóa luồng dữ liệu (ETL/ELT): Thiết lập quy trình nạp dữ liệu tự động từ nguồn dữ liệu lâm sàng (Synthea COVID-19) vào hồ dữ liệu, xử lý thành công hơn 4.4 triệu bản ghi, đảm bảo tính toàn vẹn và nhất quán của dữ liệu.

Phân tích sâu các chỉ số dịch tễ học: Tại tầng Gold, hệ thống đã thực hiện được 5 bài toán phân tích quan trọng:

- Tỷ lệ tử vong theo nhân khẩu học (nhóm tuổi, giới tính).

- Lộ trình điều trị lâm sàng (từ cách ly tại nhà đến ICU và sử dụng máy thở).

- Phân tích triệu chứng và biến chứng (Sepsis, ARDS, suy tim...).

- Đánh giá gánh nặng y tế thông qua chỉ số thời gian lưu trú trung bình (ALOS).

Hệ thống điều phối và truy vấn mạnh mẽ: Sử dụng Apache Airflow để quản lý lịch trình chạy job và Trino SQL Engine để hỗ trợ truy vấn báo cáo tức thời với hiệu suất cao trên nền tảng lưu trữ MinIO (S3).

3.2. Hạn chế và thách thức

Bên cạnh những kết quả đạt được, dự án vẫn còn một số điểm hạn chế và khó khăn cần đối mặt:

Yêu cầu tài nguyên hệ thống cao: Việc vận hành đồng thời Spark, Trino, Airflow và Hive Metastore đòi hỏi cấu hình phần cứng (RAM/CPU) rất lớn, gây khó khăn khi triển khai trên các môi trường máy tính cá nhân.

Độ phức tạp trong quản lý hạ tầng: Gặp thách thức trong việc điều phối thứ tự khởi tạo các dịch vụ (race conditions), đặc biệt là việc đồng bộ hóa Database cho Airflow và Metastore khi khởi động bằng Docker Compose.

Xử lý dữ liệu lâm sàng: Dữ liệu y tế có độ nhiễu cao và thiếu nhất quán trong cách mô tả triệu chứng, đòi hỏi các thuật toán xử lý chuỗi (RegEx) phức tạp và tốn nhiều công sức chuẩn hóa tại tầng Silver.

Tối ưu hóa hiệu năng: Việc tinh chỉnh cấu hình Spark để đạt hiệu suất tối ưu khi xử lý dữ liệu quy mô lớn trong môi trường container vẫn còn nhiều dư địa để cải thiện.

3.3. Hướng phát triển tương lai

Để hoàn thiện và mở rộng giá trị của hệ thống, các hướng phát triển tiếp theo bao gồm:

Ứng dụng học máy (AI/ML): Sử dụng thư viện Spark MLlib để xây dựng các mô hình dự báo rủi ro tử vong hoặc dự đoán nhu cầu trang thiết bị y tế (máy thở, giường bệnh) dựa trên các triệu chứng lâm sàng sớm của bệnh nhân.

Chuyển đổi sang Kubernetes (K8s): Di chuyển hệ thống từ Docker Compose sang môi trường K8s để đảm bảo khả năng mở rộng linh hoạt (Auto-scaling) và tính sẵn sàng cao (High Availability) cho môi trường sản xuất.

Xử lý dữ liệu thời gian thực (Streaming): Tích hợp Apache Kafka để hỗ trợ nạp dữ liệu theo thời gian thực từ các hệ thống quản lý bệnh viện (HIS/EMR), thay vì chỉ nạp theo lô (Batch) như hiện tại.

TÀI LIỆU THAM KHẢO

- [1] E. Capriolo, D. Wampler, and J. Rutherglen, *Programming Hive*. Sebastopol, CA: O'Reilly Media, 2012.
- [2] M. Fuller, M. Moser, và M. Traverso, *Trino: The Definitive Guide*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2023.
- [3] T. Shiran, J. Hughes, và A. Merced, *Apache Iceberg: The Definitive Guide*. Sebastopol, CA: O'Reilly Media, 2024.
- [4] A. Kaplan và A. Kara, *The Data Lakehouse For Dummies*, 2nd Databricks Special Edition. Hoboken, NJ: John Wiley & Sons, 2026
- [5] HuaweiBlogTeam, Huawei Blog, Decoupled Storage-Compute Architecture: The New De facto Standard for Distributed Databases, 2023, Truy cập tại: <https://blog.huawei.com/en/post/2023/11/30/decoupled-storage-compute-architecture-standard-distributed-databases>
- [6] Databricks, Databricks Documentation, What is the medallion lakehouse architecture?, 2025, Truy cập tại <https://docs.databricks.com/aws/en/lakehouse/medallion>
- [7] Apache Software Foundation, *Apache Spark Documentation*, 2025. Truy cập tại: <https://spark.apache.org>.
- [8] Apache Software Foundation, *Apache Hive Documentation*, 2025. Truy cập tại: <https://hive.apache.org>.
- [9] Apache Software Foundation, *Apache Airflow Documentation*, 2025. Truy cập tại: <https://airflow.apache.org>.
- [10] Docker Inc., *Docker Documentation*, 2025. Truy cập tại: <https://www.docker.com>.
- [11] MinIO Inc., *MinIO Documentation*, 2025. Truy cập tại: <https://min.io/docs/minio/container/index.html>.
- [12] Apache Software Foundation, *Apache Superset Documentation*, 2025. Truy cập tại: <https://superset.apache.org/docs/intro>.